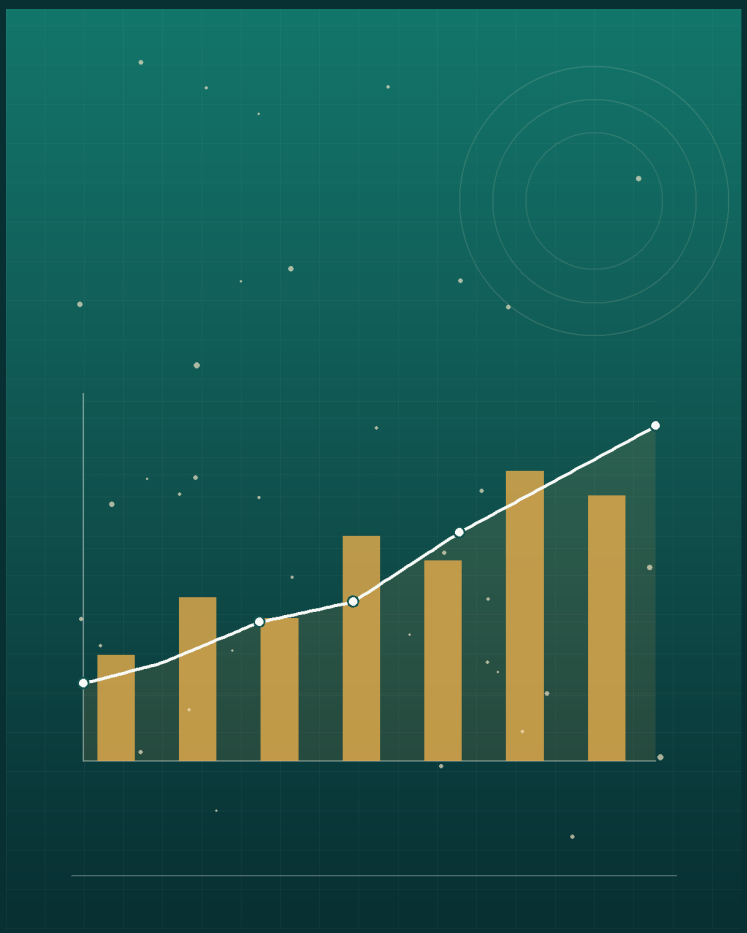
LE GUIDE PRATIQUE · ÈDITION 2026

Analyse de données avec R et Quarto

Livre de formation progressive pour débutants

Des premières lignes de code à l'analyse, aux graphiques et aux rapports professionnels.



Samba Diop

samba.diop3@univ-thies.sn

Table of contents

1 Votre parcours commence ici **1**
1.1 Bienvenue dans le livre . . . 1
1.2 Ce que vous saurez faire . . . 1
1.3 Pour qui . . . 1
1.4 Parcours pédagogique . . . 2
1.5 Comment apprendre . . . 2
1.6 Prérequis techniques . . . 3
1.7 Entrer dans le premier chapitre . . . 3

I Partie I — Introduction **7**

2 Introduction à l'analyse de données **9**
2.1 Introduction . . . 9
2.2 Objectifs pédagogiques . . . 9
2.3 Prérequis . . . 9
2.4 Qu'est-ce que l'analyse de données ? . . . 9
2.5 Pourquoi apprendre R ? . . . 10
2.6 Présentation de R, RStudio et Quarto . . . 10
2.7 Installation et configuration . . . 10
2.8 Premier programme en R . . . 11
2.9 Organisation d'un projet d'analyse . . . 12
2.10 Démonstration : un mini-parcours . . . 12
2.11 Erreurs fréquentes . . . 13
2.12 Exercices pratiques . . . 13
2.13 Corrections . . . 14
2.14 Points clés . . . 14
2.15 Ressources complémentaires . . . 14
2.16 Suite . . . 14

II Partie II — Fondamentaux de R **15**

3 Les fondamentaux de R **17**
3.1 Introduction . . . 17
3.2 Objectifs pédagogiques . . . 17
3.3 Prérequis . . . 17
3.4 Syntaxe générale de R . . . 17
3.5 Variables et affectation . . . 18
3.6 Types de données . . . 18
3.7 Opérateurs . . . 19
3.8 Vecteurs . . . 19

3.9 Matrices et tableaux . . . 20
3.10 Listes . . . 21
3.11 Data frames et tibbles . . . 21
3.12 Indexation et sélection . . . 22
3.13 Conditions : `if`, `else`, `ifelse()` . . . 22
3.14 Boucles `for` et `while` . . . 22
3.15 Fonctions personnalisées . . . 23
3.16 Gestion des erreurs . . . 23
3.17 Packages et bibliothèques . . . 24
3.18 Bonnes pratiques de programmation . . . 24
3.19 Démonstration complète . . . 24
3.20 Erreurs fréquentes . . . 25
3.21 Exercices . . . 25
3.22 Corrections . . . 25
3.23 Points clés . . . 26
3.24 Ressources . . . 26
3.25 Suite . . . 26

III Partie III — Acquisition des données 27

4 Acquisition et importation des données 29
4.1 Introduction . . . 29
4.2 Objectifs pédagogiques . . . 29
4.3 Prérequis . . . 29
4.4 Comprendre les sources de données . . . 29
4.5 Importer des fichiers CSV . . . 30
4.6 Lire des fichiers Excel . . . 30
4.7 Importer des fichiers texte . . . 31
4.8 Exporter des données . . . 31
4.9 Jeux de données intégrés . . . 31
4.10 Chemins de fichiers et dossiers de travail . . . 32
4.11 Principes d'accès aux bases de données . . . 32
4.12 API et données Web (principes hors ligne) . . . 32
4.13 Vérifier la qualité des données importées . . . 33
4.14 Démonstration complète . . . 34
4.15 Erreurs fréquentes . . . 34
4.16 Exercices . . . 35
4.17 Corrections . . . 35
4.18 Points clés . . . 36
4.19 Ressources . . . 36
4.20 Suite . . . 36

IV Partie IV — Préparation des données 37

5 Nettoyage et préparation des données 39
5.1 Introduction . . . 39
5.2 Objectifs pédagogiques . . . 39
5.3 Prérequis . . . 39
5.4 Qualité des données . . . 39
5.5 Examiner la structure . . . 40
5.6 Renommer, sélectionner, filtrer, trier . . . 40
5.7 Créer des variables et modifier les types . . . 40
5.8 Traiter les valeurs manquantes . . . 41

5.9 Doublons . . . 41
5.10 Valeurs aberrantes . . . 41
5.11 Chaînes de caractères (`stringr`) . . . 42
5.12 Dates (`lubridate`) . . . 42
5.13 Regrouper et résumer . . . 43
5.14 Fusionner plusieurs tables . . . 43
5.15 Large long (`tidyr`) . . . 44
5.16 Pipeline de préparation reproductible . . . 44
5.17 Démonstration complète . . . 45
5.18 Erreurs fréquentes . . . 45
5.19 Exercices . . . 45
5.20 Corrections . . . 46
5.21 Points clés . . . 47
5.22 Ressources . . . 47
5.23 Suite . . . 47

V Partie V — Statistiques descriptives 49

6 Exploration et statistiques descriptives 51
6.1 Introduction . . . 51
6.2 Objectifs pédagogiques . . . 51
6.3 Prérequis . . . 51
6.4 Analyse exploratoire (EDA) . . . 51
6.5 Effectifs et fréquences . . . 52
6.6 Moyenne, médiane et mode . . . 52
6.7 Minimum, maximum, étendue . . . 53
6.8 Quartiles et quantiles . . . 53
6.9 Variance et écart-type . . . 53
6.10 Distribution des variables . . . 53
6.11 Comparaison de groupes . . . 54
6.12 Tableaux croisés . . . 54
6.13 Corrélation entre variables . . . 55
6.14 Interprétation en langage courant . . . 55
6.15 Limites et erreurs d'interprétation . . . 55
6.16 Démonstration complète . . . 55
6.17 Exercices . . . 56
6.18 Corrections . . . 56
6.19 Points clés . . . 57
6.20 Ressources . . . 57
6.21 Suite . . . 57

VI Partie VI — Visualisation 59

7 Visualisation des données 61
7.1 Introduction . . . 61
7.2 Objectifs pédagogiques . . . 61
7.3 Prérequis . . . 61
7.4 Pourquoi visualiser ? . . . 61
7.5 Principes de lisibilité . . . 62
7.6 Diagrammes en barres . . . 62
7.7 Histogrammes . . . 63
7.8 Boîtes à moustaches . . . 64
7.9 Graphiques de dispersion . . . 65

7.10 Graphiques linéaires . . . 66
7.11 Comparaison de plusieurs groupes . . . 67
7.12 Personnalisation : titres, axes, légendes . . . 68
7.13 Couleurs . . . 69
7.14 Facettes . . . 70
7.15 Exportation . . . 71
7.16 Grammaire ggplot2 (rappel) . . . 71
7.17 Erreurs courantes . . . 71
7.18 Mini-collection pour un rapport . . . 72
7.19 Exercices . . . 73
7.20 Corrections . . . 73
7.21 Points clés . . . 77
7.22 Ressources . . . 77
7.23 Suite . . . 77

VII Partie VII — Modélisation 79

8 Analyse statistique et modélisation 81
8.1 Introduction . . . 81
8.2 Objectifs pédagogiques . . . 81
8.3 Prérequis . . . 81
8.4 Population et échantillon . . . 81
8.5 Variables quantitatives et qualitatives . . . 82
8.6 Échantillonnage (principes) . . . 82
8.7 Probabilités et distributions de base . . . 82
8.8 Intervalles de confiance . . . 82
8.9 Tests d'hypothèses (introduction) . . . 83
8.10 Comparaison de moyennes . . . 83
8.11 Corrélation et association . . . 83
8.12 Régression linéaire simple . . . 84
8.13 Régression linéaire multiple . . . 85
8.14 Hypothèses et résidus . . . 86
8.15 Évaluation des performances . . . 87
8.16 Limites : corrélation versus causalité . . . 87
8.17 Démonstration complète . . . 88
8.18 Exercices . . . 88
8.19 Corrections . . . 89
8.20 Points clés . . . 90
8.21 Ressources . . . 90
8.22 Suite . . . 90

VIII Partie VIII — Rapports 91

9 Rapports et communication des résultats 93
9.1 Introduction . . . 93
9.2 Objectifs pédagogiques . . . 93
9.3 Prérequis . . . 93
9.4 Documents reproductibles . . . 93
9.5 Présentation de Quarto . . . 94
9.6 Structure d'un fichier `.qmd` . . . 94
9.7 Insertion de code R . . . 94
9.8 Affichage des résultats et graphiques . . . 94
9.9 Paramètres d'exécution . . . 95

9.10 Génération HTML, PDF, DOCX . . . 95
9.11 Personnalisation de la mise en page . . . 96
9.12 Bibliographie et citations . . . 96
9.13 Rapports statistiques automatisés . . . 96
9.14 R Markdown vs Quarto . . . 96
9.15 Publication et partage . . . 97
9.16 Démonstration : mini-rapport inline . . . 97
9.17 Erreurs fréquentes . . . 97
9.18 Exercices . . . 97
9.19 Corrections (indicatif) . . . 98
9.20 Points clés . . . 98
9.21 Ressources . . . 98
9.22 Suite . . . 98

IX Partie IX — Applications Shiny 99

10 Applications interactives avec Shiny 101
10.1 Introduction . . . 101
10.2 Objectifs pédagogiques . . . 101
10.3 Prérequis . . . 101
10.4 Présentation de Shiny . . . 101
10.5 Structure d'une application . . . 101
10.6 Interface utilisateur et serveur . . . 102
10.7 Entrées utilisateur . . . 102
10.8 Sorties . . . 102
10.9 Réactivité . . . 102
10.10 Filtres et graphiques interactifs . . . 103
10.11 Validation des entrées . . . 103
10.12 Organisation du code . . . 103
10.13 Lancement local . . . 103
10.14 Tableau de bord simple — application du projet . . . 103
10.15 Introduction au déploiement . . . 103
10.16 Exemple minimal exécutable (concept) . . . 104
10.17 Erreurs fréquentes . . . 104
10.18 Exercices . . . 104
10.19 Corrections (indications) . . . 105
10.20 Points clés . . . 105
10.21 Ressources . . . 105
10.22 Suite . . . 105

X Partie X — Exercices et projets 107

11 Projets de synthèse et évaluations 109
11.1 Introduction . . . 109
11.2 Projet 1 — Analyse des ventes . . . 109
11.3 Projet 2 — Données démographiques . . . 111
11.4 Projet 3 — Météo . . . 112
11.5 Projet 4 — Régression . . . 113
11.6 Projet 5 — Tableau de bord Shiny . . . 114
11.7 Projet final — Étude complète . . . 115
11.8 Quiz de révision rapide . . . 116
11.9 Points clés . . . 116
11.10 Suite . . . 117

12 Exercices de synthèse transversaux **119**
12.1 Exercice S1 — Pipeline ventes (niveaux 1–3) . . . 119
12.2 Exercice S2 — Qualité démographie . . . 119
12.3 Exercice S3 — Mini-rapport Quarto . . . 119
12.4 Exercice S4 — Shiny . . . 119
12.5 Quiz final (10 questions) . . . 119

13 Corrections des exercices de synthèse **121**
13.1 Correction S1 . . . 121
13.2 Correction S2 . . . 122
13.3 Correction S3 (contenu attendu) . . . 123
13.4 Réponses du quiz final . . . 123

XI Partie XI — Références et annexes **125**

14 Références, annexes et navigation **127**
14.1 Introduction . . . 127
14.2 Guide des erreurs fréquentes . . . 127
14.3 Tableau des fonctions R essentielles . . . 127
14.4 Publication GitHub Pages (facultatif) . . . 128
14.5 Mise à jour du projet . . . 128
14.6 Index de navigation . . . 128
14.7 Références officielles . . . 128

Références bibliographiques **129**

Appendices **131**

A Glossaire **131**

B Aide-mémoires **133**
B.1 Manipulation de données (dplyr / tidyr) . . . 133
B.2 Import / export . . . 133
B.3 ggplot2 . . . 133
B.4 Statistiques . . . 134
B.5 Quarto (CLI) . . . 134
B.6 Shiny (minimal) . . . 134
B.7 stringr / lubridate (extraits) . . . 134

Chapter 1

Votre parcours commence ici

R · STATISTIQUE · VISUALISATION · QUARTO

1.1 Bienvenue dans le livre

Un parcours complet pour apprendre à transformer des données brutes en analyses claires, graphiques utiles et rapports reproductibles.

Commencer la formation Explorer le parcours

1.2 Ce que vous saurez faire

À la fin de la formation, vous saurez prendre un jeu de données brut, le préparer, l'analyser, le visualiser et communiquer vos conclusions.

1. Installer et utiliser R, RStudio et Quarto
2. Programmer avec les structures essentielles de R
3. Importer, nettoyer et transformer des données
4. Calculer et interpréter des statistiques descriptives
5. Créer des visualisations avec `ggplot2`
6. Appliquer corrélation, régression et tests de base
7. Produire des rapports HTML, PDF et Word
8. Construire une application interactive Shiny
9. Mener un projet d'analyse de bout en bout

1.3 Pour qui

Aucune connaissance préalable de R n'est requise. Les notions statistiques sont introduites au fur et à mesure.

- Débutants absolus en programmation
- Étudiants et universitaires
- Professionnels en montée en compétences data
- Curieux de statistique appliquée

1.4 Parcours pédagogique

Onze parties progressives, de l'installation des outils jusqu'aux projets de synthèse.

1. **Introduction** — Comprendre le métier et installer les outils
2. **Fondamentaux de R** — Syntaxe, types et objets
3. **Acquisition** — Importer CSV, Excel et textes
4. **Préparation** — Nettoyer, joindre, transformer
5. **Statistiques descriptives** — Résumer et explorer
6. **Visualisation** — Communiquer par les graphiques
7. **Modélisation** — Inférence, tests et régression
8. **Rapports** — Documenter avec Quarto
9. **Shiny** — Applications interactives
10. **Projets** — Consolider par la pratique
11. **Annexes** — Glossaire et aide-mémoires

1.5 Comment apprendre

Lisez, exécutez, modifiez. L'expérimentation vaut mieux que la lecture passive.

1. Installez R, Quarto et les packages (voir le README et Section 2.7).
2. Suivez les chapitres dans l'ordre.
3. Exécutez le code dans RStudio.
4. Faites les exercices avant les corrections.
5. Réalisez les projets de la Partie X.

> **Conseil**
>
> Changez un paramètre, relancez, observez. C'est ainsi que les notions deviennent des réflexes.

1.6 Prérequis techniques

Un ordinateur suffit. Après l'installation initiale, le livre peut être reconstruit hors ligne.

- Windows, macOS ou Linux
- Connexion Internet pour installer les packages
- Jeux de données fournis dans `data/` ou intégrés à R

1.7 Entrer dans le premier chapitre

Ouvrez le parcours par l'introduction à l'analyse de données, puis installez vos outils.

Commencer →

Analyse de données avec R et Quarto

Part I

Partie I — Introduction

Chapter 2

Introduction à l'analyse de données

2.1 Introduction

L'**analyse de données** consiste à transformer des informations brutes en connaissances utiles pour décider, comprendre ou communiquer. Que vous étudiiez des ventes, des notes d'examen ou des températures, le raisonnement reste le même : collecter, préparer, explorer, modéliser et présenter.

Ce chapitre pose les bases : à quoi sert l'analyse de données, pourquoi R est un excellent choix, comment installer les outils, puis écrire et organiser votre premier programme.

2.2 Objectifs pédagogiques

À la fin de ce chapitre, vous serez capable de :

- Définir l'analyse de données et ses étapes principales
- Expliquer les atouts de R pour la statistique et la science des données
- Installer et vérifier R, Quarto et RStudio
- Exécuter un premier script R
- Organiser un projet d'analyse de façon reproductible

2.3 Prérequis

Aucun prérequis en programmation. Une familiarité basique avec l'ordinateur (fichiers, dossiers, installation de logiciels) suffit.

2.4 Qu'est-ce que l'analyse de données ?

2.4.1 Une démarche en étapes

1. **Question** — Que veut-on savoir ? (ex. : « Les ventes ont-elles augmenté au 2 semestre ? »)
2. **Données** — Où sont les informations ? (fichiers, bases, enquêtes…)
3. **Préparation** — Nettoyer, filtrer, joindre, corriger
4. **Exploration** — Résumer, visualiser, repérer des patterns
5. **Analyse** — Tests, modèles, incertitude
6. **Communication** — Rapport, tableau de bord, présentation

Vocabulaire essentiel

- **Variable** : une caractéristique mesurée (âge, prix, région).
- **Observation** : une ligne, un individu, une mesure.
- **Jeu de données** (*dataset*) : tableau structuré de variables et d'observations.
- **Reproductibilité** : pouvoir refaire l'analyse et obtenir le même résultat.

2.4.2 Domaines d'application

R est utilisé en biologie, économie, marketing, épidémiologie, sciences sociales, finance, éducation, environnement, etc. Le point commun : des données tabulaires et un besoin de transparence méthodologique.

2.5 Pourquoi apprendre R ?

R est un langage libre, spécialisé dans la statistique et la visualisation. Ses atouts :

Atout	Explication
Gratuit et open source	Pas de licence payante pour démarrer
Écosystème riche	Des milliers de packages (tidyverse, ggplot2, shiny…)
Qualité graphique	Graphiques publication-ready
Reproductibilité	Scripts + Quarto = analyses documentées
Communauté	Forums, livres, formations abondantes

R ou Python ?

Les deux sont excellents. R brille particulièrement pour la statistique, les rapports et la visualisation exploratoire. Ce livre se concentre sur R ; les compétences (logique, données, visualisation) se transfèrent ensuite.

2.6 Présentation de R, RStudio et Quarto

- **R** : le moteur (langage + environnement statistique).
- **RStudio Desktop** (Posit) : l'interface recommandée (éditeur, console, graphiques, fichiers).
- **Quarto** : système de publication pour écrire ce livre et vos rapports (HTML, PDF, Word).

RStudio n'est pas obligatoire : vous pouvez utiliser VS Code, Positron ou la console R. Pour débuter, RStudio reste le choix le plus confortable.

2.7 Installation et configuration

2.7.1 1. Installer R

1. Téléchargez R depuis https://cran.r-project.org/
2. Choisissez votre système (Windows, macOS, Linux)
3. Installez en suivant l'assistant (options par défaut acceptables)

Vérification dans un terminal :

```
R --version
# ou
Rscript --version
```

2.7.2 2. Installer Quarto

Téléchargez Quarto depuis https://quarto.org/docs/get-started/ puis vérifiez :

```
quarto --version
```

Sous Windows, Quarto peut aussi être fourni avec RStudio (dossier `resources/app/bin/quarto`).

2.7.3 3. Installer RStudio Desktop (recommandé)

Téléchargez depuis https://posit.co/download/rstudio-desktop/.

2.7.4 4. Installer les packages du livre

Dans la console R :

```
source("R/packages_requis.R")
```

Ou manuellement :

```
install.packages(c(
  "tidyverse", "readxl", "broom", "shiny",
  "janitor", "gt", "scales", "writexl"
))
```

2.7.5 5. PDF : distribution LaTeX

Pour générer le PDF du livre :

```
install.packages("tinytex")
tinytex::install_tinytex()
```

> **Attention**
>
> L'installation de TinyTeX peut prendre plusieurs minutes et nécessite une connexion Internet. Le rendu HTML et DOCX ne l'exigent pas.

2.8 Premier programme en R

Ouvrez RStudio (ou une session R) et saisissez :

```
# Calcul simple
2 + 2
```

```
#> [1] 4
```

```
# Affectation : on stocke un résultat dans un objet
montant <- 19.90
quantite <- 3
total <- montant * quantite
total
```

```
#> [1] 59.7
```

```
# Afficher un message
message("Bienvenue dans l'analyse de données avec R !")
```

2.8.1 Anatomy d'une session

- `<-` : opérateur d'affectation (lire « reçoit »)
- `#` : commentaire (ignoré par R)
- La console exécute ligne par ligne ; un script `.R` enregistre le code

```
# Un vecteur de notes
notes <- c(12, 15, 9, 18, 14)
mean(notes)   # moyenne
```

```
#> [1] 13.6
```

```
sd(notes)     # écart-type
```

```
#> [1] 3.361547
```

```
summary(notes)
```

```
#>    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
#>     9.0    12.0    14.0    13.6    15.0    18.0
```

> **!** Erreur fréquente
>
> `=` fonctionne souvent comme `<-`, mais la convention pédagogique en R est `<-`. Évitez de confondre `==` (comparaison) et `<-` (affectation).

2.9 Organisation d'un projet d'analyse

2.9.1 Structure recommandée

```
mon-projet/
  data/           # données brutes et nettoyées
  R/              # scripts réutilisables
  reports/        # fichiers .qmd
  outputs/        # figures et tableaux exportés
  README.md
```

2.9.2 Bonnes pratiques dès le premier jour

1. **Un projet RStudio** (fichier `.Rproj`) ancre le répertoire de travail.
2. **Ne modifiez pas les données brutes** : gardez une copie intacte dans `data/brut/`.
3. **Tout le traitement dans des scripts** : pas de clics non documentés.
4. **Noms de fichiers explicites** : `nettoyer_ventes.R`, pas `script2.R`.

```
# Connaître le dossier de travail
getwd()

# Lister les fichiers de données
list.files("data")
```

Dans ce livre, les chemins sont relatifs au dossier `formation-r/`.

2.10 Démonstration : un mini-parcours

```r
# Jeu de données intégré : voitures (mtcars)
data(mtcars)
head(mtcars, 3)
```

```
#>                mpg cyl disp  hp drat    wt  qsec vs am gear carb
#> Mazda RX4     21.0   6  160 110 3.90 2.620 16.46  0  1    4    4
#> Mazda RX4 Wag 21.0   6  160 110 3.90 2.875 17.02  0  1    4    4
#> Datsun 710    22.8   4  108  93 3.85 2.320 18.61  1  1    4    1
```

```r
# Combien de voitures ? Combien de variables ?
dim(mtcars)
```

```
#> [1] 32 11
```

```r
# Consommation moyenne (miles per gallon)
mean(mtcars$mpg)
```

```
#> [1] 20.09062
```

Interprétation : `mtcars` contient 32 voitures et 11 variables. La moyenne de `mpg` donne une idée de la consommation typique dans cet échantillon historique.

2.11 Erreurs fréquentes

Problème	Cause probable	Solution
`object not found`	Nom mal orthographié ou objet non créé	Vérifier `ls()` et l'orthographe
`cannot open file`	Mauvais chemin	Vérifier `getwd()` et `list.files()`
Package manquant	Non installé	`install.packages("nom")` puis `library(nom)`
Console « bloquée »	`+` en attente	Fermer une parenthèse ou Esc

2.12 Exercices pratiques

2.12.1 Niveau 1 — Compréhension

1. Citez les six étapes d'une analyse de données.
2. Quelle différence entre R et RStudio ?
3. À quoi sert Quarto dans ce projet ?

2.12.2 Niveau 2 — Application

4. Calculez la somme et la moyenne du vecteur `c(10, 20, 30, 40)`.
5. Créez un objet `prenom` contenant votre prénom et affichez-le avec `message()`.

2.12.3 Niveau 3 — Autonomie

6. Explorez `iris` : nombre de lignes, de colonnes, noms des variables, moyenne de `Sepal.Length`.

2.13 Corrections

```r
v <- c(10, 20, 30, 40)
sum(v)
```

```
#> [1] 100
```

```r
mean(v)
```

```
#> [1] 25
```

```r
prenom <- "Alex"
message("Bonjour ", prenom)
```

```r
data(iris)
dim(iris)
```

```
#> [1] 150   5
```

```r
names(iris)
```

```
#> [1] "Sepal.Length" "Sepal.Width"  "Petal.Length" "Petal.Width"  "Species"
```

```r
mean(iris$Sepal.Length)
```

```
#> [1] 5.843333
```

2.14 Points clés

- L'analyse de données est une **démarche**, pas seulement du code.
- **R** calcule ; **RStudio** facilite ; **Quarto** documente et publie.
- Vérifiez les installations avec `Rscript --version` et `quarto --version`.
- Organisez vos fichiers dès le début pour la reproductibilité.

2.15 Ressources complémentaires

- Site officiel R : https://www.r-project.org/
- Documentation Quarto : https://quarto.org/
- *R for Data Science* (Wickham, Çetinkaya-Rundel, and Grolemund 2023)

2.16 Suite

Le prochain chapitre (Chapter 3) introduit la syntaxe, les types et les structures de données de R.

Part II

Partie II — Fondamentaux de R

Chapter 3

Les fondamentaux de R

3.1 Introduction

Ce chapitre pose le socle de la programmation en R : syntaxe, types, structures (vecteurs, matrices, listes, data frames), conditions, boucles, fonctions et packages. Maîtriser ces bases rend tout le reste — tidyverse, statistiques, graphiques — beaucoup plus fluide.

3.2 Objectifs pédagogiques

- Utiliser la syntaxe et l'affectation
- Distinguer les types atomic et les structures
- Indexer et sélectionner des éléments
- Écrire des conditions, boucles et fonctions
- Charger des packages et gérer les erreurs simples

3.3 Prérequis

Avoir installé R (Section 2.7) et exécuté un premier calcul (Section 2.8).

3.4 Syntaxe générale de R

```
# Commentaire
x <- 10          # affectation
y <- x + 5       # expression
print(y)         # affichage explicite
```

```
#> [1] 15
```

```
y                # affichage implicite en console
```

```
#> [1] 15
```

R est **sensible à la casse** : `Notes` ≠ `notes`. Les noms d'objets commencent par une lettre et peuvent contenir chiffres, `.` et `_`.

Vocabulaire

- **Objet** : toute valeur nommée en mémoire.
- **Fonction** : bloc réutilisable (`mean()`, `sum()`…).
- **Argument** : paramètre passé à une fonction.
- **Valeur de retour** : résultat produit par une fonction.

3.5 Variables et affectation

```
prix <- 12.5
quantite <- 4L          # L = entier (integer)
produit <- "Cahier"
disponible <- TRUE
```

Préférez `<-` à `=`. Pour inspecter : `class()`, `typeof()`, `str()`.

```
class(prix)
```

```
#> [1] "numeric"
```

```
class(quantite)
```

```
#> [1] "integer"
```

```
class(produit)
```

```
#> [1] "character"
```

```
class(disponible)
```

```
#> [1] "logical"
```

3.6 Types de données

Type	Exemple	Test
`numeric`	`3.14`	`is.numeric()`
`integer`	`3L`	`is.integer()`
`character`	`"bonjour"`	`is.character()`
`logical`	`TRUE` / `FALSE`	`is.logical()`
`factor`	catégories ordonnées ou non	`is.factor()`

```
niveau <- factor(
  c("Bac", "Licence", "Master", "Licence", "Bac"),
  levels = c("Bac", "Licence", "Master", "Doctorat"),
  ordered = TRUE
)
niveau
```

```
#> [1] Bac      Licence Master  Licence Bac
#> Levels: Bac < Licence < Master < Doctorat
```

```
table(niveau)
```

```
#> niveau
```

```
#>      Bac  Licence   Master Doctorat
#>        2        2        1        0
```

Les **facteurs** stockent des catégories. Attention : certaines opérations numériques sur facteurs donnent des résultats surprenants — convertissez d'abord avec `as.character()` ou `as.numeric(as.character(...))` selon le cas.

3.7 Opérateurs

```
# Arithmétiques
10 + 3; 10 - 3; 10 * 3; 10 / 3; 10 %/% 3; 10 %% 3; 2^5
```

```
#> [1] 13
```

```
#> [1] 7
```

```
#> [1] 30
```

```
#> [1] 3.333333
```

```
#> [1] 3
```

```
#> [1] 1
```

```
#> [1] 32
```

```
# Comparaison (résultat logique)
5 > 3
```

```
#> [1] TRUE
```

```
5 == 5
```

```
#> [1] TRUE
```

```
5 != 4
```

```
#> [1] TRUE
```

```
# Logiques
TRUE & FALSE   # ET élément par élément
```

```
#> [1] FALSE
```

```
TRUE | FALSE   # OU
```

```
#> [1] TRUE
```

```
!TRUE          # NON
```

```
#> [1] FALSE
```

> ⚠ **`&&` vs `&`**
>
> `&` et `|` travaillent **vecteur par vecteur**. `&&` et `||` ne regardent que le premier élément — utiles dans `if()`, dangereux ailleurs.

3.8 Vecteurs

Le vecteur est la structure fondamentale de R.

```r
notes <- c(12, 15, 9, 18, 14)
length(notes)
```

```
#> [1] 5
```

```r
notes[1]          # 1er élément (indexation à partir de 1 !)
```

```
#> [1] 12
```

```r
notes[c(1, 3)]    # plusieurs positions
```

```
#> [1] 12  9
```

```r
notes[notes >= 12]  # filtrage logique
```

```
#> [1] 12 15 18 14
```

```r
# Opérations vectorisées
notes * 2
```

```
#> [1] 24 30 18 36 28
```

```r
sqrt(notes)
```

```
#> [1] 3.464102 3.872983 3.000000 4.242641 3.741657
```

```r
1:10
```

```
#>  [1]  1  2  3  4  5  6  7  8  9 10
```

```r
seq(0, 1, by = 0.25)
```

```
#> [1] 0.00 0.25 0.50 0.75 1.00
```

```r
rep(c("A", "B"), times = 3)
```

```
#> [1] "A" "B" "A" "B" "A" "B"
```

3.9 Matrices et tableaux

```r
m <- matrix(1:6, nrow = 2, ncol = 3, byrow = TRUE)
m
```

```
#>      [,1] [,2] [,3]
#> [1,]    1    2    3
#> [2,]    4    5    6
```

```r
m[1, 2]
```

```
#> [1] 2
```

```r
m[1, ]
```

```
#> [1] 1 2 3
```

```r
colSums(m)
```

```
#> [1] 5 7 9
```

Les **arrays** généralisent aux dimensions > 2. Pour l'analyse de données tabulaire, on préfère les data frames.

3.10 Listes

Une liste peut contenir des objets de types et longueurs différents.

```
eleve <- list(
  nom = "Samira",
  notes = c(14, 16, 12),
  admis = TRUE
)
eleve$nom
```

```
#> [1] "Samira"
```

```
eleve[["notes"]]
```

```
#> [1] 14 16 12
```

```
eleve[[2]][1]
```

```
#> [1] 14
```

3.11 Data frames et tibbles

```
df <- data.frame(
  produit = c("Stylo", "Cahier", "Agenda"),
  prix = c(1.5, 3.2, 8.0),
  stock = c(100, 40, 15)
)
df
```

```
#>   produit prix stock
#> 1   Stylo  1.5   100
#> 2  Cahier  3.2    40
#> 3  Agenda  8.0    15
```

```
df$prix
```

```
#> [1] 1.5 3.2 8.0
```

```
df[df$prix < 5, ]
```

```
#>   produit prix stock
#> 1   Stylo  1.5   100
#> 2  Cahier  3.2    40
```

Le **tibble** (package `tibble` / tidyverse) est un data frame modernisé : affichage plus lisible, comportement plus strict.

```
library(tibble)
tb <- as_tibble(df)
tb
```

```
#> # A tibble: 3 x 3
#>   produit  prix stock
#>   <chr>   <dbl> <dbl>
#> 1 Stylo     1.5   100
#> 2 Cahier    3.2    40
#> 3 Agenda    8      15
```

3.12 Indexation et sélection

```
data(mtcars)
mtcars[1:3, c("mpg", "cyl")]
```

```
#>                mpg cyl
#> Mazda RX4     21.0   6
#> Mazda RX4 Wag 21.0   6
#> Datsun 710    22.8   4
```

```
mtcars$cyl == 4
```

```
#>  [1] FALSE FALSE  TRUE FALSE FALSE FALSE FALSE  TRUE  TRUE FALSE FALSE FALSE
#> [13] FALSE FALSE FALSE FALSE FALSE  TRUE  TRUE  TRUE  TRUE FALSE FALSE FALSE
#> [25] FALSE  TRUE  TRUE  TRUE FALSE FALSE FALSE  TRUE
```

```
mtcars[mtcars$cyl == 4 & mtcars$mpg > 25, c("mpg", "cyl", "wt")]
```

```
#>                 mpg cyl    wt
#> Fiat 128       32.4   4 2.200
#> Honda Civic    30.4   4 1.615
#> Toyota Corolla 33.9   4 1.835
#> Fiat X1-9      27.3   4 1.935
#> Porsche 914-2  26.0   4 2.140
#> Lotus Europa   30.4   4 1.513
```

3.13 Conditions : `if`, `else`, `ifelse()`

```
note <- 14
if (note >= 10) {
  message("Admis")
} else {
  message("Ajourné")
}

# Version vectorisée
notes <- c(8, 12, 15, 9)
ifelse(notes >= 10, "Admis", "Ajourné")
```

```
#> [1] "Ajourné" "Admis"   "Admis"   "Ajourné"
```

3.14 Boucles `for` et `while`

```
total <- 0
for (n in notes) {
  total <- total + n
}
total
```

```
#> [1] 44
```

```
# Équivalent vectorisé (préférable)
sum(notes)
```

```
#> [1] 44
```

> **Préférez la vectorisation**
>
> En R, `sum(x)`, `mean(x)`, `lapply()` ou `dplyr` remplacent souvent les boucles, de façon plus claire et souvent plus rapide.

```r
i <- 1
acc <- 0
while (i <= 5) {
  acc <- acc + i
  i <- i + 1
}
acc
```

```
#> [1] 15
```

3.15 Fonctions personnalisées

```r
moyenne_ponderee <- function(valeurs, poids) {
  if (length(valeurs) != length(poids)) {
    stop("valeurs et poids doivent avoir la même longueur")
  }
  sum(valeurs * poids) / sum(poids)
}

moyenne_ponderee(c(10, 15, 20), c(1, 2, 1))
```

```
#> [1] 15
```

Arguments par défaut :

```r
bonjour <- function(prenom = "monde") {
  paste("Bonjour", prenom)
}
bonjour()
```

```
#> [1] "Bonjour monde"
```

```r
bonjour("Léa")
```

```
#> [1] "Bonjour Léa"
```

3.16 Gestion des erreurs

```r
lire_positif <- function(x) {
  tryCatch(
    {
      if (x < 0) stop("La valeur doit être positive")
      sqrt(x)
    },
    error = function(e) {
      message("Erreur : ", e$message)
      NA_real_
    }
  )
```

```
}
lire_positif(9)
```

```
#> [1] 3
```

```
lire_positif(-1)
```

```
#> [1] NA
```

3.17 Packages et bibliothèques

```
install.packages("dplyr")   # une fois
library(dplyr)              # à chaque session
# ou : dplyr::filter(...)   # sans charger tout le package
```

```
# Packages chargés (aperçu)
head(.packages(), 10)
```

```
#> [1] "tibble"    "stats"     "graphics"  "grDevices" "utils"     "datasets"
#> [7] "methods"   "base"
```

3.18 Bonnes pratiques de programmation

1. Noms explicites : `prix_ht` plutôt que `x1`
2. Un script = un objectif clair
3. Commentez le *pourquoi*, pas l'évidence
4. Évitez de dupliquer : créez une fonction
5. Fixez la graine pour la reproductibilité : `set.seed(2024)`
6. Ne stockez pas de mots de passe dans les scripts

```
set.seed(2024)
sample(1:100, 5)
```

```
#> [1] 66 37 45 60 17
```

3.19 Démonstration complète

```
set.seed(1)
notes <- round(rnorm(30, mean = 12, sd = 3), 1)
notes <- pmin(20, pmax(0, notes))

resume_notes <- function(x) {
  list(
    n = length(x),
    moyenne = mean(x),
    mediane = median(x),
    ecart_type = sd(x),
    taux_reussite = mean(x >= 10)
  )
}

resume_notes(notes)
```

```
#> $n
#> [1] 30
#>
#> $moyenne
#> [1] 12.25333
#>
#> $mediane
#> [1] 12.8
#>
#> $ecart_type
#> [1] 2.774357
#>
#> $taux_reussite
#> [1] 0.8333333
```

3.20 Erreurs fréquentes

Erreur	Explication
Indexation à 0	En R, le 1er élément est à l'indice **1**
`+` bloqué	Parenthese/guillemet non fermé
Facteur traité comme nombre	Convertir correctement avant calcul
Masque recyclé	Longueurs incompatibles → avertissement

3.21 Exercices

3.21.1 Niveau 1

1. Créez un vecteur `ages` de 5 âges et calculez min, max, moyenne.
2. Quelle est la différence entre `c(1,2)` et `list(1,2)` ?

3.21.2 Niveau 2

3. Écrivez une fonction `est_pair(n)` qui retourne `TRUE` si `n` est pair.
4. À partir de `mtcars`, sélectionnez les voitures à 6 cylindres et `hp > 110`.

3.21.3 Niveau 3

5. Écrivez `resume_numeric(x)` qui gère le cas où `x` contient des `NA` (`na.rm = TRUE`) et retourne un named vector (n, mean, sd).
6. Simulez 100 tirages d'un dé équilibré et comptez les fréquences avec `table()`.

3.22 Corrections

```
ages <- c(22, 34, 19, 45, 31)
min(ages); max(ages); mean(ages)
```

```
#> [1] 19
```

```
#> [1] 45
```

```
#> [1] 30.2
```

```r
est_pair <- function(n) {
  n %% 2 == 0
}
est_pair(4)
```

```
#> [1] TRUE
```

```r
est_pair(7)
```

```
#> [1] FALSE
```

```r
mtcars[mtcars$cyl == 6 & mtcars$hp > 110, c("mpg", "cyl", "hp")]
```

```
#>                mpg cyl  hp
#> Merc 280      19.2   6 123
#> Merc 280C     17.8   6 123
#> Ferrari Dino 19.7   6 175
```

```r
resume_numeric <- function(x) {
  c(
    n = sum(!is.na(x)),
    mean = mean(x, na.rm = TRUE),
    sd = sd(x, na.rm = TRUE)
  )
}
resume_numeric(c(1, 2, NA, 4))
```

```
#>        n     mean       sd
#> 3.000000 2.333333 1.527525
```

```r
set.seed(42)
de <- sample(1:6, 100, replace = TRUE)
table(de)
```

```
#> de
#>  1  2  3  4  5  6
#> 15 24 10 19 19 13
```

3.23 Points clés

- Tout est (presque) un **vecteur** ; les data frames organisent des vecteurs en colonnes.
- L'indexation commence à **1** ; le filtrage logique est puissant.
- Préférez fonctions + vectorisation aux longues boucles.
- Les packages étendent R : installez une fois, chargez à chaque session.

3.24 Ressources

- Aide intégrée : `?mean`, `help(factor)`
- R Core Team (2024), Wickham et al. (2019)

3.25 Suite

1 : importer CSV, Excel et vérifier la qualité des données.

Part III

Partie III — Acquisition des données

Chapter 4

Acquisition et importation des données

4.1 Introduction

Avant d'analyser, il faut **obtenir** les données et les **lire correctement** dans R. Ce chapitre couvre les sources courantes (CSV, Excel, texte), l'export, les jeux intégrés, les chemins de fichiers, ainsi que les principes d'accès aux bases et au Web — le tout de façon utilisable hors ligne une fois les fichiers présents dans `data/`.

4.2 Objectifs pédagogiques

- Identifier les principales sources de données
- Importer CSV, Excel et fichiers texte
- Exporter des résultats
- Gérer les chemins et le dossier de travail
- Vérifier la qualité d'un import
- Comprendre (sans dépendance réseau) bases de données et API

4.3 Prérequis

Chapter 3 (data frames, fonctions). Fichiers disponibles dans `data/`.

4.4 Comprendre les sources de données

Source	Exemple	Package / fonction
CSV	exports ERP, open data	`readr::read_csv`, `read.csv`
Excel	fichiers métier	`readxl::read_excel`
Texte	logs, exports TSV	`read.table`, `read_tsv`
R intégré	`mtcars`, `iris`	`data()`
Base SQL	PostgreSQL, SQLite	`DBI`, `RSQLite`
API / Web	JSON, HTML	`httr2`, `jsonlite`

Vocabulaire

- **Encodage** : représentation des caractères (UTF-8 recommandé).
- **Séparateur** : `,` `;` ou tabulation entre colonnes.
- **En-tête** : première ligne = noms de variables.
- **Données brutes** : fichier original non modifié.

4.5 Importer des fichiers CSV

```
library(readr)

ventes <- read_csv("data/ventes.csv", show_col_types = FALSE)
head(ventes, 3)
```

```
#> # A tibble: 3 x 7
#>   id_vente date       region produit quantite prix_unitaire montant
#>   <chr>    <date>     <chr>  <chr>      <dbl>         <dbl>   <dbl>
#> 1 V0001    2023-03-07 Nord   Cahier         6          7.32    43.9
#> 2 V0002    2023-02-06 Nord   Agenda        19          5.45   104.
#> 3 V0003    2023-02-14 Nord   Stylo          4          9.78    39.1
```

```
nrow(ventes)
```

```
#> [1] 201
```

```
colnames(ventes)
```

```
#> [1] "id_vente"      "date"          "region"        "produit"
#> [5] "quantite"      "prix_unitaire" "montant"
```

Avec le R de base :

```
ventes_base <- read.csv("data/ventes.csv", fileEncoding = "UTF-8")
# stringsAsFactors = FALSE est le défaut depuis R 4.0
```

`readr` vs `read.csv`

`read_csv` est plus rapide, devine mieux les types et retourne un tibble. Pour ce livre, nous privilégions `readr` (tidyverse).

Fichier « sale » (noms irréguliers) :

```
brut <- read_csv("data/brut/ventes_brutes.csv", show_col_types = FALSE)
names(brut)
```

```
#> [1] "ID"           "Date_Vente"   "REGION"       "Produit"      "Qte"
#> [6] "PrixUnitaire" "TOTAL"
```

4.6 Lire des fichiers Excel

```
library(readxl)

# Lister les feuilles
excel_sheets("data/ventes.xlsx")
```

```
#> [1] "Ventes" "Resume"
```

```
ventes_xl <- read_excel("data/ventes.xlsx", sheet = 1)
head(ventes_xl, 2)
```

```
#> # A tibble: 2 x 7
#>   id_vente date                region produit quantite prix_unitaire montant
#>   <chr>    <dttm>              <chr>  <chr>      <dbl>         <dbl>   <dbl>
#> 1 V0001    2023-03-07 00:00:00 Nord   Cahier         6          7.32    43.9
#> 2 V0002    2023-02-06 00:00:00 Nord   Agenda        19          5.45   104.
```

Arguments utiles : `sheet`, `range`, `skip`, `na`.

4.7 Importer des fichiers texte

```
meteo_txt <- read_tsv("data/meteo_extrait.txt", show_col_types = FALSE)
head(meteo_txt, 3)
```

```
#> # A tibble: 3 x 5
#>   date       temperature_c precipitations_mm humidite_pct ville
#>   <date>             <dbl>             <dbl>        <dbl> <chr>
#> 1 2023-01-01          10.1               1.8           74 Exempleville
#> 2 2023-01-02          12.1               0.6           61 Exempleville
#> 3 2023-01-03          12.4               0.2           67 Exempleville
```

Ou :

```
meteo2 <- read.table(
  "data/meteo_extrait.txt",
  header = TRUE,
  sep = "\t",
  fileEncoding = "UTF-8"
)
```

4.8 Exporter des données

```
write_csv(ventes, "outputs/ventes_export.csv")
write.csv(ventes, "outputs/ventes_export_base.csv", row.names = FALSE)
```

```
dir.create("outputs", showWarnings = FALSE)
write_csv(head(ventes, 10), "outputs/ventes_apercu.csv")
file.exists("outputs/ventes_apercu.csv")
```

```
#> [1] TRUE
```

4.9 Jeux de données intégrés

```
data(mtcars)
data(iris)
# Liste partielle
head(ls("package:datasets"), 15)
```

```
#>  [1] "ability.cov"   "airmiles"      "AirPassengers" "airquality"
#>  [5] "anscombe"      "attenu"        "attitude"      "austres"
```

```
#>  [9] "beaver1"       "beaver2"       "BJsales"       "BJsales.lead"
#> [13] "BOD"           "cars"          "ChickWeight"
```

Des packages apportent aussi des données (`palmerpenguins`, `ggplot2::mpg`, etc.).

4.10 Chemins de fichiers et dossiers de travail

```
getwd()
```

```
#> [1] "C:/Users/Alhamdoulilah/Desktop/LIVRE R/formation-r"
```

```
list.files("data")
```

```
#> [1] "brut"              "clients.csv"       "commandes.csv"
#> [4] "demographie.csv"   "etudes_notes.csv"  "meteo.csv"
#> [7] "meteo_extrait.txt" "ventes.csv"        "ventes.xlsx"
```

```
file.exists("data/ventes.csv")
```

```
#> [1] TRUE
```

> ⚠ Évitez `setwd()` dans les scripts partagés
>
> Le chemin absolu d'un collègue diffère du vôtre. Préférez un **projet RStudio** (`.Rproj`) ou le package `here`.

```
# Bon : relatif au projet
read_csv("data/ventes.csv")

# Fragile : chemin absolu machine-dépendant
# read_csv("C:/Users/Moi/Desktop/ventes.csv")
```

4.11 Principes d'accès aux bases de données

Sans installer de serveur, retenez le schéma mental :

```
# Exemple conceptuel avec SQLite
library(DBI)
library(RSQLite)
con <- dbConnect(RSQLite::SQLite(), "data/exemple.sqlite")
dbListTables(con)
df <- dbGetQuery(con, "SELECT * FROM ventes LIMIT 10")
dbDisconnect(con)
```

Idées clés : connexion → requête SQL → data frame → fermer la connexion. Ne committez jamais de mots de passe dans Git.

4.12 API et données Web (principes hors ligne)

Les API renvoient souvent du **JSON**. Le flux typique :

1. Requête HTTP (`httr2` / `httr`)
2. Parsing JSON (`jsonlite`)
3. Normalisation en data frame (`tidyr::unnest`, etc.)

Pour ce livre, les exercices restent sur fichiers locaux afin de fonctionner **sans Internet**.

4.13 Vérifier la qualité des données importées

```
library(dplyr)

glimpse(ventes)
```

```
#> Rows: 201
#> Columns: 7
#> $ id_vente      <chr> "V0001", "V0002", "V0003", "V0004", "V0005", "V0006", "V~
#> $ date          <date> 2023-03-07, 2023-02-06, 2023-02-14, 2023-07-07, 2023-09~
#> $ region        <chr> "Nord", "Nord", "Nord", "Ouest", NA, "Ouest", "Nord", "S~
#> $ produit       <chr> "Cahier", "Agenda", "Stylo", "Marqueur", "Cahier", "Clas~
#> $ quantite      <dbl> 6, 19, 4, 20, 10, 6, 14, 3, 18, 1, 17, 9, 15, 9, 7, 1, 8~
#> $ prix_unitaire <dbl> 7.32, 5.45, 9.78, 3.16, 13.57, 12.47, 19.46, 23.10, 16.9~
#> $ montant       <dbl> 43.92, 103.55, 39.12, 63.20, 135.70, 74.82, 272.44, 69.3~
```

```
summary(ventes)
```

```
#>    id_vente             date               region            produit
#>  Length:201         Min.   :2023-01-01   Length:201         Length:201
#>  Class :character   1st Qu.:2023-04-15   Class :character   Class :character
#>  Mode  :character   Median :2023-07-07   Mode  :character   Mode  :character
#>                     Mean   :2023-07-03
#>                     3rd Qu.:2023-09-29
#>                     Max.   :2023-12-29
#>
#>     quantite     prix_unitaire      montant
#>  Min.   : 1.00   Min.   : 1.62   Min.   :  3.72
#>  1st Qu.: 5.00   1st Qu.: 7.32   1st Qu.: 41.58
#>  Median :10.00   Median :12.39   Median :104.09
#>  Mean   :10.38   Mean   :12.93   Mean   :129.52
#>  3rd Qu.:15.00   3rd Qu.:19.02   3rd Qu.:195.36
#>  Max.   :20.00   Max.   :24.73   Max.   :483.20
#>  NA's   :1
```

```
# Valeurs manquantes par colonne
colSums(is.na(ventes))
```

```
#>      id_vente          date        region       produit      quantite
#>             0             0             2             0             1
#> prix_unitaire       montant
#>             0             0
```

```
# Doublons potentiels
sum(duplicated(ventes))
```

```
#> [1] 1
```

Checklist d'import :

1. Nombre de lignes / colonnes attendu ?
2. Types corrects (dates en date, nombres en numeric) ?
3. Encodage des accents OK ?
4. NA, doublons, valeurs extrêmes ?

```
meteo <- read_csv("data/meteo.csv", show_col_types = FALSE)
summary(meteo$temperature_c)
```

```
#>    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
#>   -3.60    5.80   11.90   12.11   18.30   27.00
```

```
range(meteo$date)
```

```
#> [1] "2023-01-01" "2023-12-31"
```

4.14 Démonstration complète

```
library(readr)
library(dplyr)

ventes <- read_csv("data/ventes.csv", show_col_types = FALSE)

rapport_import <- list(
  lignes = nrow(ventes),
  colonnes = ncol(ventes),
  na_total = sum(is.na(ventes)),
  periode = range(ventes$date, na.rm = TRUE),
  montant_moyen = mean(ventes$montant, na.rm = TRUE)
)
rapport_import
```

```
#> $lignes
#> [1] 201
#>
#> $colonnes
#> [1] 7
#>
#> $na_total
#> [1] 3
#>
#> $periode
#> [1] "2023-01-01" "2023-12-29"
#>
#> $montant_moyen
#> [1] 129.5235
```

4.15 Erreurs fréquentes

Symptôme	Cause	Remède
Accents cassés	Mauvais encodage	`locale = locale(encoding = "UTF-8")` ou `fileEncoding`
Une seule colonne	Mauvais séparateur	`read_csv2` (`;`) ou `delim`
Dates en caractère	Parsing	`col_date()` ou `lubridate`
Fichier introuvable	Mauvais `getwd()`	Projet RStudio + chemins relatifs

4.16 Exercices

4.16.1 Niveau 1

1. Importez `data/demographie.csv` et affichez `glimpse()`.
2. Combien de feuilles dans `data/ventes.xlsx` ?

4.16.2 Niveau 2

3. Importez `data/meteo_extrait.txt` et calculez la température moyenne.
4. Exportez les 20 premières lignes de `ventes` vers `outputs/exo_ventes.csv`.

4.16.3 Niveau 3

5. Écrivez une fonction `diagnostiquer(chemin)` qui retourne un tibble : `n_lignes`, `n_cols`, `n_na`, `n_doublons`.
6. Comparez `read_csv("data/brut/ventes_brutes.csv")` et `read_csv("data/ventes.csv")` : que remarquez-vous sur les noms de colonnes ?

4.17 Corrections

```
demo <- read_csv("data/demographie.csv", show_col_types = FALSE)
glimpse(demo)
```

```
#> Rows: 150
#> Columns: 6
#> $ id             <dbl> 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, ~
#> $ age            <dbl> 26, 68, 150, 61, 22, 30, 54, 31, 60, 30, 64, 44, 55, 27~
#> $ sexe           <chr> "Femme", "Homme", "Femme", "Homme", "Homme", "Homme", "~
#> $ ville          <chr> "Lille", "Marseille", "Paris", "Marseille", "Toulouse",~
#> $ revenu_mensuel <dbl> 1388, 2050, 1104, 2117, 2953, 2382, 50000, 3007, 1655, ~
#> $ niveau_etudes  <chr> "Licence", "Bac", "Bac", "Autre", "Master", "Licence", ~
```

```
excel_sheets("data/ventes.xlsx")
```

```
#> [1] "Ventes" "Resume"
```

```
m <- read_tsv("data/meteo_extrait.txt", show_col_types = FALSE)
mean(m$temperature_c)
```

```
#> [1] 14.03333
```

```
write_csv(head(ventes, 20), "outputs/exo_ventes.csv")
```

```
diagnostiquer <- function(chemin) {
  x <- read_csv(chemin, show_col_types = FALSE)
  tibble(
    fichier = basename(chemin),
    n_lignes = nrow(x),
    n_cols = ncol(x),
    n_na = sum(is.na(x)),
    n_doublons = sum(duplicated(x))
  )
}
diagnostiquer("data/ventes.csv")
```

```
#> # A tibble: 1 x 5
#>   fichier    n_lignes n_cols  n_na n_doublons
#>   <chr>         <int>  <int> <int>      <int>
#> 1 ventes.csv      201      7     3          1
```

4.18 Points clés

- Préférez **chemins relatifs** et fichiers dans `data/`.
- Vérifiez toujours types, NA et dimensions après import.
- CSV / Excel / texte couvrent 90 % des besoins débutants.
- Bases et API suivent le même principe : connecter → lire → tabulariser.

4.19 Ressources

- `vignette("readr")`, documentation readxl
- Wickham et al. (2019)

4.20 Suite

2 : nettoyer, transformer et joindre les données avec dplyr et tidyr.

Part IV

Partie IV — Préparation des données

Chapter 5

Nettoyage et préparation des données

5.1 Introduction

Des données mal préparées produisent des analyses trompeuses. Ce chapitre enseigne le **nettoyage** et la **transformation** avec `dplyr`, `tidyr`, `stringr` et `lubridate` : renommer, filtrer, créer des variables, traiter les NA, détecter doublons et aberrants, joindre des tables et pivoter.

5.2 Objectifs pédagogiques

- Examiner et comprendre la structure d'un jeu de données
- Chaîner des verbes dplyr (`%>%` / `|>`)
- Traiter NA, doublons et valeurs aberrantes
- Manipuler textes et dates
- Agréger, joindre et pivoter
- Construire un pipeline reproductible

5.3 Prérequis

1. Packages : `tidyverse`, `janitor` (utile), `lubridate`.

```
library(tidyverse)
library(lubridate)
if (requireNamespace("janitor", quietly = TRUE)) {
  library(janitor)
} else {
  message("Package janitor non installé : clean_names() sera illustré sans exécution forcée.")
}
```

5.4 Qualité des données

Une bonne qualité implique : types corrects, noms cohérents, peu de NA injustifiés, pas de doublons involontaires, valeurs dans des plages plausibles, documentation des règles de nettoyage.

> **!** Règle d'or
>
> Ne modifiez jamais le fichier brut. Créez une version nettoyée via un script.

5.5 Examiner la structure

```
ventes <- read_csv("data/ventes.csv", show_col_types = FALSE)
glimpse(ventes)
```

```
#> Rows: 201
#> Columns: 7
#> $ id_vente      <chr> "V0001", "V0002", "V0003", "V0004", "V0005", "V0006", "V~
#> $ date          <date> 2023-03-07, 2023-02-06, 2023-02-14, 2023-07-07, 2023-09~
#> $ region        <chr> "Nord", "Nord", "Nord", "Ouest", NA, "Ouest", "Nord", "S~
#> $ produit       <chr> "Cahier", "Agenda", "Stylo", "Marqueur", "Cahier", "Clas~
#> $ quantite      <dbl> 6, 19, 4, 20, 10, 6, 14, 3, 18, 1, 17, 9, 15, 9, 7, 1, 8~
#> $ prix_unitaire <dbl> 7.32, 5.45, 9.78, 3.16, 13.57, 12.47, 19.46, 23.10, 16.9~
#> $ montant       <dbl> 43.92, 103.55, 39.12, 63.20, 135.70, 74.82, 272.44, 69.3~
```

```
skim_lite <- summary(ventes)
```

5.6 Renommer, sélectionner, filtrer, trier

```
ventes_n <- ventes |>
  rename(ca = montant) |>
  select(id_vente, date, region, produit, quantite, ca) |>
  filter(!is.na(region), ca > 0) |>
  arrange(desc(ca))

head(ventes_n, 3)
```

```
#> # A tibble: 3 x 6
#>   id_vente date       region produit  quantite    ca
#>   <chr>    <date>     <chr>  <chr>       <dbl> <dbl>
#> 1 V0072    2023-03-14 Est    Cahier         20  483.
#> 2 V0141    2023-10-23 Sud    Marqueur       18  445.
#> 3 V0109    2023-08-01 Nord   Cahier         19  431.
```

> **i** Vocabulaire dplyr
>
> - `select` : colonnes
> - `filter` : lignes
> - `mutate` : créer / modifier des variables
> - `arrange` : tri
> - `summarise` + `group_by` : agrégats

5.7 Créer des variables et modifier les types

```
ventes2 <- ventes |>
  mutate(
    date = as.Date(date),
    mois = floor_date(date, "month"),
    trimestre = quarter(date),
    categorie_ca = case_when(
      montant < 20 ~ "petit",
      montant < 100 ~ "moyen",
```

```
      TRUE ~ "gros"
    ),
    region = factor(region)
  )
table(ventes2$categorie_ca)
```

```
#>
#>  gros moyen petit
#>   104    77    20
```

5.8 Traiter les valeurs manquantes

```
colSums(is.na(ventes))
```

```
#>      id_vente          date        region       produit      quantite
#>             0             0             2             0             1
#> prix_unitaire       montant
#>             0             0
```

```
# Options selon le contexte :
# 1. Filtrer
ventes_complet <- ventes |> filter(!is.na(region), !is.na(quantite))

# 2. Imputer une valeur (avec prudence)
ventes_imp <- ventes |>
  mutate(quantite = replace_na(quantite, median(quantite, na.rm = TRUE)))

# 3. Documenter et garder les NA pour analyse de non-réponse
```

> ⚠ Ne pas imputer « par réflexe »
>
> Remplacer tous les NA par 0 peut biaiser une moyenne. Choisissez une règle **motivée** et notez-la dans le rapport.

5.9 Doublons

```
sum(duplicated(ventes))
```

```
#> [1] 1
```

```
ventes_uniques <- ventes |> distinct()
nrow(ventes) - nrow(ventes_uniques)
```

```
#> [1] 1
```

5.10 Valeurs aberrantes

```
demo <- read_csv("data/demographie.csv", show_col_types = FALSE)

# Règle IQR pour l'âge
Q1 <- quantile(demo$age, 0.25)
Q3 <- quantile(demo$age, 0.75)
```

```
IQR_age <- Q3 - Q1
bornes <- c(Q1 - 1.5 * IQR_age, Q3 + 1.5 * IQR_age)
bornes
```

```
#>   25%   75%
#> -15.5 108.5
```

```
aberrants_age <- demo |> filter(age < bornes[1] | age > bornes[2])
aberrants_age
```

```
#> # A tibble: 1 x 6
#>      id   age sexe  ville revenu_mensuel niveau_etudes
#>   <dbl> <dbl> <chr> <chr>          <dbl> <chr>
#> 1     3   150 Femme Paris           1104 Bac
```

Les aberrants ne sont pas automatiquement des erreurs : un revenu très élevé peut être réel. **Examinez** avant de supprimer.

5.11 Chaînes de caractères (`stringr`)

```
produits <- ventes |>
  mutate(
    produit_min = str_to_lower(produit),
    commence_c = str_detect(produit, "^C"),
    longueur = str_length(produit)
  )
head(produits[, c("produit", "produit_min", "commence_c")], 4)
```

```
#> # A tibble: 4 x 3
#>   produit  produit_min commence_c
#>   <chr>    <chr>       <lgl>
#> 1 Cahier   cahier      TRUE
#> 2 Agenda   agenda      FALSE
#> 3 Stylo    stylo       FALSE
#> 4 Marqueur marqueur    FALSE
```

5.12 Dates (`lubridate`)

```
meteo <- read_csv("data/meteo.csv", show_col_types = FALSE) |>
  mutate(
    date = ymd(date),
    annee = year(date),
    mois = month(date, label = TRUE, abbr = TRUE),
    jour_semaine = wday(date, label = TRUE)
  )
head(meteo[, c("date", "mois", "jour_semaine")], 3)
```

```
#> # A tibble: 3 x 3
#>   date       mois  jour_semaine
#>   <date>     <ord> <ord>
#> 1 2023-01-01 janv  "dim\\."
#> 2 2023-01-02 janv  "lun\\."
#> 3 2023-01-03 janv  "mar\\."
```

5.13 Regrouper et résumer

```
ca_region <- ventes |>
  filter(!is.na(region)) |>
  group_by(region) |>
  summarise(
    n = n(),
    ca_total = sum(montant, na.rm = TRUE),
    ca_moyen = mean(montant, na.rm = TRUE),
    .groups = "drop"
  ) |>
  arrange(desc(ca_total))
ca_region
```

```
#> # A tibble: 4 x 4
#>   region     n ca_total ca_moyen
#>   <chr>  <int>    <dbl>    <dbl>
#> 1 Ouest     59    7621.     129.
#> 2 Est       51    7164.     140.
#> 3 Sud       42    5995.     143.
#> 4 Nord      47    5096.     108.
```

5.14 Fusionner plusieurs tables

```
clients <- read_csv("data/clients.csv", show_col_types = FALSE)
commandes <- read_csv("data/commandes.csv", show_col_types = FALSE)

commandes_enrichies <- commandes |>
  left_join(clients, by = "client_id")

head(commandes_enrichies, 3)
```

```
#> # A tibble: 3 x 6
#>   commande_id client_id montant date       nom       segment
#>   <chr>       <chr>       <dbl> <date>     <chr>     <chr>
#> 1 CMD001      C004         271. 2023-02-16 Client_4  Association
#> 2 CMD002      C028         286. 2023-09-11 Client_28 Association
#> 3 CMD003      C018         190. 2023-02-07 Client_18 Particulier
```

```
# CA par segment
commandes_enrichies |>
  group_by(segment) |>
  summarise(ca = sum(montant), n = n(), .groups = "drop")
```

```
#> # A tibble: 3 x 3
#>   segment        ca     n
#>   <chr>       <dbl> <int>
#> 1 Association 9855.    36
#> 2 Entreprise  3747.    13
#> 3 Particulier 6393.    31
```

Types de jointures : `left_join`, `inner_join`, `full_join`, `anti_join` (non appariés).

5.15 Large ⇄ long (`tidyr`)

```
# Exemple long → large
ca_long <- ca_region |> select(region, ca_total)
ca_wide <- ca_long |>
  pivot_wider(names_from = region, values_from = ca_total)
ca_wide
```

```
#> # A tibble: 1 x 4
#>   Ouest   Est   Sud  Nord
#>   <dbl> <dbl> <dbl> <dbl>
#> 1 7621. 7164. 5995. 5096.
```

```
# Large → long
ca_long2 <- ca_wide |>
  pivot_longer(everything(), names_to = "region", values_to = "ca_total")
ca_long2
```

```
#> # A tibble: 4 x 2
#>   region ca_total
#>   <chr>     <dbl>
#> 1 Ouest     7621.
#> 2 Est       7164.
#> 3 Sud       5995.
#> 4 Nord      5096.
```

5.16 Pipeline de préparation reproductible

```
preparer_ventes <- function(chemin = "data/ventes.csv") {
  read_csv(chemin, show_col_types = FALSE) |>
    distinct() |>
    mutate(
      date = as.Date(date),
      region = str_trim(region)
    ) |>
    filter(!is.na(region), !is.na(quantite), montant > 0) |>
    mutate(
      mois = floor_date(date, "month"),
      produit = factor(produit),
      region = factor(region)
    )
}

ventes_clean <- preparer_ventes()
glimpse(ventes_clean)
```

```
#> Rows: 197
#> Columns: 8
#> $ id_vente      <chr> "V0001", "V0002", "V0003", "V0004", "V0006", "V0007", "V~
#> $ date          <date> 2023-03-07, 2023-02-06, 2023-02-14, 2023-07-07, 2023-04~
#> $ region        <fct> Nord, Nord, Nord, Ouest, Ouest, Nord, Sud, Est, Ouest, O~
#> $ produit       <fct> Cahier, Agenda, Stylo, Marqueur, Classeur, Stylo, Stylo,~
#> $ quantite      <dbl> 6, 19, 4, 20, 6, 14, 3, 18, 1, 17, 9, 15, 9, 7, 1, 8, 11~
#> $ prix_unitaire <dbl> 7.32, 5.45, 9.78, 3.16, 12.47, 19.46, 23.10, 16.95, 11.3~
```

```
#> $ montant       <dbl> 43.92, 103.55, 39.12, 63.20, 74.82, 272.44, 69.30, 305.1~
#> $ mois          <date> 2023-03-01, 2023-02-01, 2023-02-01, 2023-07-01, 2023-04~
```

```
write_csv(ventes_clean, "data/ventes_clean.csv")
```

5.17 Démonstration complète

```
demo_clean <- demo |>
  filter(age >= 18, age <= 100) |>
  mutate(
    revenu_mensuel = if_else(revenu_mensuel > 20000, NA_real_, revenu_mensuel),
    sexe = factor(sexe),
    ville = factor(ville)
  )

demo_clean |>
  group_by(ville) |>
  summarise(
    n = n(),
    age_moyen = mean(age),
    revenu_median = median(revenu_mensuel, na.rm = TRUE),
    .groups = "drop"
  )
```

```
#> # A tibble: 5 x 4
#>   ville         n age_moyen revenu_median
#>   <fct>     <int>     <dbl>         <dbl>
#> 1 Lille        24      49.4         1994.
#> 2 Lyon         40      44.6         2289
#> 3 Marseille    34      49.1         2118.
#> 4 Paris        25      45.6         2338
#> 5 Toulouse     26      45.4         2257
```

5.18 Erreurs fréquentes

Erreur	Cause	Solution
`filter(region == NA)`	Mauvaise syntaxe	`is.na(region)` / `!is.na`
Jointure qui multiplie les lignes	Clé non unique	Vérifier `count(client_id)`
`summarise` sans `group_by` voulu	Oubli	Contrôler le nombre de lignes résultat
Dates en character	Pas de parsing	`ymd()` / `as.Date()`

5.19 Exercices

5.19.1 Niveau 1

1. À partir de `ventes`, sélectionnez `produit` et `montant`, triez par montant décroissant.
2. Comptez les NA de `region`.

5.19.2 Niveau 2

3. Calculez le CA total par `produit`.
4. Joignez `commandes` et `clients` et calculez le montant moyen par `segment`.

5.19.3 Niveau 3

5. Nettoyez `data/brut/ventes_brutes.csv` : noms propres (`clean_names`), types, suppression doublons, filtre NA.
6. Pour `demographie`, créez `tranche_age` (18–30, 31–50, 51+) et un résumé du revenu médian par tranche.

5.20 Corrections

```
ventes |> select(produit, montant) |> arrange(desc(montant)) |> head(5)
```

```
#> # A tibble: 5 x 2
#>   produit  montant
#>   <chr>      <dbl>
#> 1 Cahier      483.
#> 2 Marqueur    445.
#> 3 Cahier      431.
#> 4 Marqueur    405.
#> 5 Cahier      386
```

```
sum(is.na(ventes$region))
```

```
#> [1] 2
```

```
ventes |>
  group_by(produit) |>
  summarise(ca = sum(montant, na.rm = TRUE), .groups = "drop")
```

```
#> # A tibble: 5 x 2
#>   produit     ca
#>   <chr>    <dbl>
#> 1 Agenda   4354.
#> 2 Cahier   5520.
#> 3 Classeur 4978.
#> 4 Marqueur 5856.
#> 5 Stylo    5326.
```

```
commandes |>
  left_join(clients, by = "client_id") |>
  group_by(segment) |>
  summarise(montant_moyen = mean(montant), .groups = "drop")
```

```
#> # A tibble: 3 x 2
#>   segment     montant_moyen
#>   <chr>               <dbl>
#> 1 Association          274.
#> 2 Entreprise           288.
#> 3 Particulier          206.
```

```
brut_clean <- read_csv("data/brut/ventes_brutes.csv", show_col_types = FALSE) |>
  janitor::clean_names() |>
  rename(
    id_vente = id,
    date = date_vente,
    quantite = qte,
    montant = total
  ) |>
```

```
  mutate(date = as.Date(date), produit = str_trim(produit)) |>
  distinct() |>
  filter(!is.na(region))
nrow(brut_clean)
```

```
#> [1] 198
```

```
demo |>
  mutate(
    tranche_age = case_when(
      age <= 30 ~ "18-30",
      age <= 50 ~ "31-50",
      TRUE ~ "51+"
    )
  ) |>
  group_by(tranche_age) |>
  summarise(revenu_median = median(revenu_mensuel, na.rm = TRUE), n = n(), .groups = "drop")
```

```
#> # A tibble: 3 x 3
#>   tranche_age revenu_median     n
#>   <chr>               <dbl> <int>
#> 1 18-30                2226    37
#> 2 31-50                2133    44
#> 3 51+                  2215    69
```

5.21 Points clés

- Un pipeline clair bat une série de clics manuels.
- Chaque transformation doit être **justifiée** et **vérifiable**.
- Jointures et pivots sont au cœur de la mise en forme analytique.

5.22 Ressources

- Wickham et al. (2019), documentation dplyr / tidyr
- Package `janitor` pour les noms de colonnes

5.23 Suite

3 : résumer et interpréter avec des statistiques descriptives.

Part V

Partie V — Statistiques descriptives

Chapter 6

Exploration et statistiques descriptives

6.1 Introduction

Les **statistiques descriptives** résument un jeu de données sans encore généraliser à une population plus large. Elles répondent à : combien ? quelle tendance ? quelle dispersion ? quelles associations apparentes ?

6.2 Objectifs pédagogiques

- Calculer effectifs, tendances centrales et dispersion
- Lire quantiles, distributions et tableaux croisés
- Comparer des groupes
- Calculer et interpréter une corrélation (sans causalité)
- Éviter les erreurs d'interprétation courantes

6.3 Prérequis

2. Notions de moyenne vues en Chapter 3.

```
library(tidyverse)
ventes <- read_csv("data/ventes.csv", show_col_types = FALSE) |>
  filter(!is.na(region), !is.na(quantite))
demo <- read_csv("data/demographie.csv", show_col_types = FALSE) |>
  filter(age >= 18, age <= 100, revenu_mensuel < 20000)
```

6.4 Analyse exploratoire (EDA)

L'EDA combine tableaux et graphiques pour comprendre la structure avant toute modélisation. Questions types : valeurs manquantes ? asymétrie ? groupes très différents ? relations linéaires ?

> **i** Vocabulaire
>
> - **Tendance centrale** : moyenne, médiane, mode
> - **Dispersion** : étendue, variance, écart-type, IQR
> - **Distribution** : comment les valeurs se répartissent

- **Corrélation** : association linéaire entre deux variables quantitatives

6.5 Effectifs et fréquences

```
# Effectifs
table(ventes$region)
```

```
#>
#>   Est  Nord Ouest   Sud
#>    50    47    59    42
```

```
# Fréquences relatives
prop.table(table(ventes$region)) |> round(3)
```

```
#>
#>   Est  Nord Ouest   Sud
#> 0.253 0.237 0.298 0.212
```

```
ventes |> count(region, sort = TRUE) |>
  mutate(freq = n / sum(n))
```

```
#> # A tibble: 4 x 3
#>   region     n  freq
#>   <chr>  <int> <dbl>
#> 1 Ouest     59 0.298
#> 2 Est       50 0.253
#> 3 Nord      47 0.237
#> 4 Sud       42 0.212
```

6.6 Moyenne, médiane et mode

```
x <- ventes$montant
mean(x)
```

```
#> [1] 130.5734
```

```
median(x)
```

```
#> [1] 105.875
```

```
# Mode (valeur la plus fréquente) - pour variable discrète / qualitative
mode_stat <- function(v) {
  tb <- sort(table(v), decreasing = TRUE)
  names(tb)[1]
}
mode_stat(ventes$produit)
```

```
#> [1] "Marqueur"
```

Interprétation : si moyenne médiane, la distribution est souvent tirée vers la droite (quelques gros montants). La médiane est plus robuste aux extrêmes.

```
# Démonstration avec un extreme
y <- c(10, 12, 11, 13, 12, 200)
mean(y)
```

```
#> [1] 43
```

```
median(y)
```

```
#> [1] 12
```

6.7 Minimum, maximum, étendue

```
min(x); max(x); range(x)
```

```
#> [1] 3.72
```

```
#> [1] 483.2
```

```
#> [1]   3.72 483.20
```

```
diff(range(x))  # étendue
```

```
#> [1] 479.48
```

6.8 Quartiles et quantiles

```
quantile(x, probs = c(0, 0.25, 0.5, 0.75, 1))
```

```
#>      0%     25%     50%     75%    100%
#>   3.720  42.535 105.875 196.185 483.200
```

```
IQR(x)  # Q3 - Q1
```

```
#> [1] 153.65
```

```
summary(x)
```

```
#>    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
#>    3.72   42.53  105.88  130.57  196.19  483.20
```

6.9 Variance et écart-type

```
var(x)   # variance
```

```
#> [1] 10963.07
```

```
sd(x)    # écart-type (même unité que x)
```

```
#> [1] 104.7047
```

L'**écart-type** mesure l'écart typique à la moyenne. Plus il est grand, plus les observations sont dispersées.

6.10 Distribution des variables

```
# Aperçu numérique
demo |>
  summarise(
    age_mean = mean(age),
    age_sd = sd(age),
    revenu_mean = mean(revenu_mensuel),
```

```
    revenu_sd = sd(revenu_mensuel),
    revenu_skew_approx = mean(revenu_mensuel) - median(revenu_mensuel)
  )
```

```
#> # A tibble: 1 x 5
#>   age_mean age_sd revenu_mean revenu_sd revenu_skew_approx
#>      <dbl>  <dbl>       <dbl>     <dbl>              <dbl>
#> 1     46.6   17.7       2180.      551.              -11.8
```

Histogrammes et densités seront détaillés en Chapter 7.

6.11 Comparaison de groupes

```
ventes |>
  group_by(region) |>
  summarise(
    n = n(),
    ca_moyen = mean(montant),
    ca_median = median(montant),
    ca_sd = sd(montant),
    .groups = "drop"
  )
```

```
#> # A tibble: 4 x 5
#>   region     n ca_moyen ca_median ca_sd
#>   <chr>  <int>    <dbl>     <dbl> <dbl>
#> 1 Est       50     143.     123.  120.
#> 2 Nord      47     108.      62.8  98.2
#> 3 Ouest     59     129.     108.   92.2
#> 4 Sud       42     143.     112.  109.
```

Comparez toujours **n**, tendance centrale **et** dispersion : deux moyennes proches avec des écarts-types très différents ne racontent pas la même histoire.

6.12 Tableaux croisés

```
demo |>
  count(sexe, niveau_etudes) |>
  pivot_wider(names_from = sexe, values_from = n, values_fill = 0)
```

```
#> # A tibble: 5 x 3
#>   niveau_etudes Femme Homme
#>   <chr>         <int> <int>
#> 1 Autre            10     9
#> 2 Bac              18    20
#> 3 Doctorat          3     5
#> 4 Licence          26    33
#> 5 Master           12    12
```

```
# Avec proportions par ligne
tab <- table(demo$sexe, demo$niveau_etudes)
prop.table(tab, margin = 1) |> round(2)
```

```
#>
```

```
#>         Autre  Bac Doctorat Licence Master
#>   Femme  0.14 0.26     0.04    0.38   0.17
#>   Homme  0.11 0.25     0.06    0.42   0.15
```

6.13 Corrélation entre variables

```
# Pearson (linéaire)
cor(demo$age, demo$revenu_mensuel, use = "complete.obs")
```

```
#> [1] 0.01489224
```

```
cor(mtcars$wt, mtcars$mpg)
```

```
#> [1] -0.8676594
```

```
mtcars |>
  select(mpg, wt, hp, disp) |>
  cor() |>
  round(2)
```

```
#>        mpg    wt    hp  disp
#> mpg   1.00 -0.87 -0.78 -0.85
#> wt   -0.87  1.00  0.66  0.89
#> hp   -0.78  0.66  1.00  0.79
#> disp -0.85  0.89  0.79  1.00
```

> **!** Corrélation causalité
>
> Une corrélation forte indique une association statistique, **pas** qu'une variable cause l'autre. Un facteur tiers peut expliquer les deux.

6.14 Interprétation en langage courant

Exemple : `cor(mtcars$wt, mtcars$mpg)` est négatif et marqué.

Résultat observé : les voitures plus lourdes tendent à avoir une consommation en miles/gallon plus faible dans cet échantillon.

Interprétation prudente : il existe une association linéaire négative entre poids et `mpg` dans `mtcars`. Cela ne prouve pas, à elle seule, le mécanisme causal (moteur, aérodynamique, etc.).

6.15 Limites et erreurs d'interprétation

1. Décrire un échantillon n'équivaut pas à conclure sur toute une population (→ inférence, Chapter 8).
2. La moyenne seule cache multimodalité et outliers.
3. Des catégories déséquilibrées faussent les lectures de proportions.
4. Corréler deux séries temporelles sans précaution peut créer des artefacts.
5. Multiplier les tris jusqu'à trouver un résultat « significatif » (data dredging).

6.16 Démonstration complète

```
set.seed(2024)
rapport <- list(
```

```
  ca_total = sum(ventes$montant),
  ca_moyen = mean(ventes$montant),
  ca_median = median(ventes$montant),
  top_produit = ventes |> count(produit, sort = TRUE) |> slice(1),
  cor_qte_montant = cor(ventes$quantite, ventes$montant)
)
rapport
```

```
#> $ca_total
#> [1] 25853.53
#>
#> $ca_moyen
#> [1] 130.5734
#>
#> $ca_median
#> [1] 105.875
#>
#> $top_produit
#> # A tibble: 1 x 2
#>   produit      n
#>   <chr>    <int>
#> 1 Marqueur    43
#>
#> $cor_qte_montant
#> [1] 0.6708037
```

6.17 Exercices

6.17.1 Niveau 1

1. Calculez moyenne et médiane de `demo$age`.
2. Affichez la table des effectifs de `niveau_etudes`.

6.17.2 Niveau 2

3. Comparez le revenu moyen et médian par `sexe`.
4. Calculez l'IQR de `montant` dans `ventes`.

6.17.3 Niveau 3

5. Produisez un tableau région × produit avec le CA total (`sum(montant)`).
6. Interprétez en une phrase `cor(mtcars$hp, mtcars$mpg)`.

6.18 Corrections

```
mean(demo$age); median(demo$age)
```

```
#> [1] 46.64189
```

```
#> [1] 47
```

```
demo |> count(niveau_etudes, sort = TRUE)
```

```
#> # A tibble: 5 x 2
```

```
#>   niveau_etudes     n
#>   <chr>         <int>
#> 1 Licence          59
#> 2 Bac              38
#> 3 Master           24
#> 4 Autre            19
#> 5 Doctorat          8
```

```
demo |>
  group_by(sexe) |>
  summarise(moy = mean(revenu_mensuel), med = median(revenu_mensuel), .groups = "drop")
```

```
#> # A tibble: 2 x 3
#>   sexe    moy   med
#>   <chr> <dbl> <dbl>
#> 1 Femme 2204.  2215
#> 2 Homme 2159.  2146
```

```
IQR(ventes$montant)
```

```
#> [1] 153.65
```

```
ventes |>
  group_by(region, produit) |>
  summarise(ca = sum(montant), .groups = "drop") |>
  pivot_wider(names_from = produit, values_from = ca, values_fill = 0)
```

```
#> # A tibble: 4 x 6
#>   region Agenda Cahier Classeur Marqueur Stylo
#>   <chr>   <dbl>  <dbl>    <dbl>    <dbl> <dbl>
#> 1 Est     1757.  1443.    1228.    1418. 1295.
#> 2 Nord     282.  1560.    1201.     705. 1349.
#> 3 Ouest   1874.  1425.    1726.    1115. 1482.
#> 4 Sud      418.   934.     824.    2618. 1201.
```

```
cor(mtcars$hp, mtcars$mpg)
```

```
#> [1] -0.7761684
```

```
# Interprétation : association linéaire négative - plus de chevaux, mpg souvent plus faible dans cet éch
```

6.19 Points clés

- Décrire = résumer sans sur-interpréter.
- Médiane et IQR sont robustes ; moyenne et écart-type sont sensibles aux extrêmes.
- Toute corrélation demande une phrase de prudence sur la causalité.

6.20 Ressources

- Çetinkaya-Rundel and Hardin (2023), Wickham, Çetinkaya-Rundel, and Grolemund (2023) (parties EDA)

6.21 Suite

Chapter 7 : donner forme visuelle à ces résumés avec ggplot2.

Part VI

Partie VI — Visualisation

Chapter 7

Visualisation des données

7.1 Introduction

Un bon graphique révèle des patterns que les tableaux peinent à montrer. Ce chapitre présente les principes de lisibilité et la **grammaire des graphiques** de `ggplot2` : barres, histogrammes, boîtes à moustaches, nuages de points, lignes, facettes et export.

7.2 Objectifs pédagogiques

- Choisir un type de graphique adapté à la question
- Construire des graphiques ggplot2 lisibles
- Personnaliser titres, axes, couleurs et facettes
- Exporter des figures pour un rapport
- Éviter les erreurs courantes de représentation

7.3 Prérequis

3. Package `ggplot2` (tidyverse).

```
library(tidyverse)
library(scales)

ventes <- read_csv("data/ventes.csv", show_col_types = FALSE) |>
  filter(!is.na(region))
meteo <- read_csv("data/meteo.csv", show_col_types = FALSE) |>
  mutate(date = as.Date(date))
demo <- read_csv("data/demographie.csv", show_col_types = FALSE) |>
  filter(age <= 100, revenu_mensuel < 20000)
```

7.4 Pourquoi visualiser ?

- Détecter outliers, asymétries, clusters
- Comparer des groupes rapidement
- Communiquer à un public non statisticien

Vocabulaire ggplot2

- **aes** : mapping esthétique (x, y, color, fill, size…)
- **geom** : type de marque (barre, point, ligne…)
- **scale** : échelle (couleurs, axes)
- **facet** : petits multiples
- **theme** : apparence (fond, grilles, polices)

7.5 Principes de lisibilité

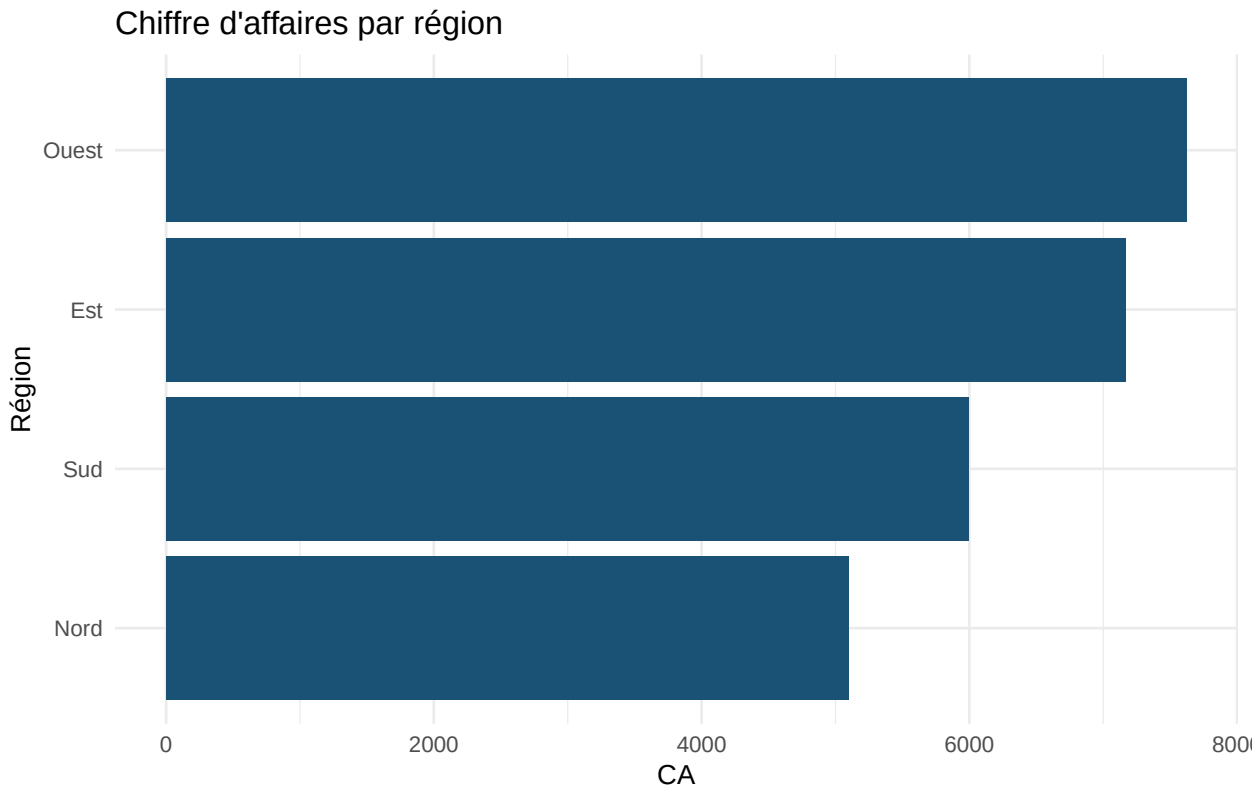
1. Un graphique = **un message** principal
2. Axes légendés avec unités
3. Couleurs accessibles (contraste, pas seulement rouge/vert)
4. Pas de 3D inutile ni de dual-axis trompeur
5. Titre informatif (« CA par région en 2023 ») plutôt que décoratif

7.6 Diagrammes en barres

Objectif : comparer des quantités entre catégories.

```
ca_region <- ventes |>
  group_by(region) |>
  summarise(ca = sum(montant), .groups = "drop")

ggplot(ca_region, aes(x = reorder(region, ca), y = ca)) +
  geom_col(fill = "#1a5276") +
  coord_flip() +
  labs(
    title = "Chiffre d'affaires par région",
    x = "Région", y = "CA"
  ) +
  theme_minimal()
```

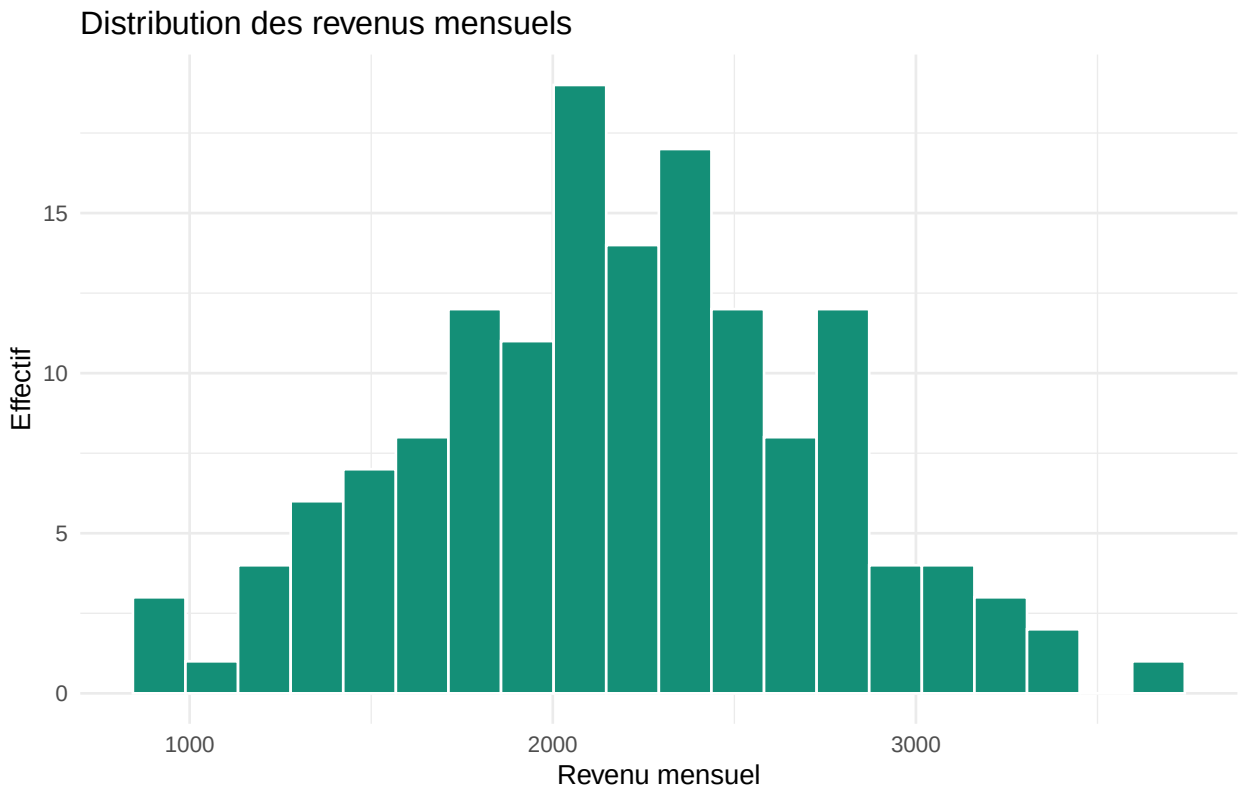


Interprétation : la longueur des barres montre le CA total ; le tri facilite le classement.

7.7 Histogrammes

Objectif : voir la distribution d'une variable quantitative.

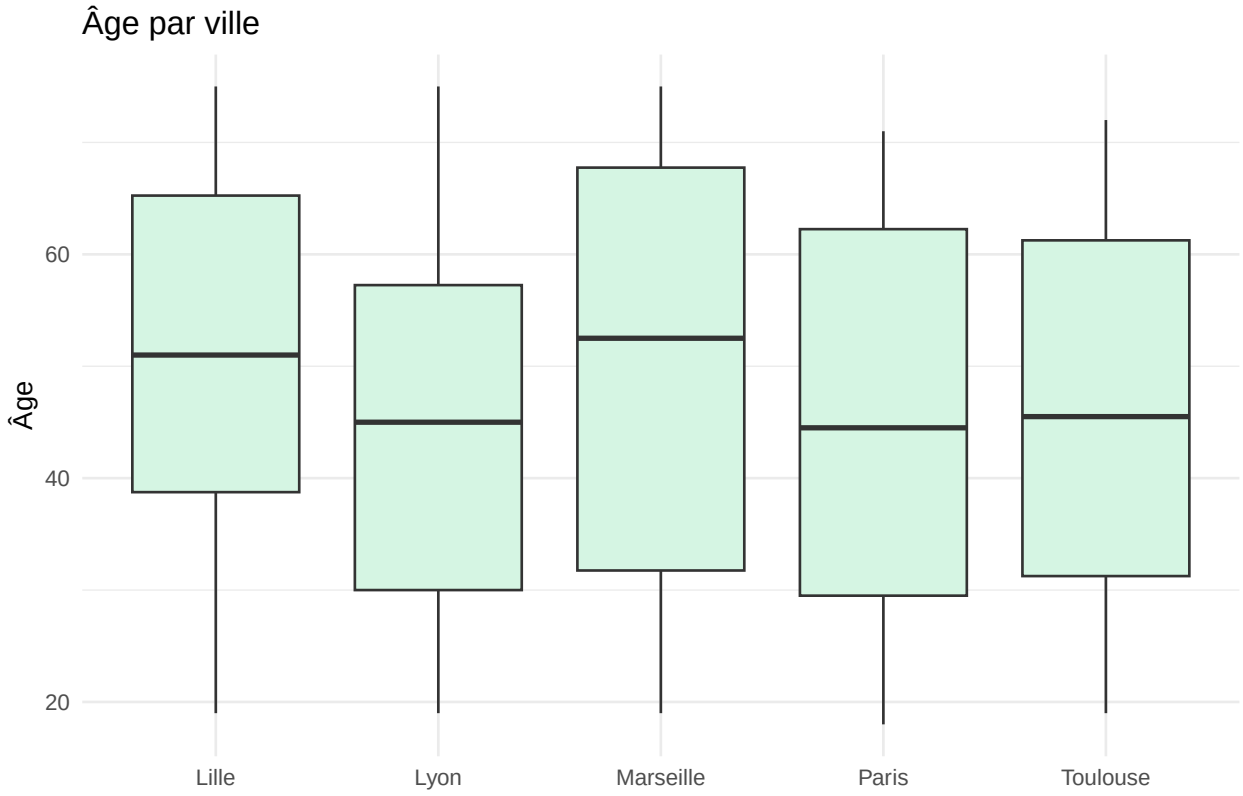
```
ggplot(demo, aes(x = revenu_mensuel)) +
  geom_histogram(bins = 20, fill = "#148f77", color = "white") +
  labs(
    title = "Distribution des revenus mensuels",
    x = "Revenu mensuel", y = "Effectif"
  ) +
  theme_minimal()
```



7.8 Boîtes à moustaches

Objectif : comparer distributions entre groupes (médiane, IQR, extrêmes).

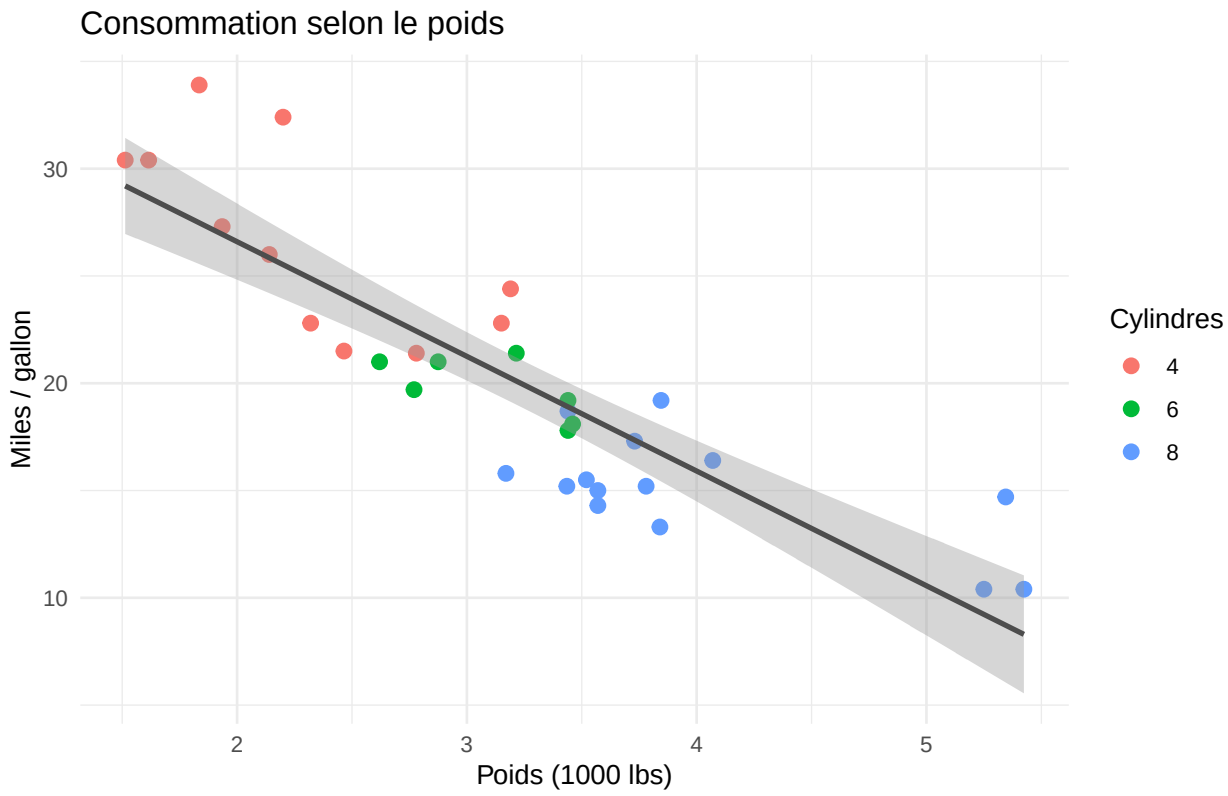
```
ggplot(demo, aes(x = ville, y = age)) +
  geom_boxplot(fill = "#d5f5e3") +
  labs(title = "Âge par ville", x = NULL, y = "Âge") +
  theme_minimal()
```



7.9 Graphiques de dispersion

Objectif : relation entre deux quantitatives.

```
ggplot(mtcars, aes(x = wt, y = mpg)) +
  geom_point(aes(color = factor(cyl)), size = 2.5) +
  geom_smooth(method = "lm", se = TRUE, color = "gray30") +
  labs(
    title = "Consommation selon le poids",
    x = "Poids (1000 lbs)", y = "Miles / gallon",
    color = "Cylindres"
  ) +
  theme_minimal()
```



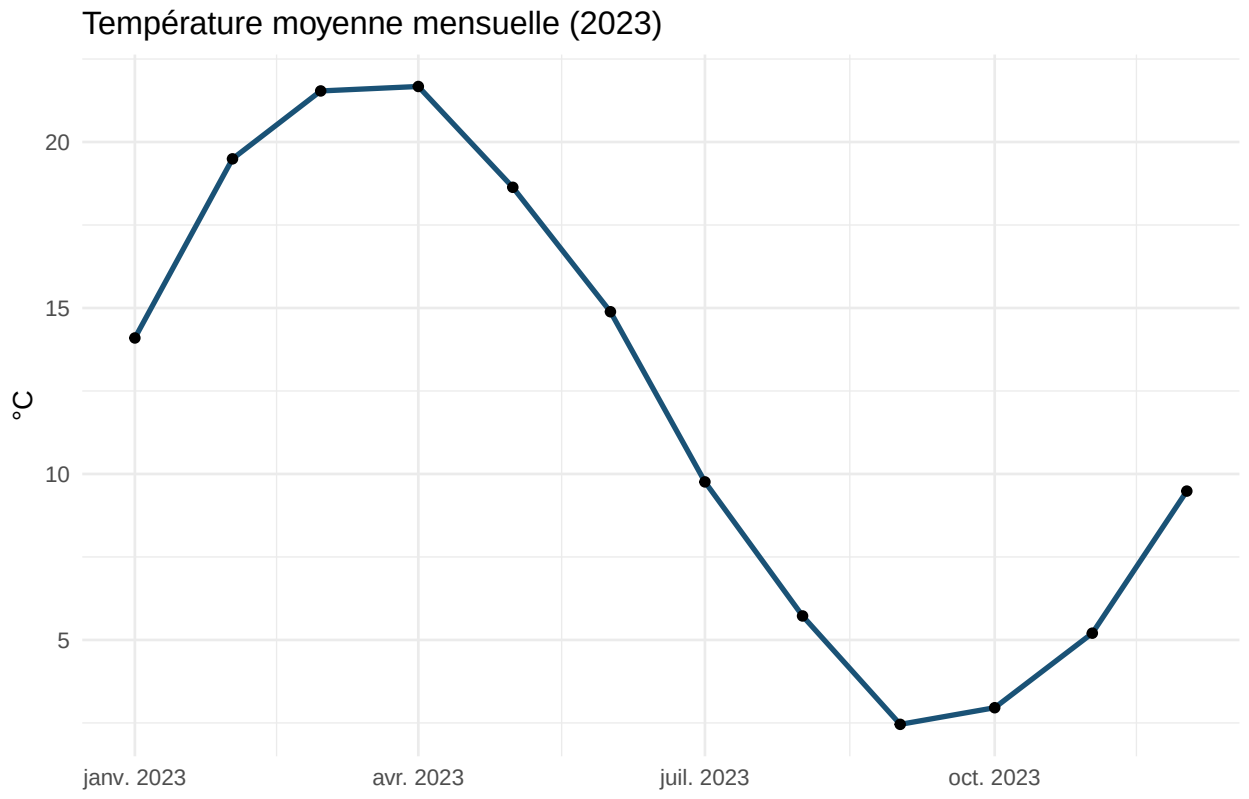
Interprétation : tendance négative ; le lissage linéaire aide à voir la pente, sans prouver la causalité.

7.10 Graphiques linéaires

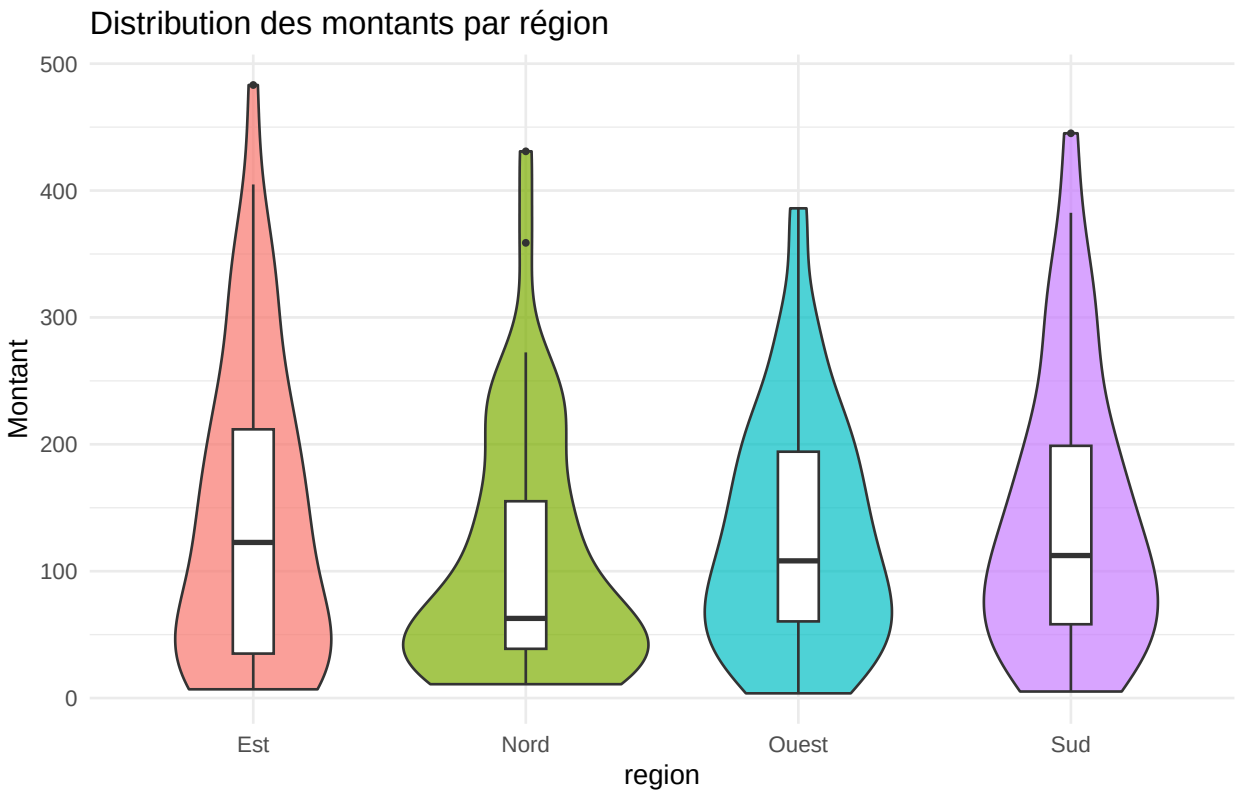
Objectif : évolution dans le temps.

```
meteo_mois <- meteo |>
  mutate(mois = floor_date(date, "month")) |>
  group_by(mois) |>
  summarise(temp = mean(temperature_c), .groups = "drop")

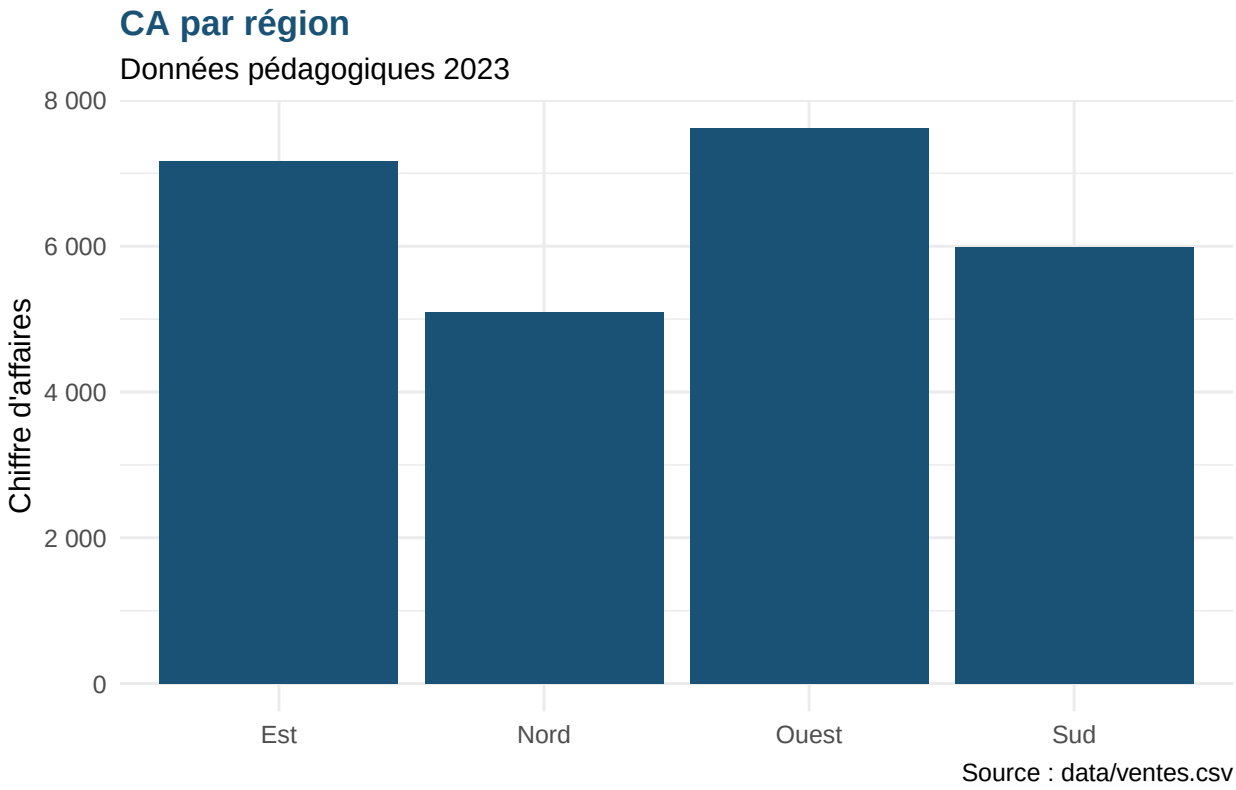
ggplot(meteo_mois, aes(x = mois, y = temp)) +
  geom_line(color = "#1a5276", linewidth = 1) +
  geom_point() +
  labs(
    title = "Température moyenne mensuelle (2023)",
    x = NULL, y = "°C"
  ) +
  theme_minimal()
```



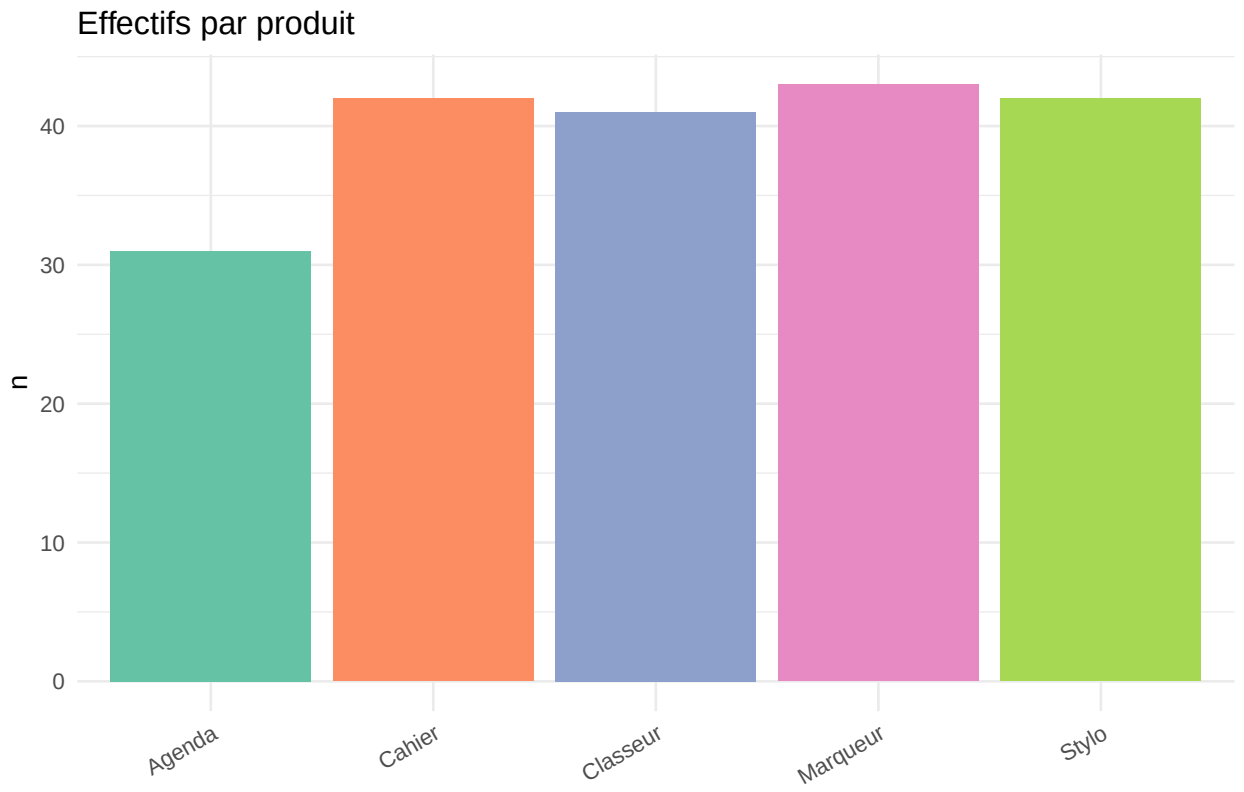

```
ggplot(ventes, aes(x = region, y = montant, fill = region)) +
  geom_violin(alpha = 0.7, show.legend = FALSE) +
  geom_boxplot(width = 0.15, fill = "white", outlier.size = 0.8) +
  labs(title = "Distribution des montants par région", y = "Montant") +
  theme_minimal()
```



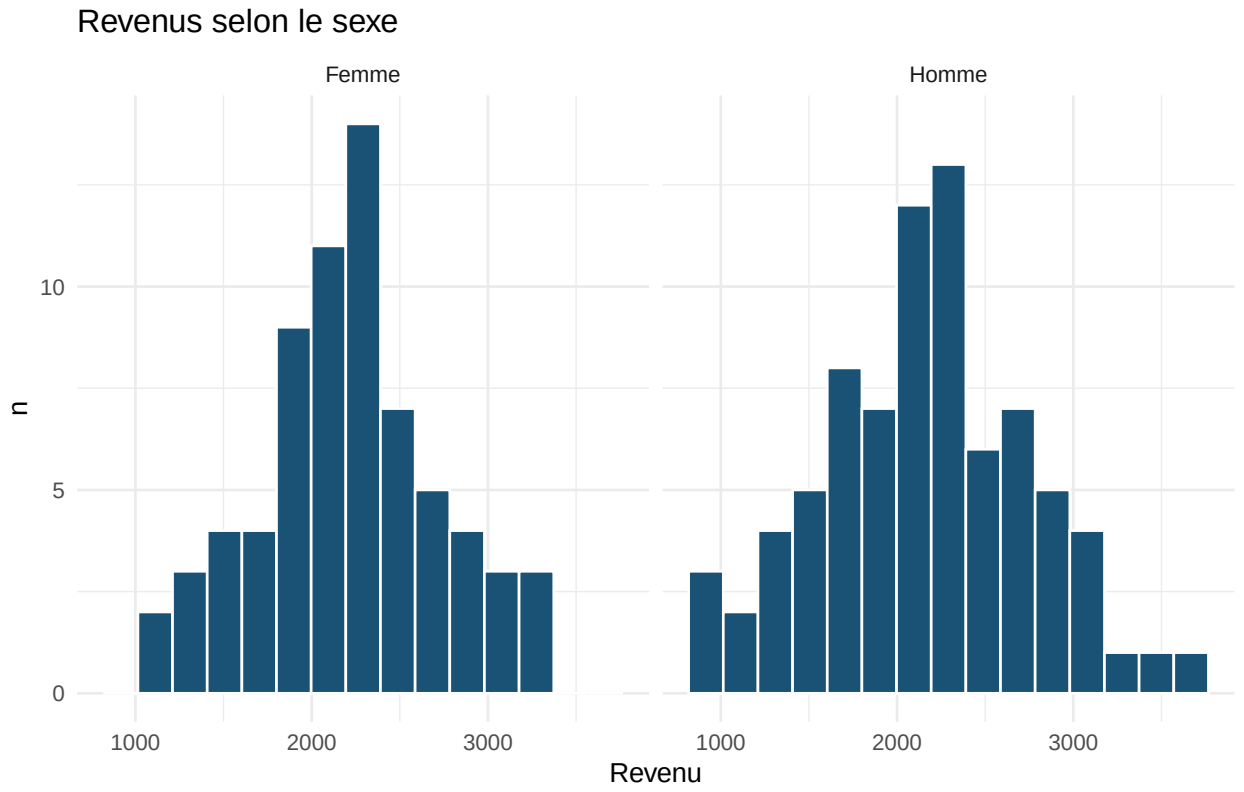

```r
ggplot(ca_region, aes(x = region, y = ca)) +
  geom_col(fill = "#1a5276") +
  scale_y_continuous(labels = label_number(big.mark = " ")) +
  labs(
    title = "CA par région",
    subtitle = "Données pédagogiques 2023",
    caption = "Source : data/ventes.csv",
    x = NULL, y = "Chiffre d'affaires"
  ) +
  theme_minimal(base_size = 12) +
  theme(plot.title = element_text(face = "bold", color = "#1a5276"))
```




```r
ggplot(ventes, aes(x = produit, fill = produit)) +
  geom_bar(show.legend = FALSE) +
  scale_fill_brewer(palette = "Set2") +
  labs(title = "Effectifs par produit", x = NULL, y = "n") +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 30, hjust = 1))
```




```r
ggplot(demo, aes(x = revenu_mensuel)) +
  geom_histogram(bins = 15, fill = "#1a5276", color = "white") +
  facet_wrap(~sexe) +
  labs(title = "Revenus selon le sexe", x = "Revenu", y = "n") +
  theme_minimal()
```



7.15 Exportation

```
p <- ggplot(ca_region, aes(region, ca)) + geom_col()
ggsave("outputs/ca_region.png", p, width = 7, height = 4, dpi = 150)
```

```
dir.create("outputs", showWarnings = FALSE)
p <- ggplot(ca_region, aes(region, ca)) +
  geom_col(fill = "#1a5276") +
  theme_minimal()
ggsave("outputs/ca_region.png", p, width = 7, height = 4, dpi = 120)
file.exists("outputs/ca_region.png")
```

```
#> [1] TRUE
```

7.16 Grammaire ggplot2 (rappel)

```
ggplot(donnees, aes(...)) +
  geom_*() +
  scale_*() +
  facet_*() +
  labs() +
  theme_*()
```

7.17 Erreurs courantes

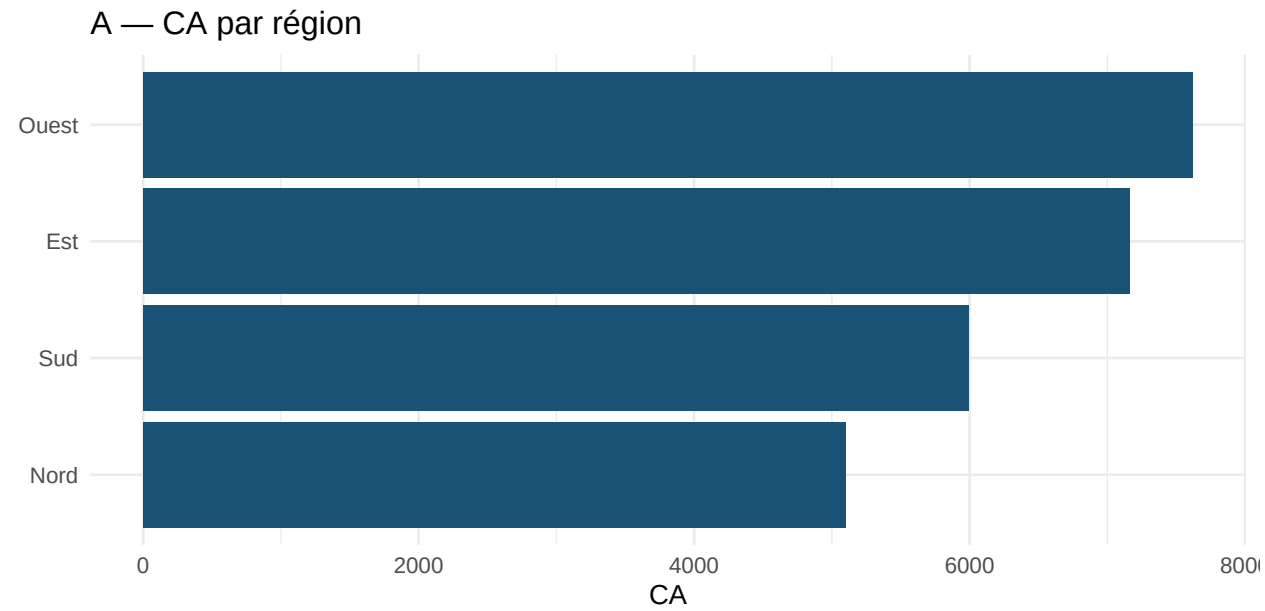
Erreur	Pourquoi c'est problématique
Camembert à 10 catégories	Illisible ; préférer barres
Tronquer l'axe Y sans le dire	Exagère les différences
Trop de couleurs / 3D	Charge cognitive, biais
Nuage de points surabondant	Surplotting → `alpha`, hexbins
Titre vague (« Graphique 1 »)	Aucune information

7.18 Mini-collection pour un rapport

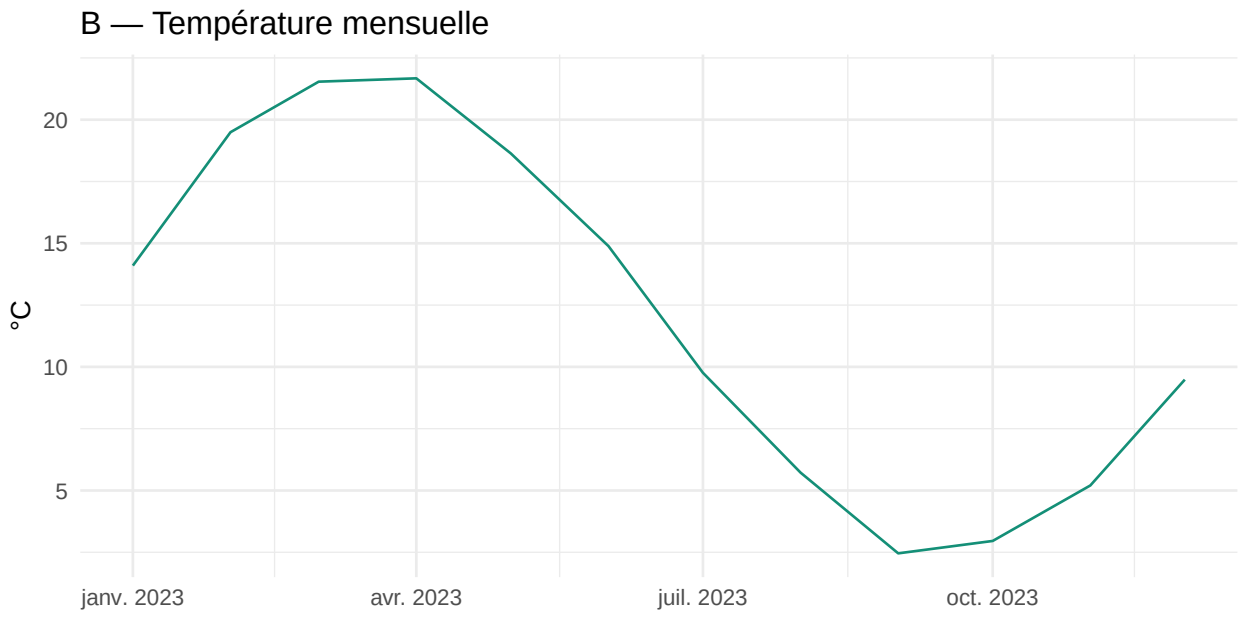
```
p1 <- ggplot(ca_region, aes(reorder(region, ca), ca)) +
  geom_col(fill = "#1a5276") + coord_flip() +
  labs(title = "A - CA par région", x = NULL, y = "CA") + theme_minimal()

p2 <- ggplot(meteo_mois, aes(mois, temp)) +
  geom_line(color = "#148f77") +
  labs(title = "B - Température mensuelle", x = NULL, y = "°C") + theme_minimal()

p1
```




```
p2
```



7.19 Exercices

7.19.1 Niveau 1

1. Histogramme de `temperature_c` (meteo).
2. Barres du nombre de ventes par `produit`.

7.19.2 Niveau 2

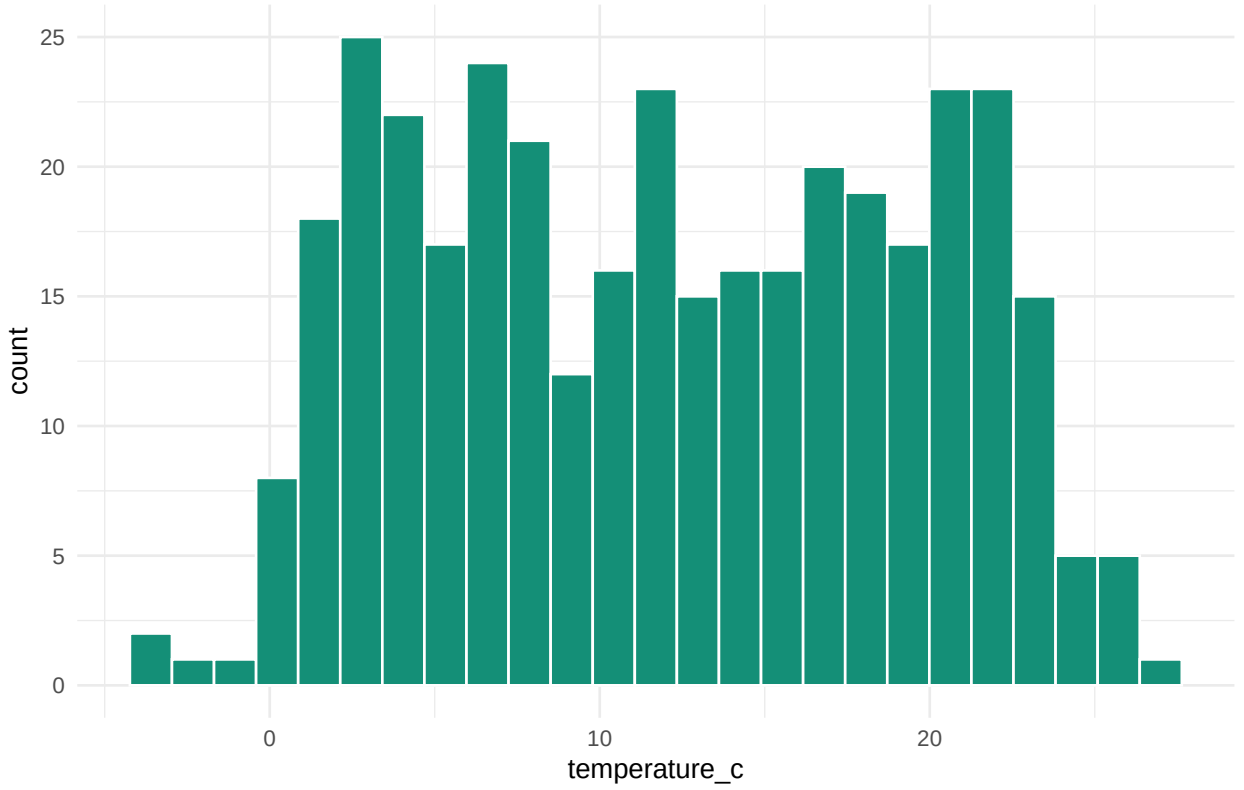
3. Scatter `age` vs `revenu_mensuel` coloré par `sexe`.
4. Boxplot de `montant` par `produit`.

7.19.3 Niveau 3

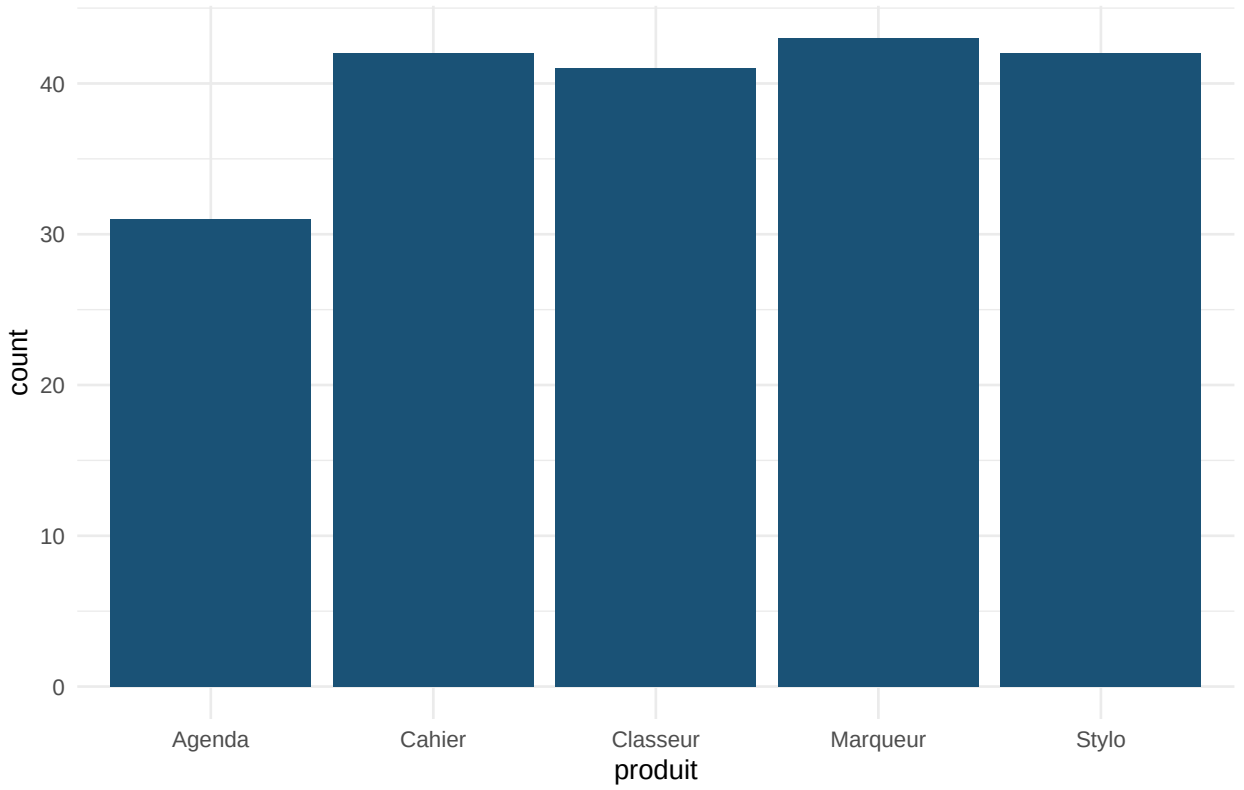
5. Facette de l'histogramme des montants par région.
6. Exportez un graphique en PNG dans `outputs/`.

7.20 Corrections

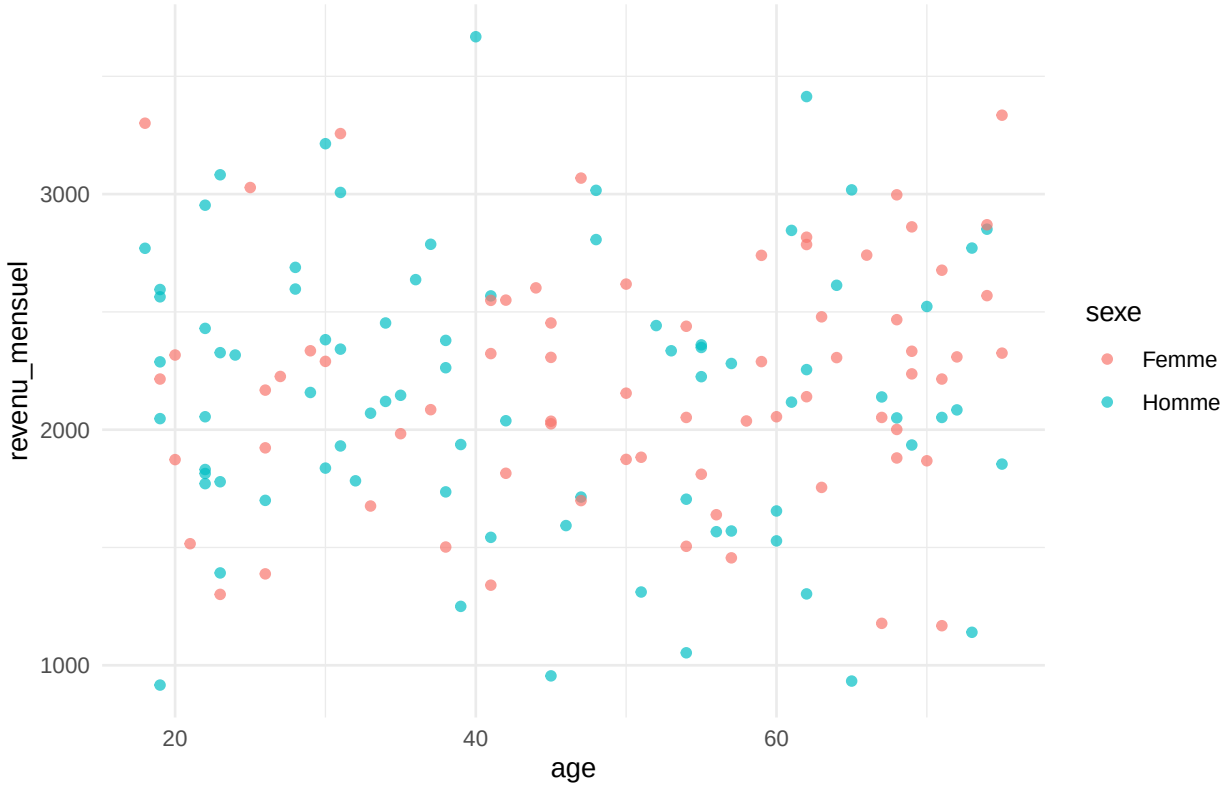
```
ggplot(meteo, aes(temperature_c)) +
  geom_histogram(bins = 25, fill = "#148f77", color = "white") +
  theme_minimal()
```



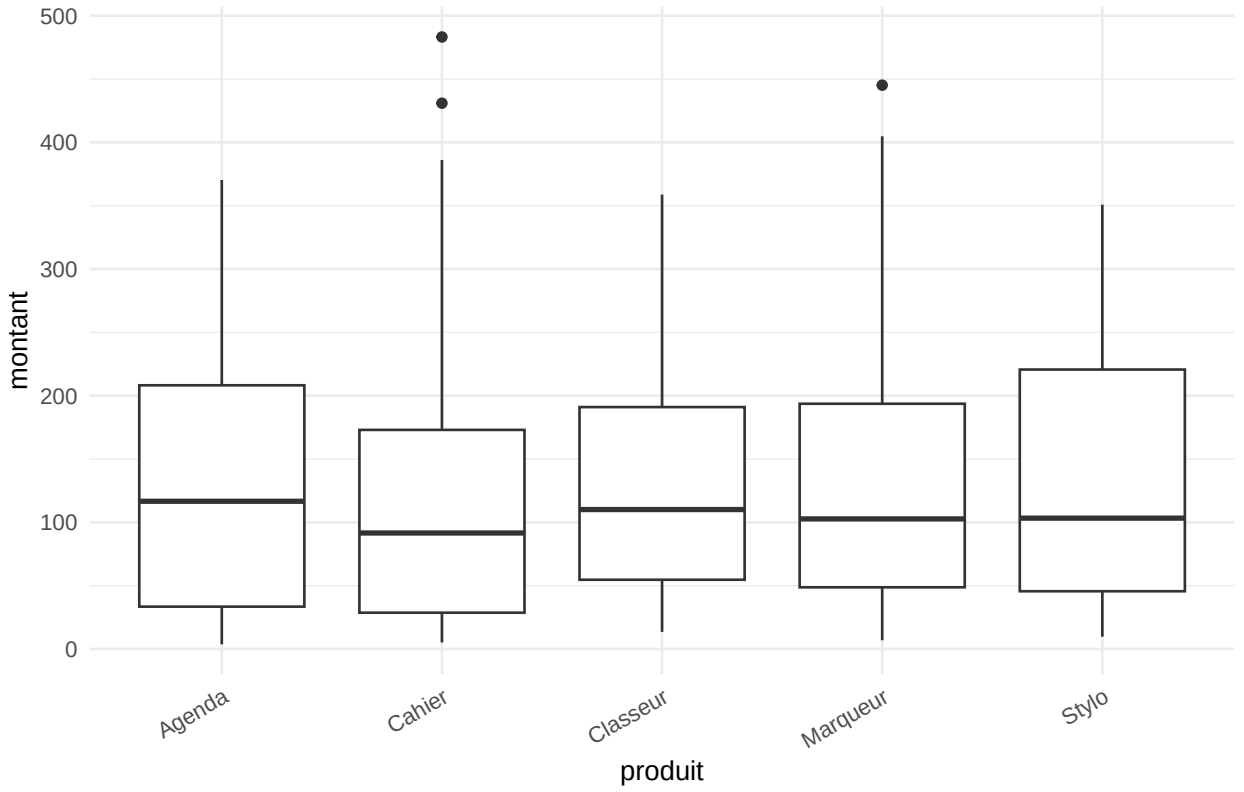

```r
ggplot(ventes, aes(produit)) + geom_bar(fill = "#1a5276") + theme_minimal()
```



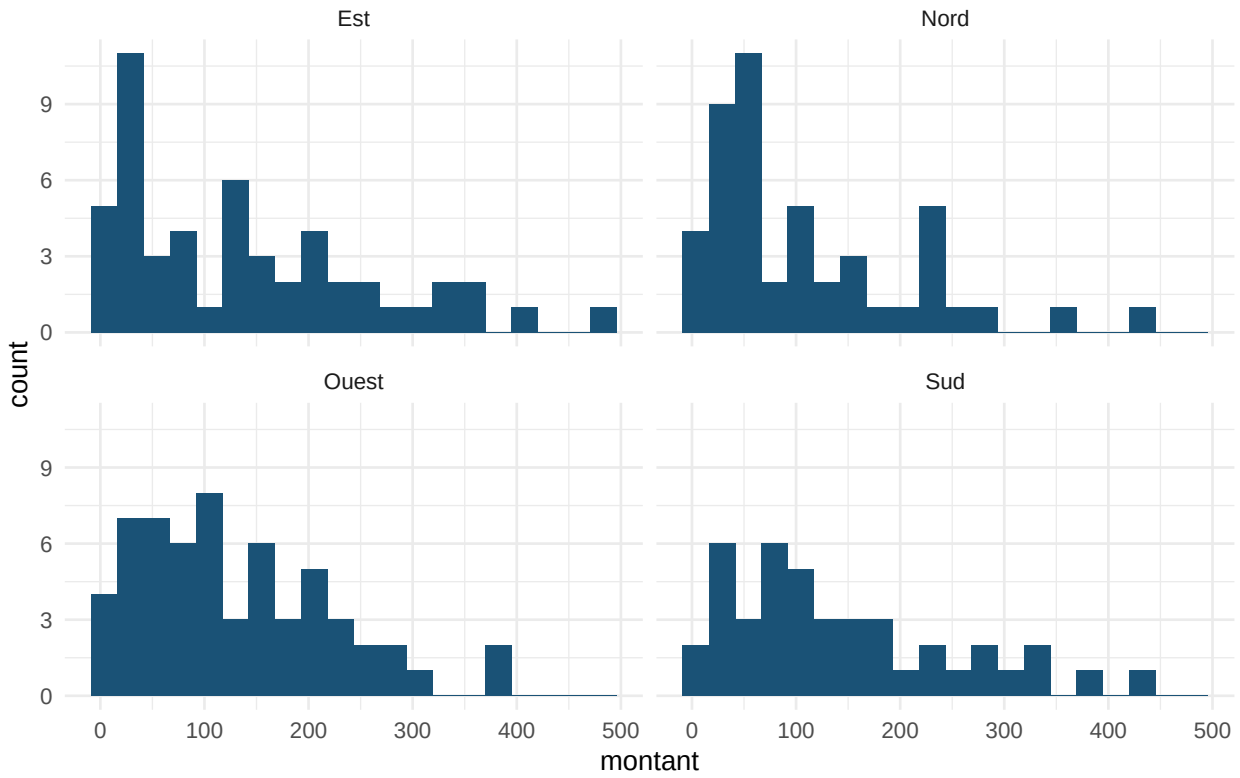
```
ggplot(demo, aes(age, revenu_mensuel, color = sexe)) +
  geom_point(alpha = 0.7) + theme_minimal()
```




```
ggplot(ventes, aes(produit, montant)) +
  geom_boxplot() +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 30, hjust = 1))
```




```
ggplot(ventes, aes(montant)) +
  geom_histogram(bins = 20, fill = "#1a5276") +
  facet_wrap(~region) +
  theme_minimal()
```



7.21 Points clés

- Le type de geom suit le type de variable et la question.
- `ggplot2` compose des couches ; gardez le message simple.
- Toujours légender et contextualiser (source, unités, période).

7.22 Ressources

- Wickham (2016), https://ggplot2.tidyverse.org/

7.23 Suite

Chapter 8 : passer de la description à l'inférence et à la régression.

Part VII

Partie VII — Modélisation

Chapter 8

Analyse statistique et modélisation

8.1 Introduction

La **statistique inférentielle** cherche à tirer des conclusions sur une population à partir d'un échantillon, en quantifiant l'incertitude. Ce chapitre introduit population/échantillon, intervalles de confiance, tests d'hypothèses, corrélation, régression linéaire et ses limites — notamment la distinction **corrélation / causalité**.

8.2 Objectifs pédagogiques

- Distinguer description et inférence
- Interpréter un intervalle de confiance et une p-valeur avec prudence
- Comparer des moyennes
- Ajuster et diagnostiquer une régression linéaire
- Évaluer les performances et les limites d'un modèle

8.3 Prérequis

3 et Chapter 7. Package `broom`.

```
library(tidyverse)
library(broom)

etudes <- read_csv("data/etudes_notes.csv", show_col_types = FALSE)
demo <- read_csv("data/demographie.csv", show_col_types = FALSE) |>
  filter(age <= 100, revenu_mensuel < 20000)
```

8.4 Population et échantillon

- **Population** : ensemble cible (tous les clients, tous les étudiants…)
- **Échantillon** : sous-ensemble observé
- **Paramètre** : caractéristique de la population (souvent inconnue)
- **Statistique** : caractéristique calculée sur l'échantillon

Vocabulaire

- **Estimateur** : règle de calcul (ex. moyenne empirique)
- **Biais** : erreur systématique
- **Variance d'échantillonnage** : variabilité d'un estimateur entre échantillons

8.5 Variables quantitatives et qualitatives

Type	Exemples	Analyses typiques
Quantitative continue	revenu, température	moyenne, régression
Quantitative discrète	nombre d'achats	effectifs, Poisson (avancé)
Qualitative nominale	région, sexe	tableaux, chi-2
Qualitative ordinale	niveau d'études	rangs, tendances

8.6 Échantillonnage (principes)

Un échantillon **aléatoire** simple donne à chaque unité la même chance d'être tirée. Les biais de sélection (volontaires uniquement, survivants…) invalident souvent l'inférence.

8.7 Probabilités et distributions de base

```
# Loi normale : densité
x <- seq(-3, 3, length.out = 200)
dens <- dnorm(x, mean = 0, sd = 1)

# Probabilité P(Z <= 1.96)  0.975
pnorm(1.96)
```

```
#> [1] 0.9750021
```

```
# Quantile 97.5 %
qnorm(0.975)
```

```
#> [1] 1.959964
```

Beaucoup de moyennes d'échantillons se rapprochent d'une normale lorsque n est assez grand (théorème central limite) — sous conditions.

8.8 Intervalles de confiance

```
# IC à 95 % pour la moyenne du revenu (approx. normale)
n <- nrow(demo)
m <- mean(demo$revenu_mensuel)
s <- sd(demo$revenu_mensuel)
erreur <- qt(0.975, df = n - 1) * s / sqrt(n)
c(bas = m - erreur, haut = m + erreur)
```

```
#>      bas     haut
#> 2090.178 2269.281
```

Interprétation correcte : la procédure à 95 % couvre le vrai paramètre dans 95 % des échantillons répétés. Ce n'est **pas** « la probabilité que le paramètre soit dans cet intervalle » au sens bayésien naïf.

8.9 Tests d'hypothèses (introduction)

1. Hypothèse nulle (H_0) (souvent : « pas d'effet / pas de différence »)
2. Hypothèse alternative (H_1)
3. Statistique de test + **p-valeur**
4. Décision au seuil (souvent 5 %), **avec contexte**

> ⚠ La p-valeur n'est pas…
>
> - la probabilité que (H_0) soit vraie
> - la taille de l'effet
> - une preuve de causalité
>
> Une p-valeur faible suggère que des données aussi extrêmes sont rares si (H_0) est vraie — pas plus.

8.10 Comparaison de moyennes

```r
# Revenu Homme vs Femme
hommes <- demo$revenu_mensuel[demo$sexe == "Homme"]
femmes <- demo$revenu_mensuel[demo$sexe == "Femme"]

t.test(hommes, femmes, var.equal = FALSE)
```

```
#>
#>  Welch Two Sample t-test
#>
#> data:  hommes and femmes
#> t = -0.50217, df = 146, p-value = 0.6163
#> alternative hypothesis: true difference in means is not equal to 0
#> 95 percent confidence interval:
#>  -223.6085  132.9976
#> sample estimates:
#> mean of x mean of y
#>  2158.608  2203.913
```

Vérifiez n, normalité approximative / robustesse, et l'écart **pratique** (différence de moyennes), pas seulement la significativité.

8.11 Corrélation et association

```r
cor.test(etudes$heures_etude, etudes$note_examen)
```

```
#>
#>  Pearson's product-moment correlation
#>
#> data:  etudes$heures_etude and etudes$note_examen
#> t = 17.669, df = 98, p-value < 2.2e-16
```

```
#> alternative hypothesis: true correlation is not equal to 0
#> 95 percent confidence interval:
#>  0.8157770 0.9124674
#> sample estimates:
#>       cor
#> 0.8724065
```

Pour deux qualitatives : test du chi-2 sur tableau croisé (`chisq.test`).

8.12 Régression linéaire simple

Modèle : (Y = _0 + _1 X +)

```
mod1 <- lm(note_examen ~ heures_etude, data = etudes)
summary(mod1)
```

```
#>
#> Call:
#> lm(formula = note_examen ~ heures_etude, data = etudes)
#>
#> Residuals:
#>      Min       1Q   Median       3Q      Max
#> -18.9041  -6.1094  -0.1172   5.9189  16.1833
#>
#> Coefficients:
#>              Estimate Std. Error t value Pr(>|t|)
#> (Intercept)   46.9101     1.5137   30.99   <2e-16 ***
#> heures_etude   2.2904     0.1296   17.67   <2e-16 ***
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
#>
#> Residual standard error: 7.371 on 98 degrees of freedom
#> Multiple R-squared:  0.7611, Adjusted R-squared:  0.7587
#> F-statistic: 312.2 on 1 and 98 DF,  p-value: < 2.2e-16
```

```
tidy(mod1)
```

```
#> # A tibble: 2 x 5
#>   term         estimate std.error statistic  p.value
#>   <chr>           <dbl>     <dbl>     <dbl>    <dbl>
#> 1 (Intercept)     46.9      1.51       31.0 1.94e-52
#> 2 heures_etude     2.29     0.130      17.7 3.14e-32
```

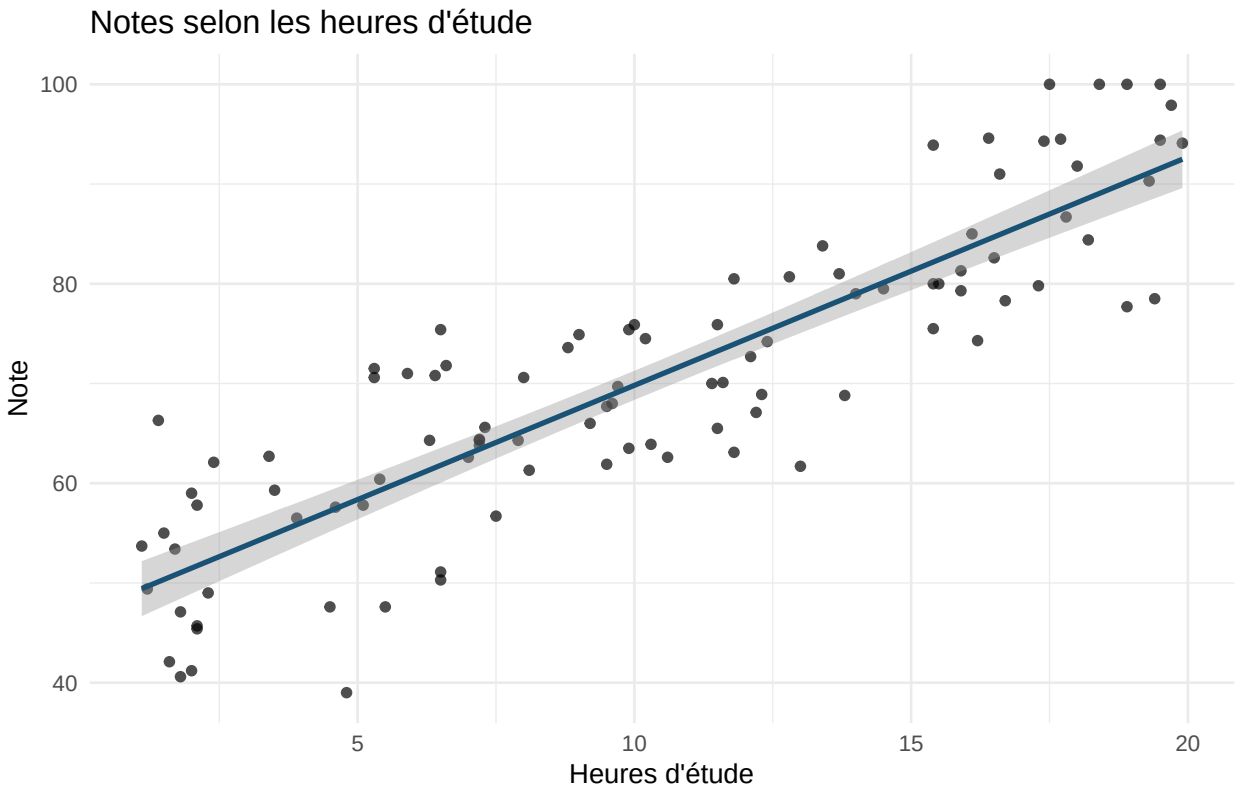
```
glance(mod1)
```

```
#> # A tibble: 1 x 12
#>   r.squared adj.r.squared sigma statistic  p.value    df logLik   AIC   BIC
#>       <dbl>         <dbl> <dbl>     <dbl>    <dbl> <dbl>  <dbl> <dbl> <dbl>
#> 1     0.761         0.759  7.37      312. 3.14e-32     1  -341.  687.  695.
#> # i 3 more variables: deviance <dbl>, df.residual <int>, nobs <int>
```

Lecture : la pente estime la variation moyenne de la note pour une heure d'étude supplémentaire **dans le modèle et l'échantillon**, toutes choses égales selon les hypothèses. (R^2) mesure la part de variance expliquée — pas la vérité causale.

```
ggplot(etudes, aes(heures_etude, note_examen)) +
  geom_point(alpha = 0.7) +
```

```
  geom_smooth(method = "lm", se = TRUE, color = "#1a5276") +
  labs(
    title = "Notes selon les heures d'étude",
    x = "Heures d'étude", y = "Note"
  ) +
  theme_minimal()
```



8.13 Régression linéaire multiple

```
mod2 <- lm(mpg ~ wt + hp + factor(cyl), data = mtcars)
tidy(mod2)
```

```
#> # A tibble: 5 x 5
#>   term         estimate std.error statistic  p.value
#>   <chr>           <dbl>     <dbl>     <dbl>    <dbl>
#> 1 (Intercept)   35.8       2.04       17.6  2.67e-16
#> 2 wt            -3.18      0.720      -4.42 1.44e- 4
#> 3 hp            -0.0231    0.0120     -1.93 6.36e- 2
#> 4 factor(cyl)6  -3.36      1.40       -2.40 2.37e- 2
#> 5 factor(cyl)8  -3.19      2.17       -1.47 1.54e- 1
```

```
glance(mod2)$r.squared
```

```
#> [1] 0.8572195
```

Attention à la **multicolinéarité** et au surajustement lorsque le nombre de prédicteurs devient grand face à n.

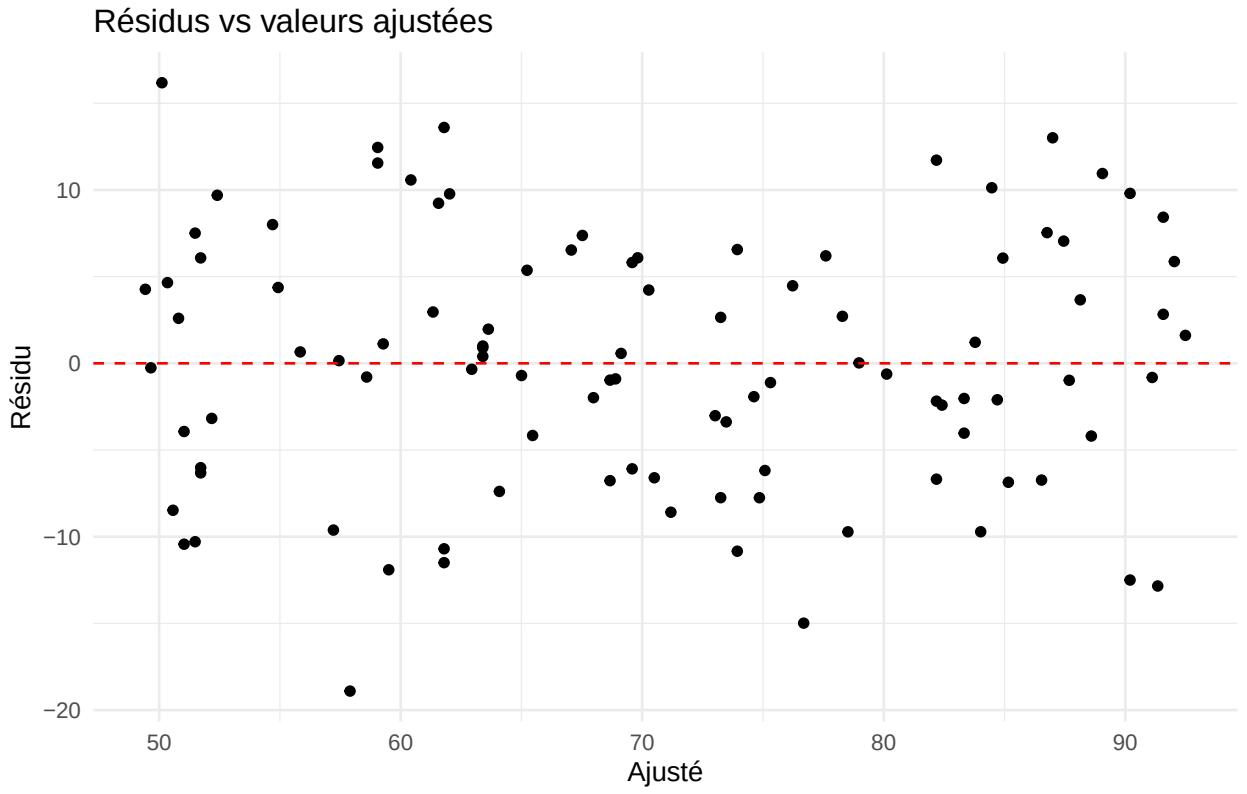
8.14 Hypothèses et résidus

Hypothèses classiques du modèle linéaire (simplifiées) :

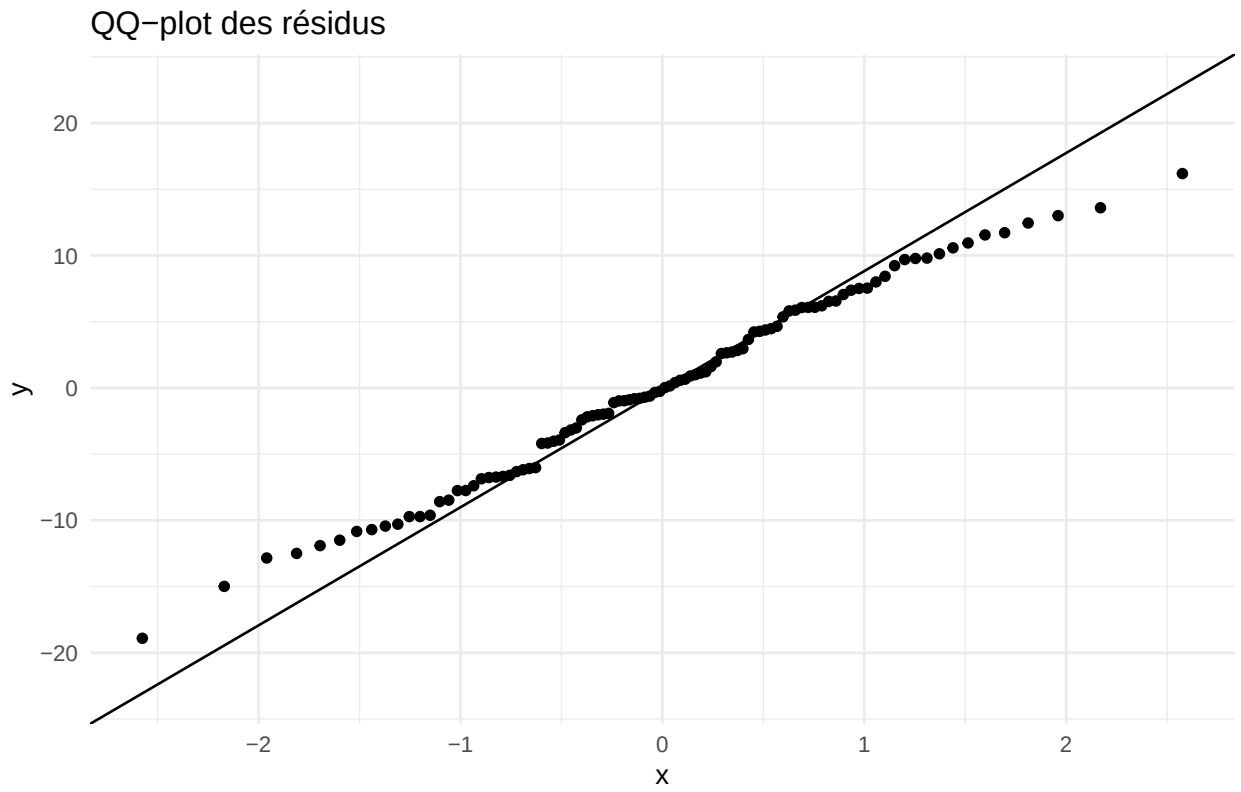
- Linéarité de la relation
- Erreurs indépendantes
- Variance approximativement constante (homoscédasticité)
- Normalité des erreurs (surtout pour inférence exacte à petit n)
- Pas de leviers/outliers trop influents sans examen

```
aug <- augment(mod1)

ggplot(aug, aes(.fitted, .resid)) +
  geom_point() +
  geom_hline(yintercept = 0, linetype = "dashed", color = "red") +
  labs(title = "Résidus vs valeurs ajustées", x = "Ajusté", y = "Résidu") +
  theme_minimal()
```




```
ggplot(aug, aes(sample = .resid)) +
  stat_qq() +
  stat_qq_line() +
  labs(title = "QQ-plot des résidus") +
  theme_minimal()
```



8.15 Évaluation des performances

```
# R² et R² ajusté
glance(mod1) |> select(r.squared, adj.r.squared, sigma, AIC)
```

```
#> # A tibble: 1 x 4
#>   r.squared adj.r.squared sigma   AIC
#>       <dbl>         <dbl> <dbl> <dbl>
#> 1     0.761         0.759  7.37  687.
```

```
# Erreur absolue moyenne (sur l'échantillon d'apprentissage - optimiste)
mean(abs(aug$.resid))
```

```
#> [1] 5.963408
```

Pour une évaluation plus honnête : séparer train/test ou validation croisée (niveau avancé).

8.16 Limites : corrélation versus causalité

Exemples de raisonnement incorrect :

- « Les ventes de glaces et les noyades corrélent → les glaces causent les noyades » (facteur tiers : chaleur)
- « Le modèle prédit bien → le levier d'action est prouvé » (sans design expérimental / inférence causale)

Les tests et régressions **éclairent** ; ils ne remplacent pas le raisonnement scientifique, le design d'étude et la connaissance du domaine.

> **!** Message clé du chapitre
>
> Significativité statistique ≠ importance pratique ≠ causalité.

8.17 Démonstration complète

```
mod_final <- lm(note_examen ~ heures_etude + filiere, data = etudes)
tidy(mod_final)
```

```
#> # A tibble: 4 x 5
#>   term            estimate std.error statistic  p.value
#>   <chr>              <dbl>     <dbl>     <dbl>    <dbl>
#> 1 (Intercept)      46.9        1.89    24.8    1.50e-43
#> 2 heures_etude      2.29       0.132   17.3    2.81e-31
#> 3 filiereLettres    0.0554     1.88     0.0294 9.77e- 1
#> 4 filiereSciences  -0.172      1.76    -0.0982 9.22e- 1
```

```
glance(mod_final)
```

```
#> # A tibble: 1 x 12
#>   r.squared adj.r.squared sigma statistic  p.value    df logLik   AIC   BIC
#>       <dbl>         <dbl> <dbl>     <dbl>    <dbl> <dbl>  <dbl> <dbl> <dbl>
#> 1     0.761         0.754  7.45      102. 9.77e-30     3  -341.  691.  704.
#> # i 3 more variables: deviance <dbl>, df.residual <int>, nobs <int>
```

```
# Prédiction pour 10 h d'étude, filière Sciences
predict(
  mod_final,
  newdata = data.frame(heures_etude = 10, filiere = "Sciences"),
  interval = "confidence"
)
```

```
#>        fit      lwr      upr
#> 1 69.68633 67.18396 72.18869
```

8.18 Exercices

8.18.1 Niveau 1

1. Calculez `cor(etudes$heures_etude, etudes$note_examen)`.
2. Que signifie (H_0) dans un test de moyenne égale à 0 pour une pente ?

8.18.2 Niveau 2

3. Ajustez `lm(revenu_mensuel ~ age + sexe, data = demo)` et lisez `tidy()`.
4. Produisez le graphique résidus vs ajustés.

8.18.3 Niveau 3

5. Comparez `mod1` (simple) et un modèle `note ~ heures_etude + filiere` via `AIC` / `adj.r.squared`.
6. Rédigez 5 lignes : ce que le modèle dit, ce qu'il ne dit pas.

8.19 Corrections

```
cor(etudes$heures_etude, etudes$note_examen)
```

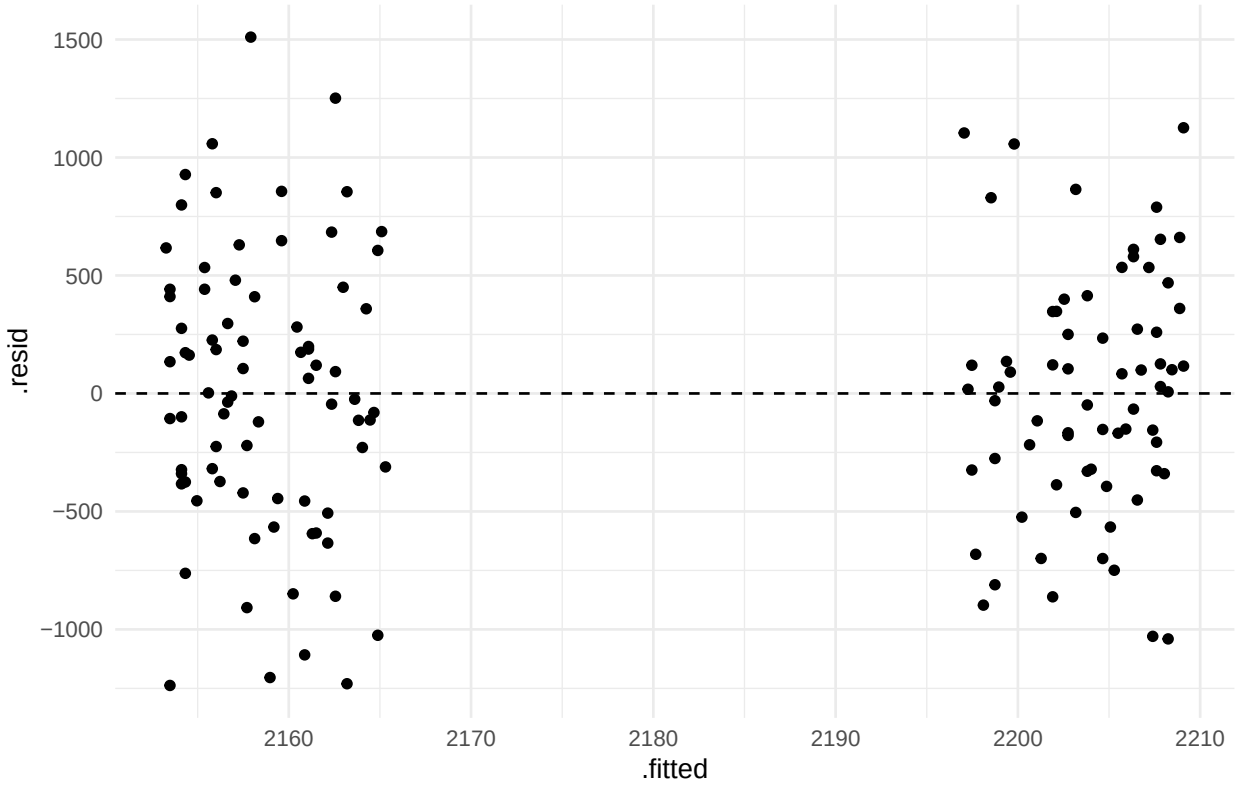
```
#> [1] 0.8724065
```

```
# H0 pour la pente : beta1 = 0 (pas d'association linéaire)
```

```
mod_d <- lm(revenu_mensuel ~ age + sexe, data = demo)
tidy(mod_d)
```

```
#> # A tibble: 3 x 5
#>   term        estimate std.error statistic  p.value
#>   <chr>          <dbl>     <dbl>     <dbl>    <dbl>
#> 1 (Intercept) 2193.       149.      14.7    1.68e-30
#> 2 age            0.211      2.64     0.0798 9.36e- 1
#> 3 sexeHomme    -43.8       93.4     -0.469  6.40e- 1
```

```
augment(mod_d) |>
  ggplot(aes(.fitted, .resid)) +
  geom_point() +
  geom_hline(yintercept = 0, linetype = "dashed") +
  theme_minimal()
```




```
m_a <- lm(note_examen ~ heures_etude, data = etudes)
m_b <- lm(note_examen ~ heures_etude + filiere, data = etudes)
glance(m_a) |> select(adj.r.squared, AIC)
```

```
#> # A tibble: 1 x 2
```

```
#>   adj.r.squared   AIC
#>           <dbl> <dbl>
#> 1         0.759  687.
```

```
glance(m_b) |> select(adj.r.squared, AIC)
```

```
#> # A tibble: 1 x 2
#>   adj.r.squared   AIC
#>           <dbl> <dbl>
#> 1         0.754  691.
```

8.20 Points clés

- Inférence = généraliser avec incertitude.
- IC, tests et régressions ont des hypothèses et des limites.
- Diagnostiquez les résidus ; méfiez-vous de la causalité spontanée.

8.21 Ressources

- Çetinkaya-Rundel and Hardin (2023), R Core Team (2024) (`?lm`, `?t.test`)
- `broom` pour des sorties tidy

8.22 Suite

Chapter 9 : communiquer analyses et graphiques avec Quarto.

Part VIII

Partie VIII — Rapports

Chapter 9

Rapports et communication des résultats

9.1 Introduction

Une analyse n'a de valeur que si elle est **compréhensible** et **reproductible**. Quarto (et R Markdown) permettent de mêler texte, code R et résultats dans un même document source, puis de générer HTML, PDF ou Word.

9.2 Objectifs pédagogiques

- Comprendre la logique des documents reproductibles
- Structurer un fichier `.qmd`
- Paramétrer l'exécution des blocs de code
- Générer HTML, PDF et DOCX
- Gérer bibliographie et mise en page
- Comparer Quarto et R Markdown

9.3 Prérequis

Avoir Quarto installé (Section 2.7). Notions de Chapter 7.

9.4 Documents reproductibles

Principe : **une source → plusieurs formats**. Modifier les données ou le code, puis recompiler, met à jour tables et figures automatiquement.

> **i** Vocabulaire
>
> - **YAML** : en-tête de métadonnées (`---` … `---`)
> - **Chunk / bloc** : segment de code exécutable
> - **Rendu (*render*)** : compilation vers HTML/PDF/DOCX
> - **Freeze / cache** : options pour accélérer les rebuilds

9.5 Présentation de Quarto

Quarto unifie R, Python, Julia et Markdown. Ce livre est lui-même un projet Quarto de type `book` (voir `_quarto.yml`).

9.6 Structure d'un fichier `.qmd`

````
---
title: "Mon rapport"
author: "Prénom Nom"
format: html
---

## Introduction

Texte en **Markdown**.

```{r}
summary(mtcars$mpg)
````

```
## Titres, listes, tableaux et liens

```markdown
## Titre niveau 2
- liste à puces
1. liste numérotée
[Lien interne](#sec-rapports)
```

Tableau Markdown :

Indicateur	Valeur
n	32
moyenne	20.09

9.7 Insertion de code R

```r
mean(mtcars$mpg)
```

```
#> [1] 20.09062
```

Options utiles dans l'en-tête du bloc : `echo`, `eval`, `warning`, `message`, `fig.width`, `fig.height`, `label`.

9.8 Affichage des résultats et graphiques

```r
library(ggplot2)
ggplot(mtcars, aes(wt, mpg)) +
  geom_point() +
  geom_smooth(method = "lm", se = FALSE) +
  theme_minimal()
```

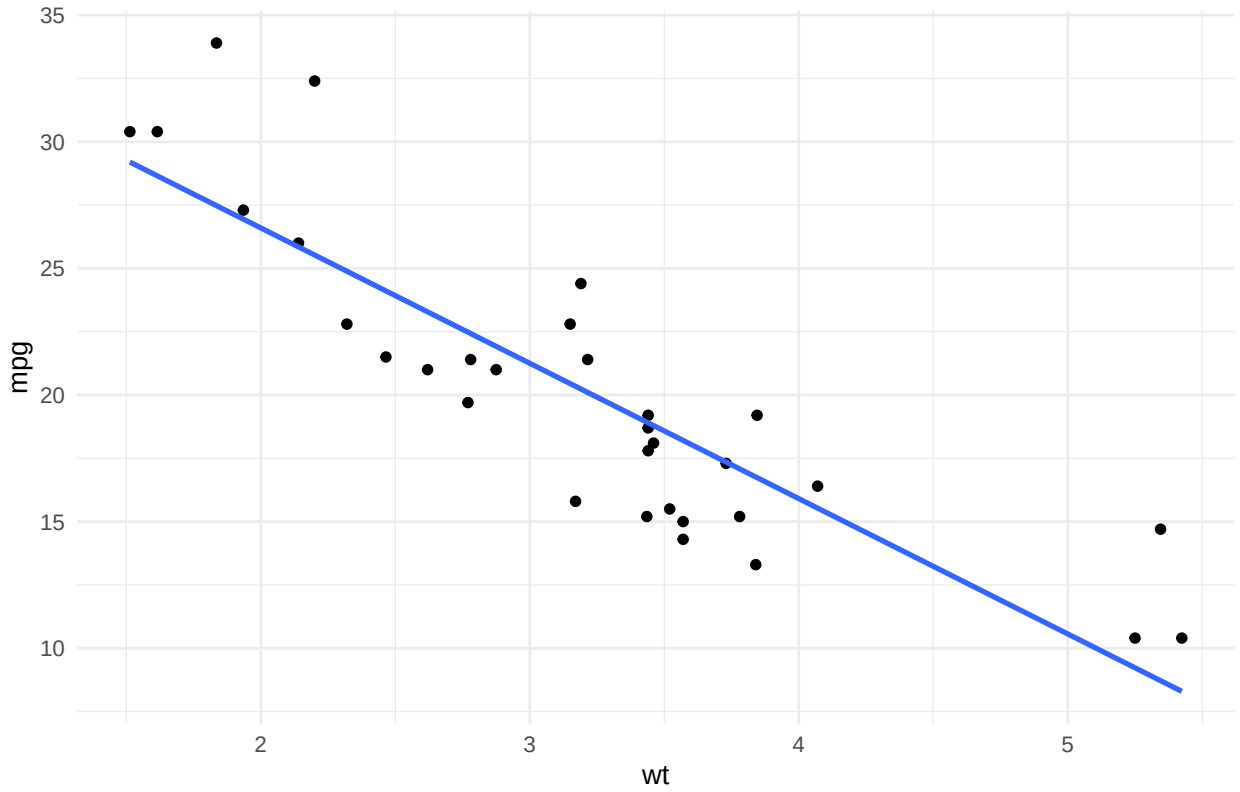


Figure 9.1: Consommation selon le poids (exemple de rapport)

9.9 Paramètres d'exécution

Option	Effet
`echo: false`	cache le code, montre le résultat
`eval: false`	montre le code sans l'exécuter
`include: false`	exécute mais n'affiche rien
`warning: false`	masque les warnings

Dans `_quarto.yml`, la section `execute:` fixe les défauts du livre.

9.10 Génération HTML, PDF, DOCX

Depuis le dossier `formation-r/` :

```
quarto preview                # HTML interactif
quarto render --to html
quarto render --to docx
quarto render --to pdf        # nécessite LaTeX / TinyTeX
```

9.10.1 Dépendances par format

Format	Besoin principal
HTML	Quarto + packages R utilisés
DOCX	Quarto (pandoc inclus)
PDF	Distribution LaTeX (TinyTeX recommandé)

```
tinytex::is_tinytex()
# sinon : tinytex::install_tinytex()
```

> Vérifier sans tout rebuild
>
> `quarto render chapters/08-rapports.qmd --to html` pour tester un fichier isolé (selon configuration).

9.11 Personnalisation de la mise en page

- HTML : `theme`, CSS (`styles/styles.css`), `toc`
- PDF : `geometry`, `documentclass`, `colorlinks`
- DOCX : `reference-doc` (modèle Word) si besoin

9.12 Bibliographie et citations

Les références sont dans `resources/references.bib`. Citation : `[@r4ds]` → (Wickham, Çetinkaya-Rundel, and Grolemund 2023).

9.13 Rapports statistiques automatisés

```
library(tidyverse)
ventes <- read_csv("data/ventes.csv", show_col_types = FALSE) |>
  filter(!is.na(region))

resume <- ventes |>
  summarise(
    n = n(),
    ca = sum(montant),
    ca_moyen = mean(montant)
  )
resume
```

```
#> # A tibble: 1 x 3
#>       n     ca ca_moyen
#>   <int>  <dbl>    <dbl>
#> 1   199 25877.     130.
```

En produisant le document chaque semaine sur des données mises à jour, le texte descriptif et les chiffres restent synchronisés.

9.14 R Markdown vs Quarto

Aspect	R Markdown	Quarto
Extension	`.Rmd`	`.qmd` (aussi `.Rmd`)
Moteur	`rmarkdown` / pandoc	Quarto CLI
Livres / sites	`bookdown`	intégré (`type: book`)
Multi-langage	possible	natif
Avenir Posit	maintenu	recommandé pour nouveaux projets

Les compétences Markdown + chunks se transfèrent presque à l'identique.

9.15 Publication et partage

Options courantes :

1. **Local** : ouvrir `outputs/book/index.html`
2. **GitHub Pages** : publier le dossier HTML généré
3. **Posit Connect / serveur interne** : déploiement organisationnel
4. Envoyer le **DOCX** ou **PDF** par e-mail

Exemple GitHub Pages (facultatif) :

```
git init
git add .
git commit -m "Livre formation R"
# Configurer GitHub Actions ou pousser le site rendu
```

Ne publiez jamais de données personnelles non anonymisées.

9.16 Démonstration : mini-rapport inline

```
ca_total <- sum(ventes$montant)
n_ventes <- nrow(ventes)
```

Sur la période couverte, on observe **199** ventes pour un CA total de 2.58766×10^{4}. Ce paragraphe se met à jour à chaque rendu.

9.17 Erreurs fréquentes

Problème	Solution
PDF échoue	Installer TinyTeX / MiKTeX
Chemin fichier introuvable	`execute-dir: project` dans `_quarto.yml`
Package manquant au render	Installer puis relancer
YAML invalide	Indentation espaces, pas de tabs

9.18 Exercices

9.18.1 Niveau 1

1. Citez trois formats de sortie Quarto.
2. À quoi sert `echo: false` ?

9.18.2 Niveau 2

3. Créez un fichier `outputs/mon_premier_rapport.qmd` localement avec un titre, un chunk `summary(iris)` et rendez-le en HTML.
4. Ajoutez une citation `[@tidyverse]` dans un paragraphe.

9.18.3 Niveau 3

5. Automatisez un résumé CA par région (tableau + barres) dans un `.qmd`.
6. Documentez pour un collègue les prérequis PDF vs HTML.

9.19 Corrections (indicatif)

```
# Dans un .qmd :
# ---
# title: "CA par région"
# format: html
# ---
# ```{r}
# library(tidyverse)
# ventes <- read_csv("data/ventes.csv", show_col_types = FALSE)
# ventes |> count(region, wt = montant)
# ```
```

Pour le PDF : installer TinyTeX, vérifier `quarto render --to pdf`, lire le log LaTeX en cas d'erreur.

9.20 Points clés

- Une source Quarto = analyses + narration synchronisées.
- HTML pour le web, DOCX pour relecture Word, PDF pour archivage imprimé.
- La bibliographie et les options d'exécution professionnalisent le rendu.

9.21 Ressources

- https://quarto.org/docs/guide/
- Posit PBC (2024), Wickham, Çetinkaya-Rundel, and Grolemund (2023)

9.22 Suite

Chapter 10 : rendre l'analyse interactive avec Shiny.

Part IX

Partie IX — Applications Shiny

Chapter 10

Applications interactives avec Shiny

10.1 Introduction

Shiny permet de créer des applications web interactives entièrement en R : l'utilisateur manipule des entrées (listes, curseurs, filtres) et voit statistiques et graphiques se mettre à jour. Ce chapitre couvre la structure UI/serveur, la réactivité, la validation, un mini tableau de bord et l'application fournie dans `shiny-app/`.

10.2 Objectifs pédagogiques

- Expliquer l'architecture d'une app Shiny
- Créer entrées, sorties et réactivité
- Filtrer des données et afficher graphiques / tables
- Valider les entrées et organiser le code
- Lancer l'application locale du projet

10.3 Prérequis

Chapter 7 et bases de 2. Package `shiny` installé.

```
shiny_ok <- requireNamespace("shiny", quietly = TRUE)
message("shiny disponible : ", shiny_ok)
```

> ⚠ Exécution dans le livre
>
> Les applications Shiny se lancent dans une session interactive. Les chunks ci-dessous sont surtout **illustratifs** (`eval=FALSE`) ; utilisez `shiny-app/app.R` pour tester réellement.

10.4 Présentation de Shiny

Shiny = interface (HTML/CSS/JS générés) + logique R. Idéal pour explorateurs de données, calculateurs et tableaux de bord internes.

10.5 Structure d'une application

Deux styles :

1. Fichiers `ui.R` + `server.R`
2. Fichier unique `app.R` (recommandé pour débuter)

```
library(shiny)

ui <- fluidPage(
  titlePanel("Titre"),
  sidebarLayout(
    sidebarPanel(/* entrées */),
    mainPanel(/* sorties */)
  )
)

server <- function(input, output, session) {
  # logique
}

shinyApp(ui, server)
```

10.6 Interface utilisateur et serveur

- **UI** : ce que l'utilisateur voit et manipule (`inputId`)
- **Serveur** : calcule `output` à partir de `input` via des fonctions `render*`

10.7 Entrées utilisateur

Fonction	Usage
`selectInput`	liste déroulante
`sliderInput`	curseur numérique
`checkboxInput`	case à cocher
`dateRangeInput`	période
`textInput`	texte libre

10.8 Sorties

`render*`	`*Output`
`renderPlot`	`plotOutput`
`renderTable` / `renderDataTable`	`tableOutput` / `dataTableOutput`
`renderText` / `renderPrint`	`textOutput` / `verbatimTextOutput`

10.9 Réactivité

```
donnees_filtrees <- reactive({
  req(input$region)
  ventes |> filter(region == input$region)
})

output$resume <- renderPrint({
```

```
  summary(donnees_filtrees()$montant)
})
```

- `reactive()` : expression mise en cache, recalculée si dépendances changent
- `req()` : attend qu'une entrée soit disponible
- `observe()` / `observeEvent()` : effets de bord (pas de valeur de retour)

10.10 Filtres et graphiques interactifs

Voir l'application complète ci-dessous : choix du jeu de données, filtre, histogramme et résumé.

10.11 Validation des entrées

```
output$plot <- renderPlot({
  validate(
    need(input$variable != "", "Choisissez une variable")
  )
  # ...
})
```

10.12 Organisation du code

```
shiny-app/
   app.R           # UI + server
   R/              # fonctions auxiliaires (optionnel)
   data/           # ou accès à ../data
```

Pour la production : modules Shiny, tests, logs.

10.13 Lancement local

```
shiny::runApp("shiny-app")
# ou ouvrir app.R dans RStudio et cliquer Run App
```

```
Rscript -e "shiny::runApp('shiny-app', launch.browser = TRUE)"
```

10.14 Tableau de bord simple — application du projet

L'application `shiny-app/app.R` permet de :

1. Choisir un jeu : ventes, démographie ou météo
2. Filtrer une catégorie (région / ville / mois)
3. Afficher résumé numérique + histogramme + aperçu tabulaire

10.15 Introduction au déploiement

Cible	Idée
Local	`runApp`

Cible	Idée
shinyapps.io	compte Posit, `rsconnect::deployApp`
Serveur Docker / Posit Connect	IT interne

Ne déployez pas de données sensibles sans contrôle d'accès.

10.16 Exemple minimal exécutable (concept)

```
library(shiny)
library(ggplot2)

ui <- fluidPage(
  titlePanel("Histo mtcars"),
  selectInput("var", "Variable", choices = names(mtcars)),
  plotOutput("hist")
)

server <- function(input, output, session) {
  output$hist <- renderPlot({
    ggplot(mtcars, aes(.data[[input$var]])) +
      geom_histogram(bins = 15, fill = "#1a5276", color = "white") +
      theme_minimal()
  })
}

# shinyApp(ui, server)
```

10.17 Erreurs fréquentes

Erreur	Cause	Remède
Sortie vide	Oubli de `output$` / `render*`	Vérifier les IDs identiques UI/serveur
Non-réactif	Lecture `input` hors reactive	Envelopper dans `reactive`/`render`
Chemin data	Mauvais working directory	Chemins relatifs à `app.R`
App qui ne démarre pas	Package manquant	`install.packages("shiny")`

10.18 Exercices

10.18.1 Niveau 1

1. Quelle différence entre UI et serveur ?
2. Citez deux `*Input` et deux `*Output`.

10.18.2 Niveau 2

3. Lancez `shiny-app` et changez de jeu de données.
4. Modifiez le nombre de bins de l'histogramme (ajoutez un `sliderInput`).

10.18.3 Niveau 3

5. Ajoutez un `downloadButton` pour exporter le tableau filtré en CSV.
6. Factorisez le code de filtrage dans une fonction `filtrer_donnees()`.

10.19 Corrections (indications)

```
# sliderInput("bins", "Nombre de classes", min = 5, max = 40, value = 15)
# geom_histogram(bins = input$bins, ...)

# downloadHandler(
#   filename = "extrait.csv",
#   content = function(file) readr::write_csv(donnees_filtrees(), file)
# )
```

10.20 Points clés

- Shiny = UI + serveur + réactivité.
- Gardez les calculs dans des `reactive` clairs.
- Testez localement avant tout déploiement.

10.21 Ressources

- Chang et al. (2024) — https://shiny.posit.co/
- Application du livre : dossier `shiny-app/`

10.22 Suite

Chapter 11 : projets de synthèse et évaluations.

Part X

Partie X — Exercices et projets

Chapter 11

Projets de synthèse et évaluations

11.1 Introduction

Les projets suivants consolident l'ensemble du parcours. Chaque projet comporte : énoncé, objectifs, données, étapes, résultats attendus et **solution détaillée**. Faites l'exercice avant de lire la correction.

11.2 Projet 1 — Analyse des ventes

11.2.1 Énoncé

Vous êtes analyste pour une petite entreprise de papeterie. On vous confie `data/ventes.csv`.

11.2.2 Objectifs

- Nettoyer les données (NA, doublons)
- Calculer le CA par région et par produit
- Produire 2 graphiques commentés
- Rédiger 5–10 lignes de conclusions

11.2.3 Étapes recommandées

1. Import + diagnostic
2. `distinct()`, filtre NA
3. Agrégats dplyr
4. Barres CA région + boxplot montants produit
5. Synthèse écrite

11.2.4 Résultats attendus

- Table CA par région triée
- Au moins un outlier ou NA documenté
- Conclusions actionnables (ex. : région / produit à suivre)

11.2.5 Solution

```
library(tidyverse)
```

```
ventes <- read_csv("data/ventes.csv", show_col_types = FALSE)
cat("Doublons :", sum(duplicated(ventes)), "\n")
```

```
#> Doublons : 1
```

```
cat("NA region :", sum(is.na(ventes$region)), "\n")
```

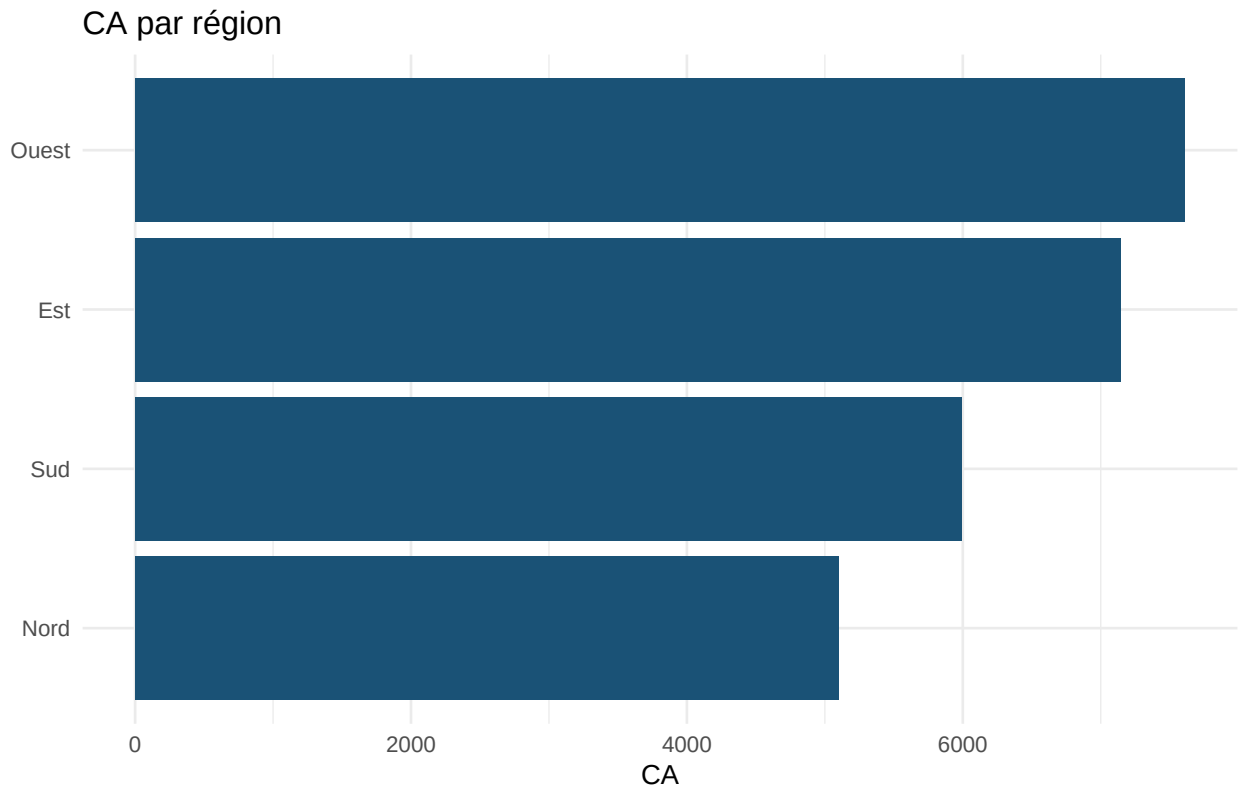
```
#> NA region : 2
```

```
ventes_c <- ventes |>
  distinct() |>
  filter(!is.na(region), !is.na(quantite))

ca_region <- ventes_c |>
  group_by(region) |>
  summarise(ca = sum(montant), n = n(), .groups = "drop") |>
  arrange(desc(ca))
ca_region
```

```
#> # A tibble: 4 x 3
#>   region    ca     n
#>   <chr>  <dbl> <int>
#> 1 Ouest  7610.    58
#> 2 Est    7141.    50
#> 3 Sud    5995.    42
#> 4 Nord   5096.    47
```

```
ggplot(ca_region, aes(reorder(region, ca), ca)) +
  geom_col(fill = "#1a5276") +
  coord_flip() +
  labs(title = "CA par région", x = NULL, y = "CA") +
  theme_minimal()
```



11.3 Projet 2 — Données démographiques

11.3.1 Énoncé

Explorez `data/demographie.csv` : structure de la population, revenus, aberrant âge/revenu.

11.3.2 Objectifs

- Détecter et traiter les valeurs aberrantes
- Résumer âge et revenu par ville et sexe
- Tableau croisé sexe × niveau d'études

11.3.3 Solution

```
demo <- read_csv("data/demographie.csv", show_col_types = FALSE)

# Aberrants évidents
demo |> filter(age > 100 | revenu_mensuel > 20000)
```

```
#> # A tibble: 2 x 6
#>      id   age sexe  ville revenu_mensuel niveau_etudes
#>   <dbl> <dbl> <chr> <chr>          <dbl> <chr>
#> 1     3   150 Femme Paris           1104 Bac
#> 2     7    54 Femme Paris          50000 Master
```

```
demo_c <- demo |>
  filter(age >= 18, age <= 100, revenu_mensuel < 20000)
```

```
demo_c |>
  group_by(ville, sexe) |>
  summarise(
    n = n(),
    age_moy = mean(age),
    revenu_med = median(revenu_mensuel),
    .groups = "drop"
  ) |>
  head(10)
```

```
#> # A tibble: 10 x 5
#>    ville     sexe      n age_moy revenu_med
#>    <chr>     <chr> <int>   <dbl>      <dbl>
#>  1 Lille     Femme    14    47.9      2134.
#>  2 Lille     Homme    10    51.5      1751
#>  3 Lyon      Femme    20    48.2      2386
#>  4 Lyon      Homme    20    41.1      2152
#>  5 Marseille Femme    16    52.3      2039
#>  6 Marseille Homme    18    46.2      2118.
#>  7 Paris     Femme     8    53.4      2038.
#>  8 Paris     Homme    16    41.1      2401
#>  9 Toulouse  Femme    11    53.4      2289
#> 10 Toulouse  Homme    15    39.5      2225
```

```
demo_c |>
  count(sexe, niveau_etudes) |>
  pivot_wider(names_from = sexe, values_from = n, values_fill = 0)
```

```
#> # A tibble: 5 x 3
#>   niveau_etudes Femme Homme
#>   <chr>         <int> <int>
#> 1 Autre            10     9
#> 2 Bac              18    20
#> 3 Doctorat          3     5
#> 4 Licence          26    33
#> 5 Master           12    12
```

11.4 Projet 3 — Météo

11.4.1 Énoncé

À partir de `data/meteo.csv`, décrivez la saisonnalité température / précipitations.

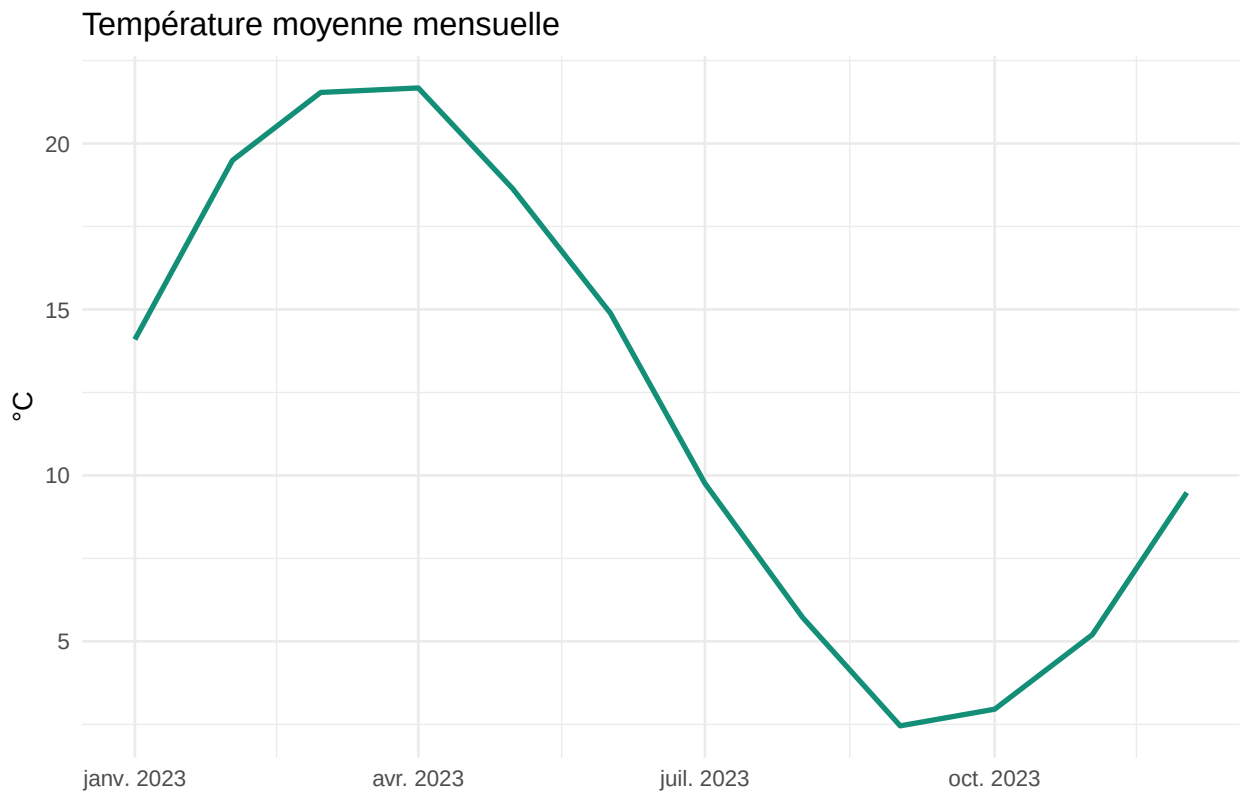
11.4.2 Solution

```
meteo <- read_csv("data/meteo.csv", show_col_types = FALSE) |>
  mutate(date = as.Date(date), mois = floor_date(date, "month"))

mensuel <- meteo |>
  group_by(mois) |>
  summarise(
    temp = mean(temperature_c),
    pluie = sum(precipitations_mm),
    .groups = "drop"
```

```
  )

ggplot(mensuel, aes(mois, temp)) +
  geom_line(color = "#148f77", linewidth = 1) +
  labs(title = "Température moyenne mensuelle", x = NULL, y = "°C") +
  theme_minimal()
```




```
cor(meteo$temperature_c, meteo$humidite_pct)
```

```
#> [1] -0.007141184
```

11.5 Projet 4 — Régression

11.5.1 Énoncé

Avec `data/etudes_notes.csv`, modélisez la note selon les heures d'étude et la filière.

11.5.2 Solution

```
library(broom)
etudes <- read_csv("data/etudes_notes.csv", show_col_types = FALSE)

mod <- lm(note_examen ~ heures_etude + filiere, data = etudes)
tidy(mod)
```

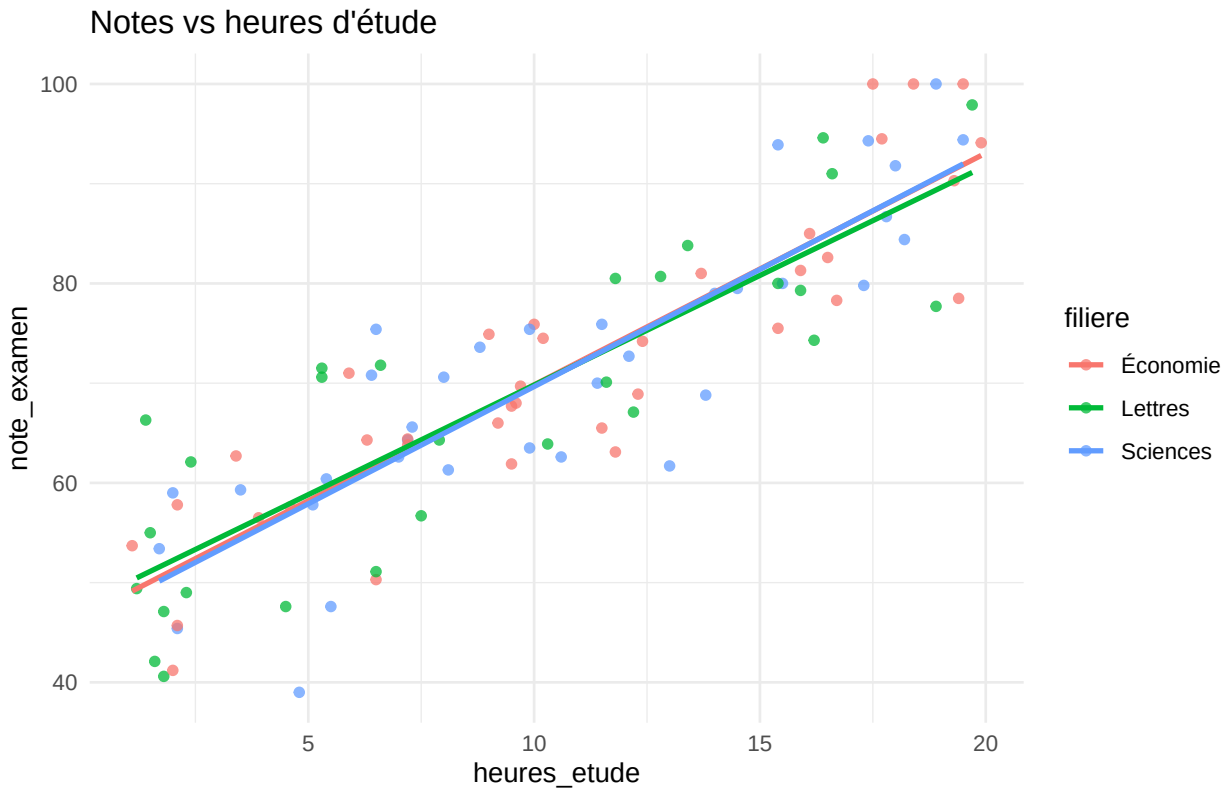
```
#> # A tibble: 4 x 5
#>   term            estimate std.error statistic  p.value
```

```
#>   <chr>              <dbl>     <dbl>     <dbl>    <dbl>
#> 1 (Intercept)      46.9       1.89    24.8    1.50e-43
#> 2 heures_etude      2.29      0.132   17.3    2.81e-31
#> 3 filiereLettres    0.0554    1.88     0.0294 9.77e- 1
#> 4 filiereSciences  -0.172     1.76    -0.0982 9.22e- 1
```

```
glance(mod) |> select(r.squared, adj.r.squared, sigma)
```

```
#> # A tibble: 1 x 3
#>   r.squared adj.r.squared sigma
#>       <dbl>         <dbl> <dbl>
#> 1     0.761         0.754  7.45
```

```
ggplot(etudes, aes(heures_etude, note_examen, color = filiere)) +
  geom_point(alpha = 0.75) +
  geom_smooth(method = "lm", se = FALSE) +
  labs(title = "Notes vs heures d'étude") +
  theme_minimal()
```



Rappel : une association positive n'implique pas que « forcer » des heures supplémentaires produira le même effet hors échantillon (causalité, motivation, etc.).

11.6 Projet 5 — Tableau de bord Shiny

11.6.1 Énoncé

Utilisez ou étendez `shiny-app/app.R` : au moins un jeu de données, un filtre, un résumé et un graphique.

11.6.2 Résultats attendus

- App qui démarre sans erreur
- Filtre opérationnel
- Histogramme réactif

11.6.3 Solution

Voir `shiny-app/app.R`. Lancement :

```
shiny::runApp("shiny-app")
```

11.7 Projet final — Étude complète

11.7.1 Énoncé

Réalisez une étude de A à Z au choix (ventes, démo ou météo) :

1. Import et contrôle qualité
2. Préparation documentée
3. Statistiques descriptives
4. Au moins 3 graphiques
5. Une analyse d'association ou régression si pertinent
6. Rapport Quarto (HTML) de 3–6 pages
7. Conclusions et limites

11.7.2 Grille d'évaluation (suggestion)

Critère	Points
Reproductibilité (chemins, seed, packages)	20
Qualité du nettoyage	20
Pertinence des analyses	20
Graphiques lisibles	15
Interprétation et limites	15
Clarté du rapport	10

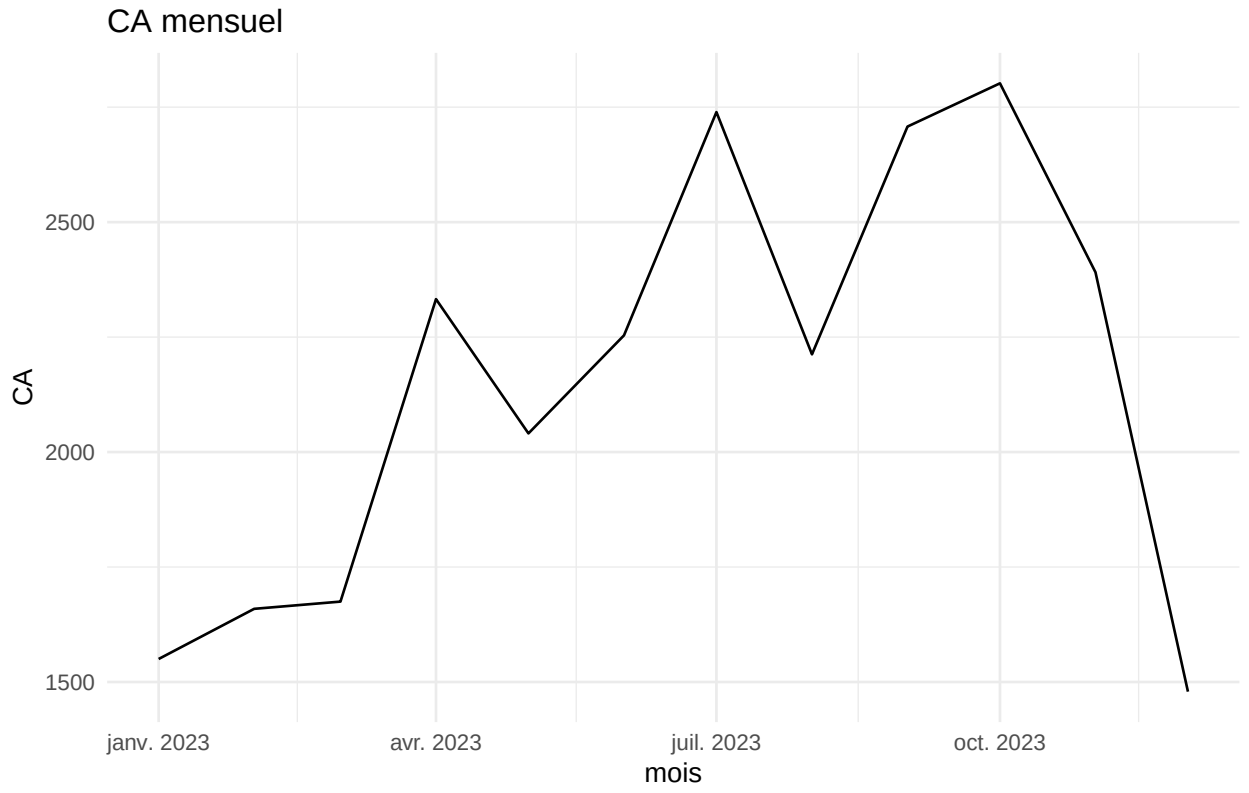
11.7.3 Solution type (ventes — squelette)

```
# 1-2. Préparation
ventes_final <- read_csv("data/ventes.csv", show_col_types = FALSE) |>
  distinct() |>
  filter(!is.na(region), !is.na(quantite), montant > 0) |>
  mutate(date = as.Date(date), mois = floor_date(date, "month"))

# 3. Descriptif
ventes_final |>
  summarise(n = n(), ca = sum(montant), moy = mean(montant))
```

```
#> # A tibble: 1 x 3
#>       n     ca   moy
#>   <int>  <dbl> <dbl>
#> 1   197 25842.  131.
```

```
# 4. Graphique
ventes_final |>
  count(mois, wt = montant) |>
  ggplot(aes(mois, n)) +
  geom_line() +
  labs(title = "CA mensuel", y = "CA") +
  theme_minimal()
```




```
# 5. Association quantité / montant
cor(ventes_final$quantite, ventes_final$montant)
```

```
#> [1] 0.6680832
```

Rédigez ensuite le rapport `.qmd` en suivant Chapter 9.

11.8 Quiz de révision rapide

1. Quelle fonction dplyr filtre les lignes ? → `filter`
2. Corrélation prouve-t-elle la causalité ? → Non
3. Commande pour rendre le livre en HTML ? → `quarto render --to html`
4. Structure Shiny minimale ? → `ui` + `server` + `shinyApp`
5. Pourquoi `set.seed` ? → Reproductibilité du aléatoire

11.9 Points clés

- Un projet réussi est **reproductible**, **interprété** et **limité** explicitement.
- Les corrections sont des modèles, pas l'unique réponse acceptable.

11.10 Suite

Chapter 14 : glossaire, aide-mémoires et références.

Chapter 12

Exercices de synthèse transversaux

Ces exercices mêlent plusieurs chapitres. Les corrections détaillées sont dans Chapter 13.

12.1 Exercice S1 — Pipeline ventes (niveaux 1–3)

1. Importez `data/ventes.csv`.
2. Supprimez les doublons et les lignes sans région.
3. Créez `mois` et calculez le CA mensuel.
4. Affichez un graphique en barres des 5 produits au CA le plus élevé.
5. Calculez la corrélation `quantite`–`montant` et commentez en une phrase.

12.2 Exercice S2 — Qualité démographie

1. Listez les observations avec `age > 100` ou `revenu_mensuel > 20000`.
2. Proposez une règle de nettoyage écrite (pas seulement du code).
3. Après nettoyage, comparez revenu médian par `niveau_etudes`.

12.3 Exercice S3 — Mini-rapport Quarto

Créez `outputs/mini_rapport_meteo.qmd` qui :

- Charge `data/meteo.csv`
- Affiche température moyenne annuelle
- Trace la courbe mensuelle
- Cite une limite de l'analyse

12.4 Exercice S4 — Shiny

Dans `shiny-app`, ajoutez un texte affichant le nombre de lignes filtrées au-dessus de l'histogramme.

12.5 Quiz final (10 questions)

1. Que retourne `typeof(1L)` ?
2. Quelle fonction tidyverse lit un CSV ?
3. `filter(is.na(x))` sélectionne-t-il les NA ?
4. Quelle geom pour une série temporelle simple ?

5. Que mesure l'écart-type ?
6. `cor = 0` implique-t-il l'indépendance totale ?
7. Quelle commande rend un PDF Quarto ?
8. UI Shiny : `plotOutput` doit être associé à quel `render*` ?
9. Pourquoi garder `data/brut/` intact ?
10. Corrélation élevée causalité ? (oui/non)

Chapter 13

Corrections des exercices de synthèse

> **Méthode**
>
> Tentez d'abord Chapter 12 sans regarder cette page.

13.1 Correction S1

```
library(tidyverse)

ventes <- read_csv("data/ventes.csv", show_col_types = FALSE) |>
  distinct() |>
  filter(!is.na(region)) |>
  mutate(date = as.Date(date), mois = floor_date(date, "month"))

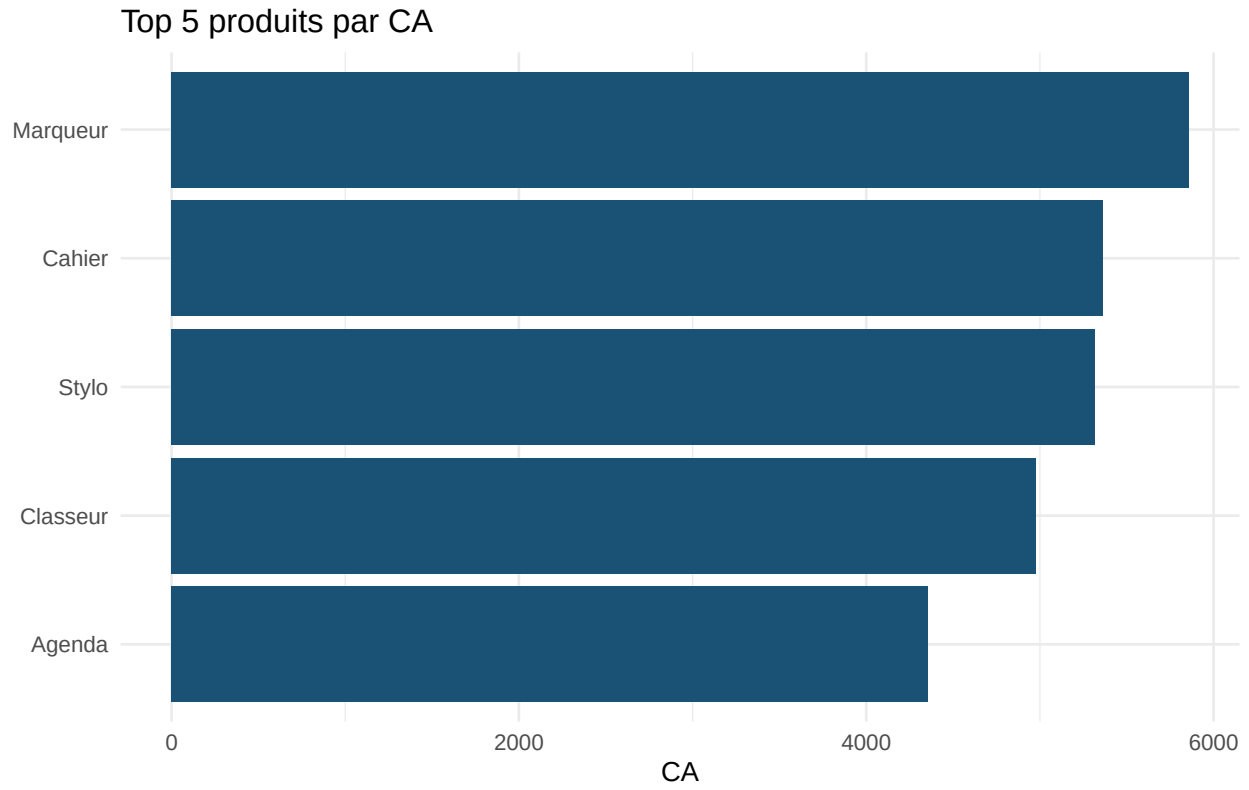
ca_mois <- ventes |>
  group_by(mois) |>
  summarise(ca = sum(montant), .groups = "drop")
head(ca_mois)
```

```
#> # A tibble: 6 x 2
#>   mois          ca
#>   <date>     <dbl>
#> 1 2023-01-01 1550.
#> 2 2023-02-01 1682.
#> 3 2023-03-01 1675.
#> 4 2023-04-01 2333.
#> 5 2023-05-01 2041.
#> 6 2023-06-01 2254.
```

```
top5 <- ventes |>
  group_by(produit) |>
  summarise(ca = sum(montant), .groups = "drop") |>
  slice_max(ca, n = 5)

ggplot(top5, aes(reorder(produit, ca), ca)) +
  geom_col(fill = "#1a5276") +
  coord_flip() +
```

```
  labs(title = "Top 5 produits par CA", x = NULL, y = "CA") +
  theme_minimal()
```




```
cor(ventes$quantite, ventes$montant)
```

```
#> [1] NA
```

```
# Phrase : association positive - davantage d'unités tend à augmenter le montant, sans impliquer un méca
```

13.2 Correction S2

```
demo <- read_csv("data/demographie.csv", show_col_types = FALSE)
demo |> filter(age > 100 | revenu_mensuel > 20000)
```

```
#> # A tibble: 2 x 6
#>      id   age sexe  ville revenu_mensuel niveau_etudes
#>   <dbl> <dbl> <chr> <chr>          <dbl> <chr>
#> 1     3   150 Femme Paris           1104 Bac
#> 2     7    54 Femme Paris          50000 Master
```

```
# Règle : exclure âge hors [18, 100] et revenus > 20000 comme erreurs de saisie
# documentées pour ce jeu pédagogique synthétique.

demo_c <- demo |>
  filter(age >= 18, age <= 100, revenu_mensuel <= 20000)

demo_c |>
```

```
  group_by(niveau_etudes) |>
  summarise(revenu_median = median(revenu_mensuel), n = n(), .groups = "drop") |>
  arrange(desc(revenu_median))
```

```
#> # A tibble: 5 x 3
#>   niveau_etudes revenu_median     n
#>   <chr>                 <dbl> <int>
#> 1 Doctorat              2334.     8
#> 2 Licence               2307     59
#> 3 Autre                 2168     19
#> 4 Master                2168.    24
#> 5 Bac                   2068.    38
```

13.3 Correction S3 (contenu attendu)

````
---
title: "Mini-rapport météo"
format: html
---

```{r}
library(tidyverse)
meteo <- read_csv("data/meteo.csv", show_col_types = FALSE) |>
  mutate(date = as.Date(date))
mean(meteo$temperature_c)
```
````

Limite : une seule ville simulée, pas de généralisation nationale.

````
## Correction S4

Dans `server`, ajouter par exemple :

```r
output$n_lignes <- renderText({
  paste("Lignes filtrées :", nrow(donnees()$data))
})
```
````

Et dans `ui` : `textOutput("n_lignes")` au-dessus de `plotOutput("histo")`.

13.4 Réponses du quiz final

1. `"integer"`
2. `readr::read_csv` (ou `read.csv`)
3. Oui, les lignes où `x` est NA
4. `geom_line` (souvent avec date en x)
5. La dispersion typique autour de la moyenne
6. Non (dépendance non linéaire possible)
7. `quarto render --to pdf`
8. `renderPlot`
9. Traçabilité et possibilité de recommencer le nettoyage
10. **Non**

Part XI

Partie XI — Références et annexes

Chapter 14

Références, annexes et navigation

14.1 Introduction

Cette partie regroupe les ressources de révision : glossaire détaillé (Appendix A), aide-mémoires (Appendix B), guide d'erreurs, références officielles et pistes de publication.

14.2 Guide des erreurs fréquentes

Message / symptôme	Cause fréquente	Solution
`object 'x' not found`	Objet non créé / typo	`ls()`, revérifier le chunk
`could not find function`	Package non chargé	`library(...)`
`Cannot open file`	Mauvais chemin	`getwd()`, projet RStudio
`unexpected ')'`	Parenthese en trop/manquant	Repérer la ligne du `+` console
PDF Quarto échoue	Pas de LaTeX	`tinytex::install_tinytex()`
Jointure explosée	Clés dupliquées	`count(id)` avant join
Graphique vide	Données filtrées à 0 lignes	`nrow()` avant ggplot
Facteur → nombres bizarres	Conversion directe	`as.numeric(as.character(f))`

14.3 Tableau des fonctions R essentielles

Fonction	Rôle
`c()`	Créer un vecteur
`data.frame()` / `tibble()`	Tableau
`mean`, `median`, `sd`, `summary`	Descriptifs
`read_csv` / `write_csv`	Import / export CSV
`filter`, `select`, `mutate`	Transformation
`group_by`, `summarise`	Agrégation
`left_join`	Jointure
`ggplot` + `geom_*`	Visualisation
`lm`, `t.test`, `cor.test`	Modèles / tests
`set.seed`	Reproductibilité

14.4 Publication GitHub Pages (facultatif)

1. Créer un dépôt Git et pousser les **sources**
2. Rendre le livre : `quarto render --to html`
3. Publier le contenu de `outputs/book/` (Actions ou branche `gh-pages`)
4. Documenter l'URL dans le README

Le livre reste consultable **localement** sans compte externe : ouvrez `outputs/book/index.html`.

14.5 Mise à jour du projet

```
# Régénérer les données synthétiques
Rscript R/generer_donnees.R

# Vérifier / installer les packages
Rscript R/packages_requis.R

# Rebuild
quarto render --to html
```

14.6 Index de navigation

Thème	Section
Installation	Section 2.7
Fondamentaux R	Chapter 3
Import	1
Nettoyage	2
Descriptives	3
Graphiques	Chapter 7
Modélisation	Chapter 8
Quarto	Chapter 9
Shiny	Chapter 10
Projets	Chapter 11
Glossaire	Appendix A
Aide-mémoire	Appendix B

14.7 Références officielles

- R : https://www.r-project.org/ (R Core Team 2024)
- Quarto : https://quarto.org/ (Posit PBC 2024)
- tidyverse : https://www.tidyverse.org/ (Wickham et al. 2019)
- ggplot2 : https://ggplot2.tidyverse.org/ (Wickham 2016)
- Shiny : https://shiny.posit.co/ (Chang et al. 2024)
- R for Data Science : https://r4ds.hadley.nz/ (Wickham, Çetinkaya-Rundel, and Grolemund 2023)

La bibliographie formatée figure en fin d'ouvrage (chapitre Références).

Références bibliographiques

Çetinkaya-Rundel, Mine, and Johanna Hardin. 2023. *Introduction to Modern Statistics*. https://openintro-ims.netlify.app/.

Chang, Winston, Joe Cheng, JJ Allaire, Carson Sievert, Barret Schloerke, Yihui Xie, Jeff Allen, Jonathan McPherson, Alan Dipert, and Barbara Borges. 2024. *Shiny: Web Application Framework for r*. https://shiny.posit.co/.

Posit PBC. 2024. *Quarto*. https://quarto.org/.

R Core Team. 2024. *R: A Language and Environment for Statistical Computing*. Vienna, Austria: R Foundation for Statistical Computing. https://www.R-project.org/.

Wickham, Hadley. 2016. *Ggplot2: Elegant Graphics for Data Analysis*. Springer-Verlag New York. https://ggplot2.tidyverse.org.

Wickham, Hadley, Mara Averick, Jennifer Bryan, Winston Chang, Lucy D'Agostino McGowan, Romain François, Garrett Grolemund, et al. 2019. "Welcome to the tidyverse." *Journal of Open Source Software* 4 (43): 1686. https://doi.org/10.21105/joss.01686.

Wickham, Hadley, Mine Çetinkaya-Rundel, and Garrett Grolemund. 2023. *R for Data Science*. 2nd ed. O'Reilly. https://r4ds.hadley.nz/.

Appendix A

Glossaire

Affectation
Stockage d'une valeur dans un objet (`x <- 3`).

API
Interface permettant d'échanger des données entre systèmes (souvent en JSON).

Boîte à moustaches
Graphique résumant médiane, quartiles et extrêmes.

Causalité
Relation de cause à effet ; non démontrée par la seule corrélation.

Chunk
Bloc de code exécutable dans Quarto / R Markdown.

Corrélation
Mesure d'association (souvent linéaire) entre deux variables quantitatives.

Data frame / tibble
Tableau où chaque colonne est une variable et chaque ligne une observation.

dplyr
Package de manipulation tabulaire (verbes `filter`, `mutate`, etc.).

Échantillon
Sous-ensemble observé d'une population.

Écart-type
Mesure de dispersion autour de la moyenne.

EDA
Exploratory Data Analysis : exploration descriptive et visuelle.

Facteur
Type R pour variables catégorielles (`factor`).

ggplot2
Package de visualisation fondé sur une grammaire des graphiques.

IC (intervalle de confiance)
Procédure quantifiant l'incertitude d'un estimateur.

Inférence
Généralisation d'un échantillon vers une population.

Jointure
Fusion de tables via une clé (`left_join`, etc.).

Markdown
Langage de balisage léger pour le texte structuré.

Médiane
Valeur centrale d'une série ordonnée.

NA
Valeur manquante en R.

Outlier / aberrant
Observation atypique ; à examiner avant traitement.

Package
Extension installable de R.

p-valeur
Sous (H_0), probabilité d'obtenir un résultat au moins aussi extrême.

Pipeline
Enchaînement d'opérations (`|>` ou `%>%`).

Population
Ensemble complet ciblé par l'étude.

Quarto
Système de publication de documents reproductibles.

R^{2}
Part de variance expliquée par un modèle linéaire.

Régression linéaire
Modèle reliant une réponse quantitative à des prédicteurs.

Reproductibilité
Capacité à refaire l'analyse et obtenir le même résultat.

Shiny
Framework R d'applications web interactives.

Tidy data
Données « ordonnées » : une variable = une colonne, une observation = une ligne.

tidyverse
Collection de packages cohérents pour la science des données.

Variable qualitative
Catégorie (sexe, région).

Variable quantitative
Mesure numérique (âge, montant).

Vecteur
Suite ordonnée de valeurs de même type en R.

YAML
En-tête de configuration d'un document Quarto.

Appendix B

Aide-mémoires

B.1 Manipulation de données (dplyr / tidyr)

```
df |>
  filter(condition) |>
  select(col1, col2) |>
  mutate(nouvelle = col1 * 2) |>
  arrange(desc(nouvelle)) |>
  group_by(categorie) |>
  summarise(n = n(), moy = mean(nouvelle), .groups = "drop")

left_join(a, b, by = "id")
pivot_longer(df, cols = -id, names_to = "nom", values_to = "val")
pivot_wider(df, names_from = nom, values_from = val)
```

B.2 Import / export

```
readr::read_csv("data/fichier.csv")
readxl::read_excel("data/fichier.xlsx", sheet = 1)
readr::write_csv(df, "outputs/sortie.csv")
```

B.3 ggplot2

```
ggplot(df, aes(x, y, color = g)) +
  geom_point() +
  geom_smooth(method = "lm") +
  facet_wrap(~g) +
  labs(title = "...", x = "...", y = "...") +
  theme_minimal()

# Barres / hist / box
geom_col() ; geom_bar() ; geom_histogram(bins = 20) ; geom_boxplot()
ggsave("outputs/fig.png", width = 7, height = 4, dpi = 150)
```

B.4 Statistiques

```r
summary(x); mean(x, na.rm = TRUE); median(x); sd(x); quantile(x)
table(x); prop.table(table(x))
cor(x, y); cor.test(x, y)
t.test(y ~ g, data = df)
mod <- lm(y ~ x1 + x2, data = df)
summary(mod); broom::tidy(mod); broom::augment(mod)
```

B.5 Quarto (CLI)

```
quarto preview
quarto render --to html
quarto render --to pdf
quarto render --to docx
quarto check
```

B.6 Shiny (minimal)

```r
ui <- fluidPage(selectInput("v", "Var", names(mtcars)), plotOutput("p"))
server <- function(input, output, session) {
  output$p <- renderPlot({ hist(mtcars[[input$v]]) })
}
shinyApp(ui, server)
```

B.7 stringr / lubridate (extraits)

```r
str_detect(x, "abc"); str_replace_all(x, " ", "_"); str_to_lower(x)
ymd("2023-01-15"); year(d); month(d); floor_date(d, "month")
```